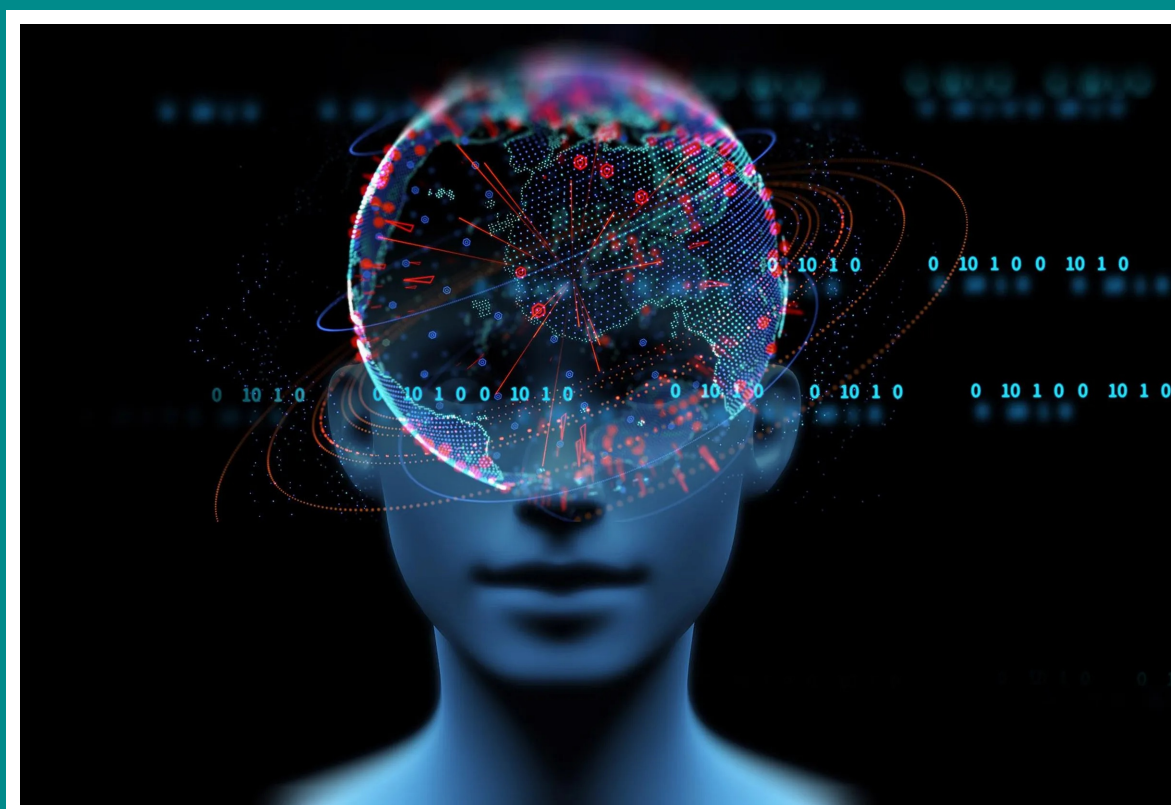


Mattia Robuschi Caprara

Basi di Dati



Prefazione

Questi appunti sono una trascrizione in LaTeX di alcuni appunti gentilmente concessi da alcuni miei compagni di corso (Alessia Colla e Luigi Quitadamo), quindi non mi prendo alcun merito.

Contents

1	Generalità	6
1.1	Test di Turing	6
1.2	La stanza cinese	6
1.3	Al forte vs Al debole (o narrow vs general)	6
1.4	Ordine del termine	6
1.4.1	Gli inizi (1952-1959)	6
1.4.2	1968	7
1.4.3	1969 - 1979	7
1.4.4	1995 - 2000: Nascita del Web e delle ontologie	7
1.4.5	2011 - present	7
1.5	Il gioco In IA	7
1.6	Ragionamento Deduttivo vs Ragionamento Induttivo	8
1.7	Conoscenza e comportamento razionale	8
1.8	Approccio simbolico vs approccio subsimbolico	8
1.8.1	Approccio simbolico	9
1.8.1.1	Logica	9
1.8.2	Approccio sub-simbolico	9
1.8.2.1	Ragionamento e apprendimento	9
1.9	Cos'è la teoria del learning	10
1.9.1	Inferenze	10
1.9.2	Deep Learning e Reti Neurali Artificiali (ANN)	11
1.10	Black Box AI & Explainable AI	11
1.10.1	Indagare un sistema di AI	11
1.10.1.1	Esempio: Un oggetto cade a terra e si rompe.	12
2	Agenti Intelligenti	12
2.1	Introduzione	12
2.1.1	Esempio	13
2.2	Agenti Intelligenti/razionale	13
2.3	Agenti e ambiente	14
2.3.1	Esempi	14
2.4	Proprietà dell'ambiente-problema	14
2.5	Struttura	15
2.5.1	Esempio: taxi	16
2.6	Tipi di rappresentazione dello stato agente	17
2.7	Espressività	17
3	Agenti Risolutori di Problemi	17
3.1	Introduzione	17
3.2	Risoluzione di problemi tramite ricerca	18
3.2.1	Esempio	18
3.3	Problemi ben definiti e soluzioni	19
3.4	Risoluzione di problemi	19
3.5	Formulazione del problema	19
3.6	Costo della ricerca	19
3.6.1	Esempio	19
3.7	Esempi di problemi - Problemi giocattolo	20
3.7.1	Il problema dell'8 puzzle	20
3.7.2	Il problema delle 8 regine	20
3.7.3	Il problema dei missionari e dei cannibali	20
3.8	Classi di problemi	21
3.8.1	A stato singolo (Search)	21

3.8.2	Problemi a stato multiplo (Search)	21
3.8.3	Problemi con imprevisti (Planning)	21
3.8.4	Problemi di esplorazione (Learning)	22
3.8.5	Problemi stati singoli vs stati multipli	22
3.8.5.1	Esempio	22
3.9	Ricerca le soluzioni	23
3.9.1	Cercare soluzioni	23
3.9.2	Algoritmo generale di ricerca	23
3.10	Valutazione delle strategie di ricerca	24
3.11	Strategie non informate	24
3.11.1	Ricerca in ampiezza (breadth-first search)	24
3.11.2	Ricerca a Costo Uniforme (Uniform Cost Search)	25
3.11.3	Ricerca in Profondità (Depth-First Search)	25
3.11.4	Ricerca Limitata in Profondità (Depth-Limited Search)	25
3.11.5	Ricerca Iterativa in Profondità (Iterative Deepening Search)	26
3.11.6	Ricerca Bidirezionale (Bidirectional Search)	26
3.12	Eliminazione della ripetizione di percorsi	26
3.13	Evitare ripetizioni di stati	26
3.14	Ricerca le soluzioni: Strategie informate	27
3.14.1	Metodi di ricerca informati ed esplorazione	27
3.14.2	Strategie informate	27
3.14.3	Ricerca lungo il Cammino Migliore (Best-First Search)	27
3.15	Ricerca lungo il cammino migliore	27
3.15.1	Greedy search (o ricerca golosa)	27
3.15.2	Algoritmo A* (ricerca a stella)	28
3.15.3	Algoritmo A* per grafi	29
3.15.4	A* Iterativo in Profondità (Iterative Deepening A* - IDA*)	29
3.15.5	SMA* (Simplified Memory-Bounded A*)	29
3.15.5.1	Esempio di ricerca con SMA*	30
3.15.5.2	Proprietà algoritmo SMA*	30
3.16	Scelta euristica	30
3.16.1	Generazione automatica di euristiche ammissibili attraverso il rilassamento dei problemi	31
4	Agenti Risolutori di Problemi - Ottimizzazione	32
4.1	Algoritmi di ricerca locale e problemi di ottimizzazione	32
4.1.1	Algoritmi di miglioramento iterativo	32
4.1.1.1	Struttura dei "vicini"	32
4.1.2	Ricerca in Salita (Hill Climbing)	33
4.1.3	Simulated Annealing	33
4.1.3.1	Esempio	33
4.1.4	Meta-euristiche	34
4.1.4.1	Meta-conoscenza	34
4.1.4.2	Meta-euristiche e scelta degli operatori	34
4.2	Problemi con vincoli	35
4.2.1	Formulazione dei problemi CSP	35
4.2.2	Tipi di vincoli	35
4.2.3	Ricerca in problemi CSP	35
4.2.4	Strategie di ricerca	36
4.2.4.1	Problemi nell'uso del backtracking	36
4.2.5	Scelta delle variabili	36
4.2.6	Scelta dei valori	36
4.2.7	Propagazione di vincoli	36
4.2.7.1	Forward checking	36
4.2.7.2	Consistenza di nodo	37
4.2.7.3	Consistenza d'arco	37
4.2.8	Complessità di AC-3	37
4.2.9	La risoluzione di un CSP richiede ancora la ricerca	37
4.2.10	Backtracking "cronologico"	37
4.2.11	Backtracking "intelligente"	38

5	Agente Logico	38
5.1	Introduzione al ragionamento automatico logico con la logica proposizionale	38
5.1.1	Studio degli stati nascosti	38
5.1.2	Sistematica	39
5.1.3	Basi di Conoscenza (knowledge Bases)	39
5.1.4	Agente logico basato su una KB	39
5.1.5	Logica moderna o logica matematica	40
5.2	Logica	41
5.2.1	Logica: Linguaggio	42
5.2.2	Ragionamento logico: Implicazione	42
5.2.3	Models	42
5.2.4	Inferenza nel modello Wumpus	42
5.2.4.1	Model Checking	42
5.2.5	Inferenza	43
5.2.6	Problema del “grounding”	43
5.2.7	Logica classica	43
5.2.7.1	Limiti	43
5.3	Logica proposizionale	43
5.3.1	Sintassi	44
5.3.2	Semantica	44
5.4	Inference by Theorem Proving	44
5.4.1	Inferenza e prova di teoremi	45
5.4.2	Proof methods	45
5.4.3	Monoliticità della logica	45
5.4.4	Proof by resolution	45
5.4.5	Logica proposizionale - Regole di inferenza	45
5.4.6	Conjunctive normal form (CNF): forma normale della KB	46
5.4.7	Horn clauses and definite clauses	46
5.4.8	Forward and backward chaining	46
5.5	La logica dei predicati	47
5.5.1	Limiti della logica proposizionale	47
5.5.2	Logica del primo ordine (1)	47
5.5.2.1	Logica del primo ordine vs logica proposizionale vs altre logiche	47
5.5.2.2	Logica del primo ordine (2)	47
5.5.2.3	Esempio	48
5.5.2.4	Logica del primo ordine: Sintassi	48
5.5.2.5	Logica del primo ordine: Semantica	49
5.5.3	I quantificatori	49
5.5.3.1	Qualificatore universale $\forall$	49
5.5.3.2	Qualificatore esistenziale $\exists$	50
5.5.4	Quantificatori annidati	50
5.5.5	Quantificatori annidati: connessioni tra $\forall$ e $\exists$	50
5.5.6	Logica del primo ordine: uguaglianza	50
5.5.7	Logica del primo ordine: Il mondo dei wumpus	50
5.5.8	Logica del primo ordine e regole di inferenza e forma a clausole	51
5.6	Modus Ponens Generalizzato	51
5.6.1	Forma a Clausole e Modus Ponens Generalizzato	51
5.6.2	Modus Ponens Generalizzato e inferenza	51
5.6.3	Inferenza nella logica del primo ordine	51
5.6.4	Vantaggi della logica	52
5.6.5	Limiti della logica	52
5.6.6	Ingegneria della conoscenza nella logica del primo ordine	52
5.6.7	The knowledge-engineering process	52
5.6.8	Prolog	53
5.7	Cenni di Pianificazione	53

6	Modelli Predittivi	53
6.1	Machine Learning and Data	53
6.1.1	Possibili applicazioni del ML	54
6.1.2	ML come un processo di analisi dati	55
6.1.3	Dove troviamo questa metodologia: Data science	55
6.2	L'approccio data-driven, l'analisi dei dati e dei modelli predittivi	56
6.2.1	Perché abbiamo bisogno di una metodologia scientifica per l'analisi dei dati	56
6.3	Data Science e una metodologia per l'analisi dei dati	56
6.3.1	Data Science	56
6.3.2	I cinque passi della Data Science	56
6.4	Data Science - Modelli predittivi	57
6.4.1	Trovarle correlazioni fra i dati	57
6.4.1.1	Esempio	57
6.5	Machine learning	58
6.5.1	Che cosa si intende con machine learning?	58
6.5.2	ML (algoritmi predittivi) vs algoritmi classici	58
6.5.3	Il machine learning non è perfetto	58
6.6	Algoritmi predittivi	58
6.6.1	Tipi di ML (algoritmi predittivi)	58
6.6.2	Classificazione degli algoritmi per scopo	59
6.6.2.1	Classificazione	59
6.6.2.2	Regressione	59
6.6.2.3	Clustering	59
6.6.3	Classificazione per modalità di apprendimento	60
6.6.3.1	Supervisionato	60
6.6.3.2	Non supervisionato	60
6.6.3.3	Semi-supervisionato	60
6.7	Costruire un modello di ML	60
6.7.1	Programmazione classica e ML	60
6.7.2	Gli algoritmi di ML	61
6.7.2.1	Implementazione del modello di machine learning	61
6.8	Problematiche comuni agli algoritmi di ML	61
6.8.1	Hyperparameter tuning	61
6.8.2	Grid Search (o parameter sweep)	61
6.8.3	Random Search	61
6.8.4	Overfitting	61
6.8.5	Utilizzo di un approccio addestramento/test	62
6.8.5.1	Capacità di generalizzazione	62
6.9	Basic algorithms in machine learning	62
6.9.1	Algoritmi di classificazione	62
6.9.2	Classificazione	62
6.9.3	Classificatore Naive (ingenuo) Bayes	62
6.9.3.1	Esempio	62
6.9.4	La probabilità condizionata e il teorema di Bayes	63
6.9.5	Classificatore di bayes vantaggi e svantaggi	63
6.9.6	Alberi decisionali (Decision Trees)	63
6.9.7	Support Vector Machines	64
6.9.8	Fuzzy rules based systems	64
6.9.9	Algoritmi di regressione	64
6.9.9.1	Regressione lineare	64
6.9.9.2	Regressione logistica	65
6.9.10	Algoritmi semi-supervisionati	65
6.9.11	Algoritmi di clustering	65
6.9.12	Model ensembles	65
6.9.13	Reti Neurali	66
6.10	Il test e la valutazione dei modelli predittivi	66
6.10.1	Hold-out e cross validation	66
6.10.2	Cross validation e matrice di confusione	66
6.10.2.1	Matrice di confusione	66
6.10.2.2	Matrice di confusione e metriche di valutazione del modello	67
6.10.2.3	Matrice di confusione e F-measure	67

6.10.3	Valutazione dei modelli di clustering	67
7	Esami Passati	67
7.1	Domanda 1	68
7.2	Domanda 2	68
7.3	Domanda 3	68
7.4	Domanda 4	68
7.5	Domanda 5	69
7.6	Domanda 6	70
7.7	Domanda 7	70
7.8	Domanda 8	70
7.9	Domanda 9	71
7.10	Domanda 10	71
7.11	Domanda 11	71
7.12	Domanda 12	72
7.13	Domanda 13	72
7.14	Domanda 14	73
7.15	Domanda 15	73
7.16	Domanda 16	73
7.17	Domanda 17	74
7.18	Domanda 18	74
7.19	Domanda 19	74
7.20	Domanda 20	75
7.21	Domanda 21	75
7.22	Domanda 22	75
7.23	Domanda 23	75
7.24	Domanda 24	76
7.25	Domanda 25	76
7.26	Domanda 26	76
7.27	Domanda 27	76
7.28	Domanda 28	77
7.29	Domanda 29	77
7.30	Domanda 30	77
7.31	Domanda 31	77
7.32	Domanda 32	78
7.33	Domadna 33	78
7.34	Domanda 34	78
7.35	Domanda 35	78
7.36	Domanda 36	78
7.37	Domanda 37	79
7.38	Domanda 38	79
7.39	Domanda 39	79
7.40	Domanda 40	79
7.41	Domanda 41	79
7.42	Domanda 42	80
7.43	Domanda 43	80
7.44	Domanda 44	80

1 Generalità

Un sistema intelligente deve essere in grado di prendere decisioni in modo autonomo, in funzione delle condizioni in cui si trova ad operare.

1.1 Test di Turing

Il test di Turing nasce nel 1950 come test per capire se una macchina si possa veramente considerare intelligente.

Condizione: "Un computer è da considerarsi intelligente se, nell'interazione a distanza con un essere umano, la persona non è in grado di distinguere se sta interagendo con un uomo o una macchina".

Test di Turing più "importante" è stato fatto nel 1991, dove dei giudici facendo delle domande dovevano capire se dall'altra parte ci fosse una persona o una macchina.

Nonostante la tecnologia non fosse ancora avanzata e la macchina rispondesse con un contesto sbagliato ad alcune domande, diversi giudici sono comunque stati ingannati.

Questo portò alla nascita di un dibattito sulla possibilità che le macchine potessero avere una coscienza o che, semplicemente, imitassero a "pappagallos" le indicazioni date dall'umano al momento della programmazione.

1.2 La stanza cinese

Nasce nel 1980

- Supponendo che il sistema superi il test di Turing, il computer prende simboli cinesi in entrata, esegue un programma e produce altri simboli.
- Nella stanza cinese c'è una persona a cui viene dato un libro con la versione inglese del programma utilizzato.
- La persona può elaborare i testi in cinese in arrivo, seguendo le istruzioni, ma non capisce realmente i simboli.
- Allo stesso modo, nemmeno la macchina riesce a comprendere ciò che legge.

Il computer è un manipolatore di simboli, quindi non capisce quello che viene espresso in cinese tanto quanto una persona che parla solo inglese.

1.3 Al forte vs Al debole (o narrow vs general)

Al debole	Al forte
Assumono le sembianze di un comportamento intelligente che può essere modellato e usato dai computer per risolvere problemi complessi	Ipotetica macchina con abilità umane cognitive
Si riferisce ad un sistema programmato per risolvere un ampio assortimento di problemi, ma opera con impostazioni predefinite	Con propria abilità di pensiero e che risolve task da sé
Non hanno mente propria	Macchine che hanno forte abilità umana cognitiva
Alexa e Siri sono i migliori esempi di Al debole	Un concetto ipotetico che non esiste ancora nella vita reale

1.4 Ordine del termine

Espressione "Artificial Intelligence" coniata nel 1956 dal matematico John McCarthy-Logic durante un seminario.

1.4.1 Gli inizi (1952-1959)

- Per fare analogie faceva uso di una buona/adatta" rappresentazione dell'informazione
- Il modo in cui qualcosa viene rappresentato su un particolare sistema di elaborazione. delle informazioni influisce in modo cruciale sul tipo di operazioni che possiamo eseguire in modo naturale e rapido



Figure 1: An example of a problem solved with. Evans' ANALOGY program (1968)

- Ciò che ha fatto capire l'importanza della rappresentazione dell'informazione e stata. la difficoltà tecnica di programmare i computer

Programma Analogico di Evan: Trasforma un'immagine in un grafo di conoscenza.

1.4.2 1968

- Capacità di calcolo limitata
- Restrizioni sul dominio (oggetti semplici rispetto al reale e poche relazioni).
- Programma:
 - Tradurre il disegno di una rappresentazione vettoriale di forme geometriche
 - Evidenziare le relazioni fra le forme dei fatti noti (la sequenza delle immagini date)
 - Selezionare un insieme di immagini possibili secondo i vincoli del dominio
 - Scegliere una di queste immagini

1.4.3 1969 - 1979

Sistemi esperti: Gli informatici cercano di trasferire la conoscenza di un esperto, in un settore limitato, in un insieme di regole: if condition then action

1.4.4 1995 - 2000: Nascita del Web e delle ontologie

Il grande successo del web ha portato in primo piano, ma in una forma nuova, la questione della rappresentazione simbolica della conoscenza.

Questioni più difficili sono l'interoperabilità delle applicazioni e la gestione delle risorse sulla rete.

Il concetto fondamentale è quello di ontologia, che ha cominciato a precisarsi negli anni '80. Le tecnologie semantiche saranno essenziali per fornire una piattaforma tecnologica adeguata.

1.4.5 2011 - present

L'accesso a grandi quantità di dati ("big data"), computer più economici e veloci e tecniche avanzate di apprendimento automatico sono stati applicati con successo a molti problemi.

I progressi nel deep learning e nella ricerca hanno portato a progressi nell'elaborazione di immagini e video, nell'analisi del testo e persino nel riconoscimento vocale.

1.5 Il gioco In IA

- **Chess games:** Lo svolgersi del gioco può essere interpretato come un albero in cui la radice è la posizione di partenza e le foglie le posizioni finali.
- **Gioco del GO.**
- **Jeopardy!:** è un quiz televisivo statunitense che consiste in una cultura generale della competizione tra i vari concorrenti, trasmesso dall'emittente NBC dal 1964.
- **Pluribus:** Gioco simile al Poker, è riuscito a replicare i comportamenti dell'essere umano, cioè il bluff;

- **Stratego:** Come racconta uno studio pubblicato su Science il 1 dicembre, l'AI - DeepNash ha imparato a bluffare con i pezzi meno di valore e a sacrificare quelli più pregiati pur di ottenere la vittoria e guadagnare informazioni sulla strategia dell'avversario. Queste scelte lo rendono imprevedibile, raggiungendo l'84% di tasso di vincita durante 50 partite svolte online.



Figure 2: Immagine di un Chess Game

1.6 Ragionamento Deduttivo vs Ragionamento Induttivo

ASI: Intelligenza artificiale speculativa. ASI è dove le macchine sono autocoscienti, oltre le capacità e conoscenze umane.

Approccio psicologico: Obiettivo di creare una macchina pensante (progetto, per ora, fallimentare).

Approccio ingegneristico: Obiettivo di programmare una macchina che risolva problemi in modo più efficiente.

Per dare una coscienza alla macchina ci vorrebbe una tecnologia completamente diversa, non sarebbe possibile utilizzare quella computerizzata che utilizziamo adesso, serve un approccio completamente diverso.

1.7 Conoscenza e comportamento razionale

La conoscenza è informazione disponibile per un'azione razionale.

Comportamento razionale: Fare la cosa più giusta, quella da cui ci si aspetta il massimo risultato, a fronte delle informazioni disponibili.

Una corretta inferenza non sempre esaurisce il comportamento razionale (es. in certe situazioni va decisa anche in assenza di inferenze). Non tutti i comportamenti razionali implicano inferenze (es. togliere una mano dal fuoco).

1.8 Approccio simbolico vs approccio subsimbolico

Gli individui agiscono sulla conoscenza e sulle percezioni del mondo attraverso una attività di ragionamento:

- **Deduttivo:** Dalle premesse alle conclusioni
- **Abduttivo:** Dagli effetti alle possibili cause
- **Induttivo:** Da fatti specifici a regole generali

Deduttivo e abduttivo hanno un approccio simbolico, induttivo uno sub-simbolico.

Questi ragionamenti servono per produrre una nuova conoscenza e, eventualmente, decidere una azione.

Top Down o simbolico: La conoscenza è rappresentata da simboli. Vengono utilizzate la logica e la capacità di fare inferenze.

Bottom UP o subsimbolico (basato sul segnale): Non esiste una rappresentazione esplicita della conoscenza. Vengono utilizzate reti neurali, approcci evolutivi, ecc.

Non esiste una metodologia migliore dell'altra, dipende solo da che contesti vengono utilizzate.

1.8.1 Approccio simbolico

L'approccio simbolico alla costruzione di un AI si serve di una descrizione esplicita della conoscenza di un dominio e fa uso di un linguaggio formale (adatto ad essere utilizzato da un automa) per attingere a questa base di conoscenza e inferire nuova conoscenza (o azioni da intraprendere).

Per poter realizzarli si sono sviluppati framework basati sulla logica come classe generale di rappresentazioni e linguaggio formale.

1.8.1.1 Logica

La logica è la scienza che fornisce agli uomini gli strumenti indispensabili per verificare con sicurezza la correttezza del ragionamento.

La logica fornisce gli strumenti formali per:

- **Esprimere** inferenze in termini di operazioni su espressioni simboliche
- **Dedurre** le conseguenze da determinate premesse
- **Studiare** la verità o la falsità su certe proposizioni data la verità o la falsità di altre proposizioni
- **Stabilire** la coerenza e la validità di una data teoria

Ragionamento deduttivo: Modo per inferire nuove conoscenze nella logica, se le premesse sono vere, allora le conclusioni sono vere.

Regole di inferenza: Composte da due parti divise dal simbolo “ $\Rightarrow$ ” che si legge come “è possibile derivare”.

Il **sillogismo** è un tipico esempio di ragionamento deduttivo.

In generale, in logica si studia la relazione tra le formule di un sistema logico-simbolico in cui:

- Il linguaggio delle formule è definito con precisione
- Il significato delle formule è stabilito in modo univoco

E' possibile trovare un'affermazione falsa partendo da due proposizioni vere utilizzando un linguaggio naturale (che per sua natura è ambiguo).

Logica moderna: Rappresenta in modo formale la corrispondenza tra espressioni linguistiche e significato inteso.

Formalizzazione (astrazione): Comprensione delle informazioni, con ampia perdita di dettagli (sfumature) e guadagno di precisione (non tutte le espressioni linguistiche sono “buoni candidati” per questa traduzione).

Uno dei compiti più famosi nello studio del ragionamento deduttivo è il compito di selezione di Wason (1966).

[Approfondimento su slide "Intro-IA-02-Generalità" pagina 99](#)

1.8.2 Approccio sub-simbolico

I modelli di IA in questo paradigma sono rappresentati implicitamente.

La rappresentazione implicita deriva dall'apprendimento dall'esperienza senza rappresentazione simbolica di regole e proprietà.

Invece di relazioni leggibili dall'uomo chiaramente definite, si implementano equazioni matematiche meno spiegabili per risolvere i problemi.

Al fine di realizzare macchine che possano dirsi intelligenti, è necessario dotarle della capacità di estendere la propria conoscenza e le proprie abilità in modo autonomo.

Campo machine learning: Apprendimento automatico, costituito da diversi componenti basati su tecniche e paradigmi quali statistica, probabilità, reti, etc.

Usato approccio bottom up (regole imparate da basso), in genere tratta di sistemi complessi.

1.8.2.1 Ragionamento e apprendimento

La capacità di apprendimento è fondamentale in un sistema intelligente per:

- Risolvere nuovi problemi (e includere la scoperta nella propria base di conoscenza)
- Non ripetere gli errori fatti in passato
- Risolvere problemi in modo migliore o più efficiente

- Avere autonomia nell'affrontare e risolvere problemi
- Adattarsi ai cambiamenti dell'ambiente circostante

Apprendimento induttivo: Viene utilizzato per apprendere dei concetti espressi in genere con delle regole la cui forma è: "nella situazione X esegui l'azione Y", "all'osservazione X associa l'osservazione Y", "alla situazione X associa la conseguenza Y".

Si parla di **generalizzazione** delle informazioni contenute nei pochi esempi a disposizione in un modello riutilizzabile in tutte altre possibili situazioni di quel tipo.

1.9 Cos'è la teoria del learning

Sistemi esperti: Un sistema esperto è un sistema basato su regole di produzione (della forma IF x THEN y, syllogismo) che, a partire da alcuni fatti (dati) cerca di dimostrare un'ipotesi o raggiungere una conclusione.

Alberi di decisione: I nodi degli alberi costituiscono le variabili, le foglie le decisioni.



Figure 3: Albero di decisione

Weather	Wind	Humidity	Decision
Sunny	NO	30-60	PLAY
Sunny	YES	90	DON'T PLAY
Overcast	NO	50	PLAY
Rainy	YES	50	DON'T PLAY
Rainy	YES	90	DON'T PLAY
Sunny	NO	80	DON'T PLAY
Overcast	YES	80	PLAY
Rainy	NO	70	PLAY

Si parla di "Machine Learning simbolico" cioè basato su concetti astratti e non rappresentati da numeri o misure.

Per creare gli alberi di decisione, solitamente si costruisce una "funzione/tabella" di verità.

1.9.1 Inferenze

Learning = Inferencing + Memorizing

Nel learning se il risultato di un'inferenza risulta utile, essa viene memorizzata.

Le inferenze possono essere **deduttive** o **induttive**.

Si consideri l'equazione fondamentale per l'inferenza:

- P sta per premessa;
- BK per conoscenza pregressa;
- $\models$ per implicazione;
- C per conseguente.

L'inferenza deduttiva deriva C , dai dati P e BK (veri), e preserva la verità.

L'inferenza induttiva sta ipotizzando P , dai dati C e BK , e preserva la falsità.

Si possono poi classificare ulteriormente in inferenza conclusiva (forte) e contingente (debole).

- **Inferenze conclusive:** Implicano regole di inferenza indipendenti dal dominio (contesto), mentre le inferenze contingenti implicano regole dipendenti dal dominio.
- **Deduzione contingente:** Produce probabili conseguenze di determinate cause.

- **Induzione contingente:** Produce probabili cause di determinate conseguenze.
- **Analogia:** area centrale del diagramma che rappresenta i processi di learning caratterizzati da induzione e deduzione.



Figure 4: Schema riassuntivo

1.9.2 Deep Learning e Reti Neurali Artificiali (ANN)

Il processo di deep learning è uno degli approcci all'apprendimento automatico che ha preso spunto dalla struttura del cervello, ovvero l'interconnessione dei vari neuroni.

Le reti neurali artificiali (ANN) si basano su questo fenomeno biologico, ma sono formate da neuroni artificiali costituiti da moduli software chiamati nodi.

I nodi utilizzano calcoli matematici (al posto dei segnali chimici cerebrali) per comunicare e trasmettere informazioni. Questa rete neurale simulata elabora i dati raggruppando punti dati e facendo previsioni.

Deep learning: Si può immaginare come un diagramma di flusso con un livello iniziale in cui vengono inseriti dati e uno finale che fornisce risultati.

Tra questi due troviamo "livelli nascosti" che elaborano informazioni a diversi livelli, modificando e adeguando il loro comportamento man mano che ricevono nuovi dati.

Alcuni limiti:

- Hanno bisogno di un numero enorme di esempi
- Hanno difficoltà a generalizzare
- Hanno scarse capacità a gestire l'inferenza e il ragionamento logico
- Hanno difficoltà a distinguere fra correlazioni e causa-effetto.

1.10 Black Box AI & Explainable AI

I sistemi Machine Learning spesso funzionano come **una scatola nera**, che fa previsioni e decisioni come fanno gli umani, ma senza essere in grado di comunicare le ragioni per farlo.

Explainable AI (XAI): Studia il motivo per cui è stata presa una decisione dall'AI in modo che i modelli di AI possano essere più interpretabili per gli utenti umani e consentire loro di capire perché il sistema è arrivato a una decisione specifica.

1.10.1 Indagare un sistema di AI

Un primo livello di indagine riguarda l'**Interpretability** (interpretabilità), cioè la possibilità di mettere in relazione causale i dati in ingresso con quelli in uscita (corrisponde alla previsione WHAT).

Un secondo livello è relativo alla **Explainability** (spiegazione), in termini comprensibili ad un essere umano, su come il modello sia arrivato ad una determinata scelta o previsione (WHY).

La interpretability di un modello può generare la rappresentazione di un processo decisionale, ma trasformarlo in una explainability utile può essere difficile poiché dipende da diversi fattori:

- Il tipo di algoritmo che genera il modello
- Il livello di spiegazione richiesto
- Il tipo di dati utilizzati nel modello.

Alcuni approcci per la giustificazione a posteriori di un modello di ML:

- Spiegazione testuale
- Spiegazione tramite esempi
- Semplificazione del modello
- Visualizzazione
- Spiegazione locale: Riduzione della complessità del problema grazie alla ricerca di spiegazioni per una sotto-porzione del dominio approssimato dal modello.
- Feature rilevanti: Quantificazione dell'influenza di una feature del problema sull'output finale. L'analisi comparata di questi attributi fornisce una spiegazione indiretta sul funzionamento del modello.



Figure 5: Schema riassuntivo

1.10.1.1 Esempio: Un oggetto cade a terra e si rompe.

- L'Interpretability può dare le ragioni meccaniche della caduta ma senza coglierne veramente le cause: l'oggetto è caduto, per questo ha toccato terra e per tale ragione si è infranto.
- L'Explainability dovrebbe dare le vere ragioni alla base del fenomeno. In questo caso potrebbe essere: l'oggetto è caduto, a causa della gravità ha ottenuto una certa accelerazione e l'energia accumulata nella caduta ha portato alla rottura dell'oggetto all'impatto.

2 Agenti Intelligenti

2.1 Introduzione

L'agire razionale rispetto allo scopo è un agire orientato sulla base della valutazione dei mezzi più adeguati per raggiungere un certo scopo.

Agente razionale usa l'intelligenza come capacità di interagire con un ambiente.

Caratteristiche degli agenti:

- Sono situati (contextualizzati):
 - Ricevono percezioni da un ambiente
 - Agiscono sull'ambiente mediante azioni (attuatori)
- Hanno abilità sociale:
 - Capaci di comunicare
- Hanno credenze, obiettivi, intenzioni
- Sono embodied:
 - Sono limitati da un corpo e potrebbero essere soggetti ad emozioni

Un agente percepisce il proprio ambiente attraverso sensori ed agisce attraverso attuatori.

Terminologia agenti:

- **Percezione:** input da sensori.



Figure 6: Schema astratto funzionamento agente

- **Sequenza percettiva:** Storia completa delle percezioni.
- **Funzione agente:**
 - Definisce l'azione da compiere per ogni sequenza percettiva (descrive completamente l'agente);
 - Implementa da un programma agente.
- **Compito (IA):** progettare il programma agente.

Serve un criterio di valutazione oggettivo dell'effetto delle azioni dell'agente (della sequenza di stati dell'ambiente).

Misura di prestazione:

- Esterna (come vogliamo che il mondo evolva?);
- Scelta dal progettista a seconda del problema considerando l'effetto desiderato sull'ambiente;
- (Possibile) Valutazione su ambienti diversi.

2.1.1 Esempio

Fissata la misura delle performance possiamo valutare come opera nel tempo sull'ambiente. Così possiamo dire



Figure 7: Schema esempio

che la razionalità è relativa a:

- La misura di prestazioni;
- Le conoscenze pregresse dell'ambiente;
- Le percezioni presenti e passate (seq. percettiva);
- Le capacità dell'agente (azioni possibili).

2.2 Agenti Intelligenti/razionale

Agente razionale: Per ogni sequenza di percezioni compie l'azione che massimizza il valore atteso della misura delle prestazioni, considerando le sue percezioni passate e la sua conoscenza pregressa.

Razionalità non onniscienza:

- Non si pretendono perfezione e conoscenza del "futuro", ma massimizzare il risultato atteso
- Potrebbero essere necessarie azioni di acquisizione di informazioni o esplorative

Razionalità non onnipotenza:

- Le capacità dell'agente sono limitate

Razionalità e apprendimento:

- Raramente tutta la conoscenza sull'ambiente può essere fornita "a priori" (dal programmatore)
- L'agente razionale deve essere in grado di modificare il proprio comportamento con l'esperienza (le percezioni passate)
- Può migliorare esplorando, apprendendo, aumentando autonomia per operare in ambienti differenti o mutevoli

2.3 Agenti e ambiente

Ambiente: Dove l'agente opera, tutto ciò serve a definire un problema per un agente (Agente razionale = soluzione).

PEAS:

- P: Performance/prestazione;
- E: Enviroment/ambiente;
- A: Actuators/attuatori;
- S: Sensors/sensori.

2.3.1 Esempi



Figure 8: Schema esempio

2.4 Proprietà dell'ambiente-problema

- Completamente/parzialmente osservabile
- Agente singolo/multi-agente
- Deterministico/stocastico/non deterministico
- Episodico/sequenziale
- Statico/dinamico
- Discreto/continuo

Ambiente completamente osservabile:

- L'apparato percettivo è in grado di dare una conoscenza completa dell'ambiente o almeno tutto quello che serve a decidere l'azione;
- Non c'è bisogno di mantenere uno stato del mondo (esterno).

Ambiente parzialmente osservabile:

- Sono presenti limiti o inaccuratezze dell'apparato sensoriale.

Ambiente singolo/multiagente:

- **Distinzione agente/non agente:** Il mondo può anche cambiare per eventi, non necessariamente per azioni di agenti.
- **Ambiente multi-agente competitivo (schacchi):** Comportamento randomizzato (è razionale);
- **Ambiente multi-agente cooperativo (o benigno):** Stesso obiettivo Comunicazione.

Predicibilità dell'ambiente:

- **Deterministico:** Se lo stato successivo è completamente determinato dallo stato corrente e dall'azione. Esempio: scacchi;
- **Stocastico:** Esistono elementi di incertezza con associata probabilità. Esempi: guida, tiro in porta;
- **Non deterministico:** Si tiene traccia di più stati possibili risultato dell'azione (ma non in base ad una probabilità).

Ambiente statico/dinamico:

- **Statico:** Il mondo non cambia mentre l'agente decide l'azione
- **Dinamico:** Cambia nel tempo, va osservata la contingenza. Tardare equivale a non agire
- **Semi-dinamico:** L'ambiente non cambia ma la valutazione dell'agente sì. Esempio: Scacchi con timer

Discreto/continuo

Possono assumere valori discreti o continui:

- Lo stato: solo un numero finito di stati
- Il tempo
- Le percezioni
- Le azioni

Noto/ignoto:

- Distinzione riferita allo stato di conoscenza dell'agente
- L'agente conosce l'ambiente oppure deve compiere azioni esplorative?
- Noto diverso da osservabile (e.g. carte coperte, regole note)

Ambienti reali: Parzialmente osservabili, stocastici, sequenziali, dinamici, continui, multi-agente, ignoti.

2.5 Struttura

Quattro tipi fondamentali in ordine di generalità crescente:

- Semplici agenti riflessi
- Agenti riflessi con stato
- Agenti basati su obiettivi
- Agenti basati sull'utilità

Tutti questi possono essere trasformati in agenti con apprendimento (learning agents).

Agente = Architettura + Programma

$$Ag: \underset{\text{percezioni}}{P} \rightarrow \underset{\text{azioni}}{Az}$$

Il programma dell'agente implementa la funzione Ag

↓

function Skeleton-Agent (*percept*) **returns** action

static: memory, the agent's memory of the world

memory ← UpdateMemory(*memory*, *percept*)

action ← Choose-Best-Action(*memory*)

memory ← UpdateMemory(*memory*, *action*)

return *action*

Agente basato su tabella: La scelta dell'azione è un accesso a una tabella che associa un'azione ad ogni possibile sequenza di percezioni.

Ha diversi problemi come: la dimensione, era difficile da costruire, no autonomia, difficile aggiornamento.

Agenti reattivi semplici:

- Questi agenti selezionano le azioni sulla base della percezione corrente, ignorando il resto della cronologia della percezione;
- Gli agenti reattivi hanno la proprietà di essere semplici, ma sono anche di intelligenza limitata;
- L'agente funzionerà solo se la decisione corretta può essere presa solo sulla base della percezione corrente, cioè solo se l'ambiente è completamente osservabile. Anche un minimo di inosservabilità può causare seri problemi.

Agenti con stato interno (modello): Indipendentemente dal tipo di rappresentazione utilizzata, è raramente possibile che l'agente determini esattamente lo stato attuale di un ambiente parzialmente osservabile.

La casella etichettata "com'è il mondo adesso" rappresenta la "migliore ipotesi" dell'agente (o talvolta la migliore ipotesi).

Agenti con obiettivo:

- Ove bisogna pianificare una sequenza di azioni per raggiungere l'obiettivo (goal)
- A volte la selezione dell'azione basata sull'obiettivo è semplice, es. quando si raggiunge l'obiettivo con una singola azione
- Altre volte sarà più complicato, ad esempio, quando l'agente deve considerare lunghe sequenze di azioni per trovare un modo per raggiungere l'obiettivo
- Un processo decisionale di questo tipo è fondamentalmente diverso dalle regole condizione-azione, in quanto implica le considerazioni del futuro (**Da approfondire!! [secondo pacco di slide da pagina 47]**)
- Sono guidati da un obiettivo nella scelta dell'azione
- Devono pianificare una sequenza di azioni per raggiungere l'obiettivo
- Meno efficienti ma più flessibili di un agente reattivo

Agenti con funzione di utilità:

- La funzione di utilità è essenzialmente un'interiorizzazione della misura della performance
- Se la funzione di utilità interna e la misura della prestazione esterna sono in accordo, allora un agente che sceglie le azioni per massimizzare la propria utilità sarà razionale in base alla misura della prestazione esterna
- Un agente basato sull'utilità deve modellare e tenere traccia del suo ambiente, compiti che hanno comportato una grande quantità di ricerca sulla percezione, rappresentazione, ragionamento e apprendimento.

Agenti che apprendono (learning agent):

1. Componente di apprendimento:
 - (a) Produce cambiamenti al programma agente;
 - (b) Migliora prestazioni, adattando i suoi componenti, aprendo dall'ambiente;
2. Elemento esecutivo (performance element):
 - (a) Il programma agente;
 - (b) Responsabile del selezionare le azioni;
3. Elemento critico: Osserva e dà feedback sul comportamento;
4. Generatore di problemi: Suggerisce nuove situazioni da esplorare.

La progettazione dell'elemento di apprendimento dipende molto dalla progettazione dell'elemento di prestazione.

2.5.1 Esempio: taxi

- L'elemento di prestazione consiste in una qualunque raccolta di conoscenze e procedure di cui dispone il taxi per selezionare le sue azioni di guida
- Il taxi esce sulla strada e guida, sfruttando questo elemento di performance
- Il modulo critico osserva il mondo e trasmette le informazioni all'elemento di apprendimento
- Da questa esperienza, l'elemento di apprendimento è in grado di formulare una regola dicendo che questa è stata una cattiva azione e l'elemento di prestazione viene modificato dall'installazione della nuova regola
- Il generatore di problemi potrebbe identificare alcune aree di comportamento che necessitano di miglioramenti e suggerire esperimenti, come provare i freni su superfici stradali diverse in condizioni diverse



Figure 9: Schema Learning Agent

2.6 Tipi di rappresentazione dello stato agente

- Atomica
- Fattorizzata (più variabili e attributi)
- Strutturata (aggiunge relazioni)



Figure 10: Tipi di rappresentazione dello stato agente

Nella rappresentazione **atomica** ogni stato del mondo è indivisibile, non ha una struttura interna. Un singolo atomo di conoscenza è una “scatola nera” la cui unica proprietà riconoscibile è quella di essere identica o diversa da un'altra scatola nera.

Nella rappresentazione **fattorizzata** si suddivide ogni stato in un insieme fisso di variabili o attributi, ognuno dei quali può avere un valore. Due diversi stati fattorizzati possono condividere alcuni attributi (pos. GPS) e non altri (avere molto o non avere gas).

Si passa meglio da uno stato all'altro, si può rappresentare l'incertezza.

Nella rappresentazione **strutturata** oggetti diversi come mucche e camion e le loro variabili descrittive e le loro relazioni possano essere descritti in modo esplicito.

Quasi tutto ciò che gli esseri umani esprimono nel linguaggio naturale riguarda gli oggetti e le loro relazioni.

2.7 Espressività

Una rappresentazione più espressiva può catturare, almeno in modo altrettanto conciso, tutto ciò che può catturare una rappresentazione meno espressiva.

Il linguaggio più espressivo è molto più conciso.

D'altra parte, il ragionamento e l'apprendimento diventano più complessi man mano che aumenta la potenza espressiva della rappresentazione.

3 Agenti Risolutori di Problemi

3.1 Introduzione

Gli agenti reattivi sono i più semplici e basano le loro azioni su mappatura diretta dagli stati delle azioni.

Gli agenti basati sugli obiettivi considerano le loro azioni future e l'opportunità dei loro risultati.

L'agente risolutore di problemi è un tipo particolare di agente basato su obiettivi. Decide cosa fare ricercando sequenze di azioni che conducono a stati desiderabili:

- Gli obiettivi aiutano a selezionare le sequenze.

Quindi cerca di massimizzare la misura delle prestazioni.

Adottano il paradigma della risoluzione di problemi come ricerca in uno spazio di stati (problem solving):

- Sono agenti con modello che adottano una rappresentazione atomica dello stato;
- Sono particolari agenti con obiettivo, che pianificano l'intera sequenza di mosse prima di agire.

3.2 Risoluzione di problemi tramite ricerca

I problemi si possono modellare come Problemi di Ricerca in uno spazio degli stati (Strategie di Ricerca).

Un problema viene risolto ricercandone la soluzione in un ampio spazio di possibili soluzioni.

La ricerca di una soluzione è la traduzione della conoscenza in una rappresentazione opportuna del mondo composta da stati (situazioni del mondo) e un insieme di operatori per passare da uno stato ad un altro.

Agire per risolvere problemi:

- Formulazione dell'obiettivo:
 - Organizza il comportamento, limitando il numero di scopi da raggiungere;
 - Si basa sulla situazione corrente e sulle capacità del soggetto.
- Formulazione del problema:
 - Processo che porta a decidere quali azioni da compiere dato un obiettivo;
 - La selezione della sequenza di azioni che porta all'obiettivo è detta ricerca.

Se ci sono più alternative e l'agente non conosce lo stato risultante dopo aver compiuto ciascuna azione, potrà solo scegliere a caso.

Se possiede informazioni sugli stati, le userà per scegliere la sequenza di azioni da intraprendere.

Questa conoscenza extra si chiama **conoscenza euristica**.

Il processo di ricerca si può pensare come la costruzione ad albero in cui i nodi sono gli stati e i rami gli operatori.

Spazio degli stati

Lo spazio degli stati è l'insieme di tutti gli stati raggiungibili dallo stato iniziale con una qualunque sequenza di operatori.

È caratterizzato da:

- Uno stato iniziale in cui l'agente sa di trovarsi (non noto a priori)
- Un insieme di azioni possibili che sono disponibili da parte dell'agente (Operatori che trasformano uno stato in un altro)
- Un cammino è una sequenza di azioni che conduce da uno stato a un altro

Un problema può essere definito formalmente da cinque componenti:

1. Lo stato iniziale in cui inizia l'agente
2. Una descrizione delle possibili azioni disponibili per l'agente. Dato un particolare stato s , è possibile restituire l'insieme di azioni eseguibili in s
3. Una descrizione di ciò che fa ogni azione (si dice funzione transizione o successore)
4. Un percorso nello spazio degli stati è una sequenza di stati collegati da una sequenza di azioni
5. Test obiettivo, che determina se un determinato stato è uno stato obiettivo

3.2.1 Esempio

Si è a Oradea e si vuole arrivare a Bucarest per prendere un aereo si ha a disposizione una automobile



Figure 11: Descrizione visiva esempio

Un algoritmo di ricerca prende un problema come input e restituisce una soluzione sotto forma di una sequenza di azioni

Un agente razionale semplice opera quindi in tre fasi:

- Formulazione del problema
- Ricerca della soluzione
- Esecuzione della sequenza di azioni

3.3 Problemi ben definiti e soluzioni

Un problema è definito da un goal e dall'insieme di strumenti a disposizione per raggiungerlo.

La prima fase della risoluzione di un problema è la formulazione del problema. In questa fase bisogna decidere quali sono le azioni, quale è lo spazio degli stati (insieme degli stati raggiungibili) e quale è lo stato da cui si parte (stato iniziale).

3.4 Risoluzione di problemi

A partire dallo stato iniziale bisogna trovare una sequenza di azioni, soluzione, che soddisfi l'obiettivo.

Una buona astrazione (scelta di azioni e stati che permettono di formulare soluzioni) comporta l'eliminazione di più dettagli possibili mantenendo la validità ed assicurando che le azioni astratte siano facili da realizzare.

3.5 Formulazione del problema

Nella formulazione del problema il processo di astrazione esegue il passaggio dal problema concreto al modello del problema con la rimozione dei dettagli "inutili" per la ricerca di una soluzione.

Trovare il giusto livello di astrazione è una procedura a volte complessa che richiede, in questo tipo di agenti, una attività umana.

L'esecuzione di ogni azione ha un costo, che può variare da azione ad azione. Quindi, ad ogni sequenza di azioni è possibile assegnare un costo complessivo, per valutare quale sia la soluzione migliore.

La scelta della sequenza corrisponde alla seconda fase della risoluzione di un problema e viene individuata con il nome di ricerca. Infine, trovata la soluzione, nella terza fase, detta di esecuzione, le azioni sono eseguite per raggiungere il goal.

Costo totale della soluzione = costo di cammino + costo di ricerca.

3.6 Costo della ricerca

L'agente deve decidere quali risorse dedicare alla ricerca e quali all'esecuzione.

Per spazi degli stati piccoli, si considera il costo di cammino più basso. Per problemi complessi trovare il punto di equilibrio

3.6.1 Esempio

Vacanza in Romania.

Attualmente in Arad.

L'aereo parte domani da Bucarest.

Formulare un goal:

- Essere in Bucarest

Formulare un problema:

- Stati: varie
- Città operatori: guidare da una città all'altra

Trovare una soluzione:

- Sequenza di città, es: Arad, Sibiu, Faragas, Bucarest



Figure 12: Esempio 8 puzzle

3.7 Esempi di problemi - Problemi giocattolo

3.7.1 Il problema dell'8 puzzle

- Stati: Uno stato specifica la posizione di ciascuna delle 8 tessere.
- Operatori: Lo spazio vuoto si muove a destra, a sinistra, sopra, sotto.
- Test obiettivo: Lo stato rispecchia la configurazione obiettivo (Goal).
- Costo del cammino: Ciascun passo costa 1. Il costo del cammino coincide con la sua lunghezza.

3.7.2 Il problema delle 8 regine



Figure 13: Esempio 8 regine

Ci sono due principali tipi di formulazione:

1. Una formulazione incrementale coinvolge operatori che aumentano la descrizione dello stato, che inizia con uno stato vuoto e che per il problema delle 8 regine, questo significa che ogni azione aggiunge una regina allo stato
2. Una formulazione a stato completo che inizia con tutte e 8 le regine sul tabellone e le sposta attorno

In entrambi i casi, il costo del percorso non interessa perché conta solo lo stato finale.

Nella formulazione incrementale:

- Stati: Qualsiasi configurazione da 0 a 8 regine sulla scacchiera
- Operatori: Aggiungi una regina in qualsiasi quadrato
- Test obiettivo: 8 regine sulla scacchiera, nessuna minacciata
- Costo cammino: 0 (perché conta solo lo stato finale)

3.7.3 Il problema dei missionari e dei cannibali

3 missionari e 3 cannibali devono attraversare un fiume.

C'è una sola barca che può contenere al massimo due persone.

Per evitare di essere mangiati i missionari non devono mai essere meno dei cannibali sulla stessa sponda (stati di fallimento).

- Stato: Sequenza ordinata di tre numeri che rappresentano il numero di missionari, cannibali e barche sulla sponda del fiume da cui sono partiti
- Stato iniziale: (3,3,1) (importanza dell'astrazione)
- Operatori: Gli operatori devono portare in barca 1 missionario, 1 cannibale, 2 missionari, 2 cannibali, 1 missionario e 1 cannibale. Al più 5 operatori (grazie all'astrazione sullo stato scelta)
- Test Obiettivo: (0,0,0)
- Costo di cammino: numero di traversate

3.8 Classi di problemi

L'agente risolutore di problemi riesce a risolvere problemi in modo osservabile, statico e deterministico.

I problemi possono essere divisi in quattro classi in dipendenza della conoscenza che l'agente ha sullo stato del mondo e sulle azioni che si possono eseguire su di esso.

1. Problemi a stato singolo (mondo accessibile);
2. Problemi a stati multipli (mondo non accessibile - no sensori);
3. Problemi di contingenza (ignoranza dell'agente sulla sequenza di azioni);
4. Problemi di esplorazione (nessuna contingenza sugli effetti delle proprie azioni).

3.8.1 A stato singolo (Search)

Si ha conoscenza:

- Dello stato del mondo (tramite i sensori l'agente riceve informazioni sufficienti sullo stato in cui si trova)
- Degli effetti delle proprie azioni problema deterministico e osservabile: l'agente può calcolare esattamente in quale stato sarà dopo qualsiasi sequenza di azioni

Formulazione di un problema:

1. Stato iniziale x ;
2. Operatore/funzione successore $S(x)$;
3. Test obiettivo;
4. Funzione costo cammino.

Una soluzione è una sequenza di operatori che conducono dallo stato iniziale ad uno stato obiettivo.

3.8.2 Problemi a stato multiplo (Search)

Si ha conoscenza completa degli effetti delle azioni come nel problema a stato singolo, ma non si ha conoscenza sufficiente sullo stato del mondo (problema non osservabile, p.e. agente senza sensori).

A partire dallo stato iniziale l'agente deve ragionare su insiemi di stati in cui potrebbe giungere invece che su stati singoli.

L'agente deve parlare di insiemi di stati raggiungibili e non di singoli stati. Ogni insieme viene chiamato stato-credenza (belief state).

Formulazione di un problema:

1. Insieme di stati iniziali
2. Insieme di operatori / funzione successore $S(x)$ (per ciascuna azione viene specificato l'insieme di stati raggiunti da qualsiasi stato considerato. Un cammino collega insiemi di stati)
3. Test obiettivo
4. Funzione costo cammino

Una soluzione è un cammino che conduce ad un insieme di stati (sottoinsieme dello spazio degli stati) che sono tutti stati obiettivo.

3.8.3 Problemi con imprevisti (Planning)

In questi casi l'agente deve:

- Calcolare un intero albero di azioni piuttosto che una singola sequenza di azioni
 - Un ramo dell'albero tratta una situazione contingente possibile che si potrebbe verificare

Spesso per la soluzione è necessario agire (e verificare l'effetto della propria azione) prima di avere trovato una soluzione:

- L'agente incomincia ad eseguire un piano possibile

- Sulla base di quali soluzioni contingenti si verificano ricalcola ed attua un nuovo piano

Si sfruttano le informazioni supplementari trovate runtime per continuare a risolvere il problema.

Ogni possibile azione definisce una contingenza che deve essere pianificata.

L'agente necessita quindi di piani di contingenza per gestire le circostanze sconosciute che si potranno verificare. Un problema è detto con avversari se l'incertezza è causata da un altro agente.

3.8.4 Problemi di esplorazione (Learning)

Non si ha conoscenza né sullo stato del mondo né sugli effetti delle proprie azioni.

Lo spazio degli stati è sconosciuto. Quando gli stati e le azioni dell'ambiente sono sconosciuti, l'agente deve fare in modo di scoprirle.

I problemi di esplorazione possono essere considerati casi estremi di problemi di contingenza.

E' necessario apprendere i possibili stati del mondo e gli effetti delle proprie azioni attraverso l'esperienza.

La ricerca si svolge nel mondo reale e non in un modello: agire può comportare danni significativi per un agente privo di conoscenza.

Se sopravvive, acquisisce conoscenza che può riusare per problemi successivi.

3.8.5 Problemi stati singoli vs stati multipli

Problema a stati singoli:

- Stato sempre accessibile
- L'agente conosce esattamente che cosa produce ciascuna delle sue azioni e può calcolare esattamente in quale stato sarà dopo qualunque sequenza di azioni

Esempio a slide 54 terzo pacco di slide

Problema a stati multipli:

- Stato non completamente accessibile, l'agente deve ragionare su possibili stati che potrebbe raggiungere e non sui singoli
- L'effetto delle azioni potrebbe essere sconosciuto o imprevisto

Problemi di contingenza: Conoscenza incompleta degli effetti delle azioni, la soluzione è usare gli alberi di stati.

Problemi di esplorazione: Nessuna informazione degli effetti delle azioni e dello stato del mondo, l'agente deve compiere azioni esplorative (provare ad agire).

3.8.5.1 Esempio

Mondo dell'aspirapolvere con legge di Murphy.

“Aspirare” talvolta deposita sporco, ma solo se in quel punto non c'è già sporco (legge di Murphy).

Se l'aspirapolvere si trova in 4 aspirare potrebbe rimanere nello stato 4 oppure andare a finire nello stato 2.

Se ho capacità di rilevamento durante la fase di esecuzione ecco che posso scegliere l'azione di aspirare solo nel caso ci sia sporco.



Figure 14: Esempio aspirapolvere

3.9 Ricercare le soluzioni

3.9.1 Cercare soluzioni

Soluzione: Sequenza di azioni, quindi gli algoritmi di ricerca funzionano considerando varie possibili sequenze di azioni.

Passi da seguire:

1. Determinazione obiettivo
2. Formulazione del problema
3. Determinazione della soluzione mediante ricerca
4. Esecuzione del piano

Generare sequenze di azioni:

- **Espansione:** Si parte da uno stato e applicando gli operatori (o la funzione successore) si generano nuovi stati;
- **Strategia di ricerca:** Ad ogni passo scegliere quale stato espandere;
- **Albero di ricerca:** Rappresenta l'espansione degli stati a partire dallo stato iniziale. Le foglie dell'albero rappresentano gli stati da esplodere.

Strutture dati per l'albero di ricerca (struttura di un nodo)

- Lo stato nello spazio degli stati a cui il nodo corrisponde
- Il nodo genitore
- L'operatore che è stato applicato per ottenere il nodo
- La profondità del nodo
- Il costo del cammino dallo stato iniziale al nodo

3.9.2 Algoritmo generale di ricerca

Cerchiamo di definire un algoritmo generale di ricerca per il caso di problemi a stato singolo.

- Occorre definire il tipo di dato **Problem**:
 - Datatype Problem
 - Components: Initial-State, Operators, Goal-Test, Path-CostFunction
- Occorre una funzione per ricavare lo stato iniziale dalla descrizione del problema:
 - Init-State(Problem)
- Occorre una funzione per trasformare una descrizione di stato nel nodo di un albero:
 - Make-Node(State)
- Occorre definire il tipo di dato **Node**:
 - Datatype Node
 - Components: State, Parent-Node, Operator, Depth, Path-Cost
- Occorre una funzione per espandere un nodo:
 - Expand(Node, Problem, [Depth]), genera i successori del nodo se non è stata raggiunta la profondità Depth
- Servono 4 funzioni per gestire delle code di nodi:
 - Make-Queue(Elements), genera una coda contenente Elements
 - Empty(QUEUE), controlla se la coda è vuota
 - Remove-Front(QUEUE), estrae il primo elemento

- Queueing-Fn(Queue, Elements), inserisce Elements nella coda
- Infine serve una funzione per controllare se il nodo coincide con un goal del problema:
 - Goal(Node, Problem)

Le code sono caratterizzate dall'ordine in cui memorizzano i nodi inseriti.
Tre varianti comuni sono:

- First-in, first-out **FIFO** che prende l'elemento più vecchio
- Last-in, first-out o **LIFO** (stack), che prende l'elemento più nuovo
- **Priority queue**, che prende l'elemento della coda con priorità maggiore secondo una certa funzione o criterio

3.10 Valutazione delle strategie di ricerca

- Completezza
- Complessità temporale
- Complessità spaziale
- Ottimalità.

La complessità temporale e spaziale è misurata in termini di:

- b - massimo fattore di diramazione dell'albero di ricerca
- d - profondità della soluzione a costo minimo
- m - massima profondità dello spazio degli stati (può essere infinita).

Strategia: La scelta di quale stato espandere nell'albero di ricerca.

3.11 Strategie non informate

Non richiedono nessuna conoscenza specifica relativa al problema.

Non hanno informazioni aggiuntive oltre a quelle già fornite dalla definizione del problema.

Possono solo generare successori e distinguere uno stato obiettivo da uno stato non obiettivo.

Vantaggio: **Generalità**.

- breadth-first
- depth-first
- depth-first a profondità limitata
- ad approfondimento iterativo

3.11.1 Ricerca in ampiezza (breadth-first search)

Garantisce che ogni nodo di profondità d sia espanso prima di qualsiasi altro nodo di profondità maggiore.

La ricerca in ampiezza è un'istanza dell'algoritmo generale di ricerca su grafo in cui viene scelto per l'espansione il nodo non espanso più vicino alla radice.

Pertanto, i nuovi nodi (che sono sempre più profondi dei loro genitori) vanno infondo alla coda e i vecchi nodi, che sono meno profondi dei nuovi nodi, vengono espansi per primi.

- Ottimalità: è ottimo quando:
 - Il costo è una funzione non decrescente della profondità del nodo
 - Il costo è lo stesso per ciascun operatore
- Completezza: è completo visto che utilizza una strategia sistematica che considera prima le soluzioni di lunghezza 1, poi quelle di lunghezza 2 e così via
- Complessità: Se b è il fattore di ramificazione e d la profondità della soluzione, allora il numero massimo di nodi espansi è: $1 + b + b^2 + b^3 + \dots + b^d$

- Complessità temporale: $O(b^d)$
- Complessità spaziale: $O(b^d)$

In generale i problemi di ricerca di complessità esponenziale non possono essere risolti con metodi non informati tranne che nelle istanze più piccole.

3.11.2 Ricerca a Costo Uniforme (Uniform Cost Search)

L'algoritmo di ricerca a costo uniforme permette di trovare la soluzione a minor costo espandendo tra i nodi della frontiera il nodo caratterizzato dal minor costo (già valutato).

Se il costo è uguale alla profondità del nodo il funzionamento di questo algoritmo è uguale all'algoritmo di ricerca in ampiezza.

Questo algoritmo è **ottimo** quando il costo è una funzione non decrescente della profondità del nodo.

È **completo** poiché utilizza una strategia sistematica simile all'algoritmo di ricerca in ampiezza.

Ha la **stessa complessità** temporale e spaziale dell'algoritmo di ricerca in ampiezza.

Questo algoritmo è in grado di trovare la soluzione più economica, purché sia verificato il requisito: il costo del cammino non deve mai decrescere quando percorriamo il cammino stesso.

Tale restrizione ha senso se il costo di cammino di un nodo viene considerato come somma dei costi degli operatori che determinano il cammino.

3.11.3 Ricerca in Profondità (Depth-First Search)

Ad ogni livello di profondità, espande il primo nodo fino a raggiungere un goal o un punto in cui il nodo non può essere più espanso.

La scelta del nodo da cui partire è arbitraria. Mentre la ricerca in ampiezza utilizza una coda FIFO, la ricerca in profondità utilizza una coda LIFO.

Una coda LIFO significa che il nodo generato più di recente viene scelto per l'espansione.

Questo deve essere il nodo non espanso più profondo perché è uno più profondo del suo genitore, che, a sua volta, era il nodo non espanso più profondo quando è stato selezionato.

- Ottimalità: Non è ottimo perché non usa nessun criterio per favorire le soluzioni a costo minore e quindi la prima soluzione trovata può non essere la soluzione ottima
- Completezza: Non è completo perché può imbattersi in un percorso di profondità infinita
- Se b è il fattore di ramificazione e m è la massima profondità dell'albero, allora il numero massimo di nodi espansi è: $1 + b + b^2 + b^3 + \dots + b^m$
- Complessità temporale: b^m (paragonabile alla ricerca in ampiezza)
- Complessità spaziale: bm (molto modesta: i.e. riduzione a 109 rispetto a ricerca in ampiezza nel caso visto per $d = 12$).

Backtracking: Variante della ricerca di profondità, dove viene generato un solo successore alla volta e ogni nodo parzialmente espanso ricorda quale successore generare successivamente (sarà necessaria solo $O(m)$ memoria anziché $O(bm)$).

Un'altra ricerca all'indietro facilita ancora un altro trucco per risparmiare sui costi e sui tempi: l'idea di generare un successore modificando direttamente la descrizione anziché copiarla prima.

Affinché funzioni, dobbiamo essere in grado di annullare ogni modifica quando torniamo a generare il successivo successore.

3.11.4 Ricerca Limitata in Profondità (Depth-Limited Search)

Per superare il problema dell'algoritmo di ricerca in profondità nell'esplorare alberi con rami con un numero infinito di nodi si può imporre un limite alla profondità dei nodi da espandere.

La scelta del limite dovrebbe essere condizionata dalla previsione sulla profondità a cui si trova la soluzione.

- Ottimalità: non è ottimo per gli stessi motivi della ricerca in profondità
- Completezza: è completo nel caso in cui il limite sia superiore alla profondità della soluzione
- Se b è il fattore di ramificazione e se fissiamo il limite di profondità dell'albero a L , allora il numero massimo di nodi espansi è: $1 + b + b^2 + b^3 + \dots + b^L$

- Complessità temporale: b^L
- Complessità spaziale: bL

3.11.5 Ricerca Iterativa in Profondità (Iterative Deepening Search)

Il problema principale dell'algoritmo di ricerca limitato in profondità è la scelta del limite a cui fermare la ricerca.

Il modo per evitare del tutto questo problema è quello di applicare iterativamente l'algoritmo di ricerca limitato in profondità con limiti crescenti.

In questo caso l'ordine di espansione dei nodi è simile a quello della ricerca in ampiezza con la differenza che alcuni nodi sono espansi più volte.

- Questo algoritmo è completo ed è ottimo quando l'algoritmo di ricerca in ampiezza è ottimo;
- Se b è il fattore di ramificazione e se la profondità della soluzione è d , allora il numero massimo di nodi espansi dall'algoritmo di ricerca iterativa in profondità è: $(d+1)1 + (d)b + (d-1)b^2 + (d-2)b^3 + \dots + 1b^d$
- Complessità temporale: b^d
- Per $b = 10$ e $d = 5$ rendendo iterativo l'algoritmo passiamo da 111111 a 123456 nodi espansi
- Complessità spaziale: bd

Metodo preferibile quando lo spazio di ricerca è grande e la profondità della soluzione non è conosciuta.

3.11.6 Ricerca Bidirezionale (Bidirectional Search)

L'idea è quella di cercare contemporaneamente in avanti dallo stato iniziale e indietro dal goal.

Se b è il fattore di ramificazione e d è la profondità della soluzione, allora la complessità temporale dell'algoritmo di ricerca bidirezionale è: $2b^{(d/2)}$ che può essere approssimato con: $b^{(d/2)}$. Per utilizzarlo bisogna superare alcuni problemi:

- La ricerca all'indietro richiede di generare i predecessori di un nodo. Per fare ciò, gli operatori devono essere reversibili.
- Come si tratta il caso in cui ci sono diversi goal possibili?
- Bisogna controllare in modo efficiente se un nodo è già stato trovato dall'altro metodo di ricerca.
- Che tipo di ricerca è preferibile utilizzare nei due sensi?

3.12 Eliminazione della ripetizione di percorsi

Se non evitiamo di ripetere la ricerca sugli stessi nodi lo spazio di ricerca può essere infinito.

Esistono alcuni controlli per aggirare il problema:

- Viene proibito di ritornare nel nodo che ha generato il nodo corrente
- Viene proibito di ritornare in un nodo antenato del nodo corrente
- Viene proibita la generazione di un nodo già esistente

3.13 Evitare ripetizioni di stati



Figure 15: Albero

Uno spazio degli stati che genera un albero di ricerca esponenziale.

Il lato sinistro mostra lo spazio degli stati, nel quale ci sono due azioni possibili che conducono da A a B , due da B a C e così via.

Il lato destro mostra l'albero di ricerca corrispondente.

3.14 Ricercare le soluzioni: Strategie informate

3.14.1 Metodi di ricerca informati ed esplorazione

I metodi precedenti sono molto inefficienti, visto che non hanno nessun modo per valutare quante possibili soluzioni parziali siano vicine ad un goal del problema.

I metodi di ricerca informati o euristica usano una conoscenza specifica relativa al problema.

3.14.2 Strategie informate

Invece di espandere i nodi in modo qualunque utilizziamo conoscenza euristica sul dominio (funzioni di valutazione) per decidere quale nodo espandere per primo.

Le funzioni di valutazione danno una stima computazionale dello sforzo per raggiungere lo stato finale.

3.14.3 Ricerca lungo il Cammino Migliore (Best-First Search)

E' un'istanza dell'algoritmo generale ricerca su un albero o un grafo in cui il nodo da espandere viene scelto sulla base di una funzione di valutazione.

Il nome "best-search" è impreciso, perché non ci dovrebbe essere la ricerca ma solo una marcia diretta verso il goal.

Tutto quello che si può fare è espandere il nodo che sembra più promettente.

Il tempo speso per valutare mediante una funzione euristica il nodo da espandere deve corrispondere a una riduzione nella dimensione dello spazio esplorato.

Trade-off fra tempo necessario nel risolvere il problema (livello base) e tempo speso nel decidere come risolvere il problema (meta-livello).

Le ricerche non informate non hanno queste attività di meta-livello.

3.15 Ricerca lungo il cammino migliore

Questo metodo consiste nell'uso di una funzione di valutazione che calcola un numero che rappresenta la desiderabilità relativa all'espansione di nodo.

$h(n)$ = costo stimato del cammino più conveniente dal nodo n a un nodo obiettivo

Le pratiche più diffuse in questo metodo di ricerca sono:

- Best-first: Ovvero scegliere come nodo da controllare quello che sembra più desiderabile;
- Queuing Fn: Ordinamento dei nodi in ordine decrescente di desiderabilità.

3.15.1 Greedy search (o ricerca golosa)

Il Greedy search è un metodo che cerca di minimizzare il costo per raggiungere l'obiettivo della ricerca, espandendo ogni volta il nodo più desiderabile.

La desiderabilità di ogni nodo viene assegnata da una funzione, chiamata funzione Euristica ed indicata dalla lettera h che, però, spesso può solo fare una stima del costo del suddetto nodo e non un calcolo deterministico.

Ha molte caratteristiche che lo rendono simile alla ricerca in profondità, quindi non è né ottimo né completo,



Figure 16: Stadi di una ricerca golosa

infatti se consideriamo b è il fattore di ramificazione e m la massima profondità dello spazio di ricerca, nel peggior caso questo algoritmo ha una complessità spaziale e temporale di b^m .

A differenza dell'algoritmo di ricerca in profondità, la complessità temporale e spaziale sono uguali perché tutti i nodi vengono mantenuti in memoria, infine l'efficienza della ricerca dipende dalla bontà dell'euristica.

Questo tipo di ricerca non è detto che trovi la soluzione migliore, ovvero il cammino migliore per arrivare al goal, perché considera solo la distanza dal goal, e non anche il "costo" nel raggiungere il nodo N dalla radice.

3.15.2 Algoritmo A* (ricerca a stella)

- Si propone di minimizzare il costo totale stimato della soluzione
- E' la forma di ricerca a costo uniforme e best-first più diffusa
- Combina i metodi di ricerca a costo uniforme e quello lungo il cammino più vicino al goal (Greedy search)
- Si ottiene un algoritmo, detto A*, che offre i vantaggi di entrambi

Questo algoritmo unisce i 2 vantaggi principali del Greedy Search, ovvero la minimizzazione del costo in termini di distanza dall'obiettivo, e dell'algoritmo di ricerca a costo uniforme, ovvero la minimizzazione del costo del percorso per raggiungere un determinato nodo, e per fare ciò l'algoritmo A* somma le due funzioni di valutazione, la sua formula euristica sarà quindi $f(n) = g(n) + h(n)$, dove:

- $g(n)$ = costo effettivo dalla radice al nodo n ;
- $h(n)$ = costo stimato dal nodo n al nodo obiettivo (goal);
- $f(n)$ = costo totale stimato di un cammino che arriva al goal passando per n .

La ricerca A* usa una euristica ammissibile cioè: $h(n) \leq h^*(n)$ dove $h^*(n)$ è il vero costo da n al goal. Definizione dell'algoritmo A*:

1. Sia L una lista dei nodi iniziali del problema
2. Sia n il nodo di L per cui $g(n) + h(n)$ è minima. Se L è vuota fallisci
3. Se n è il goal fermati e ritorna esso più la strada percorsa per raggiungerlo
4. Altrimenti rimuovi n da L e aggiungi a L tutti i nodi figli di n con label la strada percorsa partendo dal nodo iniziale e torna al punto 2



Figure 17: Stadi si una ricerca a*

L'algoritmo A* è ottimamente efficiente (optimally efficient) per ogni funzione euristica, cioè non esiste alcun altro algoritmo ottimo che espande un numero di nodi minore dell'algoritmo A* a parità di funzione euristica, ma, questo algoritmo non garantisce la soluzione ottima generale perché dipende dalla funzione euristica.

Si può provare che l'algoritmo A* è ottimo e completo se h è una euristica ammissibile, cioè se non sovrastima mai il costo per raggiungere il goal.

L'algoritmo A* è ottimo se $h(n)$ è ammissibile, ad esempio supponiamo di incontrare un nodo obiettivo sub-ottimo $G2$ e sia C^* il costo della soluzione ottima: $h(G2) = 0$ (nodo obiettivo) ed $f(G2) = g(G2) + h(G2) = g(G2) > C^*$.

Consideriamo ora di incontrare n che si trova sul cammino della soluzione ottima: $f(n) = g(n) + h(n) \leq C^*$ quindi da ciò possiamo ricavare che: $f(n) \leq C^* < f(G2)$ in modo che $G2$ non sarà espanso e l'algoritmo dovrà restituire come risultato la soluzione ottima.

Consideriamo ora di usare un grafo invece che un albero come spazio di ricerca, ci serviranno quindi 2 liste:

- Nodi espansi e rimossi dalla lista per evitare di esaminarli nuovamente (nodi chiusi)
- Nodi ancora da esaminare (nodi aperti)

3.15.3 Algoritmo A* per grafi

1. Sia La una lista aperta dei nodi iniziali del problema;
2. Sia n il nodo di La per cui $g(n) + h'(n)$ è minima. Se La è vuota fallisci
3. Se n è il goal fermati e ritorna esso più la strada percorsa per raggiungerlo
4. Altrimenti rimuovi n da La , inseriscilo nella lista dei nodi chiusi Lc e aggiungi a La tutti i nodi figli di n con label la strada percorsa partendo dal nodo iniziale e ritorna al punto 2 (se un nodo figlio è già in La , aggiornalo con la strada migliore che lo connette al nodo iniziale, se invece è già in Lc , non aggiungerlo a La , ma, se il suo costo è migliore, aggiorna il suo costo e i costi dei nodi già espansi che da lui dipendono)

Si può dimostrare che se invece di un albero ricerchiamo in un grafo l'algoritmo A* potrebbe restituire delle soluzioni subottime, questo, perché l'algoritmo scarta un cammino appena scoperto se esso porta ad un nodo già espanso, e può succedere, che venga scartato un cammino migliore di quello trovato in precedenza, e quindi che si perda una soluzione ottima, per evitare questo dobbiamo imporre un requisito supplementare su $h(n)$: il **requisito di consistenza o monotocità**.

Un euristica è **consistente** se, per ogni nodo n e ogni successore n' di n generato da un'azione a , il costo stimato per raggiungere l'obiettivo partendo da n non è superiore al costo per arrivare a n' sommato al costo stimato per andare da lì all'obiettivo, ovvero: $h(n) \leq c(n, a, n') + h(n')$

L'algoritmo A* è completo, perché, espande i nodi nell'ordine dato dai rispettivi valori di f , quindi, se un goal ha valore f^* , l'algoritmo A* è completo se non ci sono infiniti nodi con $f < f^*$.

Anche se l'algoritmo A* è l'algoritmo di ricerca ottimo che espande il minor numero di nodi, in molti casi la complessità è esponenziale in funzione della lunghezza della soluzione, questo perché il numero di nodi espansi cresce esponenzialmente a meno che l'errore della stima h del vero costo h^* non cresca più lentamente del logaritmo del costo reale del percorso: $|h(n) - h^*(n)| \leq O(\log \cdot h^*(n))$

3.15.4 A* Iterativo in Profondità (Iterative Deepening A* - IDA*)

L'algoritmo A* nei casi peggiori richiede molta memoria, per ovviare a questo problema si può usare la tecnica della ricerca iterativa in profondità.

In questo caso il limite non è la profondità del nodo, ma il costo f : ad ogni ciclo vengono espansi solo i nodi con costo minore del limite, che, viene aggiornato per il prossimo ciclo al minimo valore f dei nuovi nodi generati. Anche IDA*, come A*, è completo ed ottimo, inoltre, dato che si basa su una ricerca in profondità, ha una complessità spaziale di bd (fattore di ramificazione $\cdot$ profondità della soluzione).

La complessità temporale invece dipende dal numero di valori che la funzione euristica può assumere, dato che questo numero determina il numero di iterazioni.

Un modo per superare questo limite è incrementare il limite di costo di ε rendendo il numero di cicli proporzionale a $1/\varepsilon$, questo algoritmo è detto ε -ammissibile; non è ottimo perché può ritornare una soluzione che può essere peggio di quella ottima, ma può avere un costo massimo di ε .

3.15.5 SMA* (Simplified Memory-Bounded A*)

SMA* espande sempre la foglia migliore finché la memoria è piena, a questo punto deve cancellare un nodo in memoria, ed elimina il peggiore (quello col valore f più elevato), però, memorizza nel nodo padre il valore del nodo rimosso in modo da poter rigenerare il sotto-albero dimenticato quando tutti gli altri cammini sono falliti.



Figure 18: SMA*

3.15.5.1 Esempio di ricerca con SMA*

(riferito all'albero di esempio precedente)



Figure 19: Esempio SMA*

1. Aggiungo B e G ad A , quello col costo minimo è G , allora scarto B , mantenendo in memoria A , e proseguo espandendo G
2. Espando H ma non posso proseguire
3. Espando I
4. I è l'obiettivo ma non è detto che sia il migliore
5. Rigenero a partire da A il nodo B per verificare che ci siano dei percorsi migliori (ad esempio in questo caso l'obiettivo D ha costo più basso di I)

I nodi rimossi sono rigenerati solo nel caso in cui tutti i percorsi attivi assumano un valore peggiore del valore di un percorso stimato rimosso (ad esempio in questo caso il nodo B viene rigenerato perché ha costo 15, mentre il percorso che porta all'obiettivo I che abbiamo trovato ha un costo di 24).

3.15.5.2 Proprietà algoritmo SMA*

- Utilizza la memoria a disposizione
- Evita di rigenerare nodi in base a quanta memoria ha a disposizione
- È completo se la memoria è sufficiente a contenere il percorso che comprende la soluzione
- È ottimo se la memoria è sufficiente a contenere il percorso della soluzione, altrimenti ritorna la migliore soluzione che può raggiungere con la memoria a disposizione
- È ottimamente efficiente se può memorizzare tutto l'albero di ricerca

Quindi se la memoria è sufficiente SMA* risolve dei problemi più difficili rispetto ad A*, ma, con problemi troppo difficili, e che sarebbero risolvibili se si avesse a disposizione memoria illimitata, la necessità di ripetere più volte la generazione di nodi rende il problema intrattabile dal punto di vista temporale.

Nonostante non ci sia una teoria che formalizzi il problema del compromesso tra tempo e memoria, non sembra esistere una soluzione efficace ad esso se non quella di rinunciare al requisito dell'ottimalità ed insegnare alla macchina ad imparare meglio aggiungendo uno spazio di metalivello al dominio del problema.

Questo perché per problemi più difficili, ci saranno molti passi falsi e un algoritmo di apprendimento ad un metalivello può imparare da queste esperienze per evitare di esplorare assemblaggi di sottoalberi poco promettenti, queste tecniche sono fondamentali per il successo e cercano di minimizzare il costo totale della soluzione del problema, sia dal punto di vista del costo del cammino, che, anche, da quello del costo computazionale che serve per ottenerlo.

3.16 Scelta euristica

Consideriamo un problema molto semplice come 8-puzzle:

E' possibile definire differenti funzioni euristiche.

Una soluzione media richiede circa 20 mosse, considerando che da ogni stato del puzzle si possono generare in media 3 nuovi stati (fattore di ramificazione), una ricerca in profondità esaustiva esplorerebbe un numero enorme di possibili configurazioni (circa 3.5 miliardi).

Possiamo ridurre il numero di nodi da esplorare, ad esempio evitando di visitare più volte lo stesso stato,



Figure 20: Esempio 8-puzzle

tuttavia, anche eliminando i duplicati, il numero di stati possibili rimane molto elevato ($9! = 362.880$), quindi, per risolvere efficacemente l'8-puzzle, è necessario adottare delle buone euristiche, che ci permettano di guidare la ricerca verso soluzioni più promettenti e rendano il problema risolvibile in tempi accettabili.

Un esempio di possibili euristiche sono: h_1 numero di tessere che sono fuori posto ($h_1 = 7$)
 h_2 = la somma delle distanze orizzontali e verticali (distanza di Manhattan) dalle posizioni che le tessere devono assumere nella configurazione obiettivo ($h_2 = 2 + 3 + 3 + 2 + 4 + 2 + 0 + 2 = 18$)

Confronto dei costi di IDS (iterative-deepening search) e A^* con h_1 e h_2 (numero medio di nodi espansi in 100 esperimenti):

d	Search Cost (Node Generated)			Effective Branching Factor		
	IDS	$A'(h_1)$	$A'(h_2)$	IDS	$A'(h_1)$	$A'(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6084	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	-	539	113	-	1.44	1.23
16	-	1301	211	-	1.45	1.25
18	-	3056	363	-	1.46	1.26
20	-	7276	676	-	1.47	1.27
22	-	18094	1219	-	1.48	1.28
24	-	39135	1641	-	1.48	1.26

Dai risultati sperimentali si ricava che h_2 è una stima migliore del costo per raggiungere il goal e domina sull'altra.

La dominanza di una funzione euristica h_2 rispetto a h_1 si traduce in una maggiore efficienza nella ricerca A^* , poiché A^* con h_2 non espanderà mai più nodi rispetto ad A^* con h_1 (eccetto per alcuni nodi con $f(n) = C^*$). Questo accade perché:

- Ogni nodo con $f(n) = C^*$ sarà sicuramente espanso, il che equivale a dire che ogni nodo con $h(n) < C^* - g(n)$ sarà espanso
- Poiché h_2 è almeno pari a h_1 per ogni nodo, ogni nodo espanso da A^* con h_2 sarà anche espanso con h_1 , mentre h_1 potrebbe espandere ulteriori nodi.

Di conseguenza, è generalmente vantaggioso usare un'euristica con valori più alti (dominante), purché sia coerente e il suo calcolo non sia troppo oneroso in termini di tempo.

Le due euristiche esaminate per l'8-puzzle sono le funzioni di valutazione esatte della distanza dal goal per due versioni semplificate del problema, quindi si può dire che le soluzioni di un problema P_r , ottenuto rilassando (togliendo restrizioni alle) le regole di un problema P , sono delle buone euristiche per P .

3.16.1 Generazione automatica di euristiche ammissibili attraverso il rilassamento dei problemi

Se la definizione del problema è espressa in un linguaggio formale, è possibile generare varianti semplificate (rilassamenti) eliminando alcune restrizioni.

Questi rilassamenti permettono di ottenere euristiche ammissibili, poiché il costo di una soluzione ottima per un problema rilassato è sempre minore o uguale al costo di una soluzione ottima per il problema originale.

Ad esempio, nel problema dell'8-puzzle, un tassello può muoversi da una posizione A a B solo se A e B sono adiacenti e B è vuota. rimuovendo una o entrambe le condizioni, si ottengono diversi problemi rilassati:

1. Un tassello può muoversi se A e B sono adiacenti (da cui deriva la distanza di Manhattan)
2. Un tassello può muoversi da A a B se B è vuota
3. Un tassello può muoversi da A a B senza restrizioni

È molto importante che i problemi rilassati, generati da questa tecnica, possano essere risolti essenzialmente senza ricerca, perché le regole rilassate consentono di scomporre il problema in sotto-problemi indipendenti e se il problema rilassato è difficile da risolvere, allora sarà costoso ottenere i valori dell'euristica corrispondente.

Un problema con la generazione di nuove funzioni euristiche è che spesso non si riesce a ottenere una singola euristica “chiaramente migliore”.

Quando si dispone di un insieme di euristiche ammissibili per un problema $(h_1, h_2, \dots, h_m)$ senza una dominante, è possibile definire un'euristica composita che prenda il massimo tra queste: $h(n) = \max\{h_1(n), \dots, h_m(n)\}$, questa euristica composita è ammissibile e consistente, in genere domina le singole euristiche.

Se un problema è formalmente definito, una macchina può generare euristiche basandosi su tale definizione. Ad esempio, il programma ABSOLVER (1993) ha sviluppato la prima euristica utile per il cubo di Rubik. Altri metodi per ottenere euristiche includono l'uso di informazioni statistiche e l'apprendimento dall'esperienza tramite reti neurali o alberi decisionali eccetera. . .

4 Agenti Risolutori di Problemi - Ottimizzazione

4.1 Algoritmi di ricerca locale e problemi di ottimizzazione

4.1.1 Algoritmi di miglioramento iterativo

Per molti problemi il cammino verso l'obiettivo è irrilevante, ciò che conta è la configurazione finale e non l'ordine con cui è stata raggiunta. Si possono considerare allora algoritmi di ricerca locale che operano su un singolo stato corrente invece che su cammini multipli.

Più precisamente:

- Gli algoritmi visti fino ad ora generano una soluzione ex-novo aggiungendo ad una situazione di partenza delle componenti in un particolare ordine fino a “costruire” la soluzione completa.
- Gli algoritmi di ricerca locale partono da una soluzione iniziale e iterativamente cercano di rimpiazzare la soluzione corrente con una “migliore” in un intorno della soluzione corrente. In questi casi non interessa la “strada” per raggiungere l'obiettivo.

Ci sono dei problemi (es. 8 regine) per cui la descrizione dello stato contiene tutte le informazioni necessarie per la soluzione indipendentemente dal percorso che ha condotto a quello stato.

In questi casi l'idea generale per la soluzione può essere quella di partire da uno stato iniziale “completo” (es. 8 regine già sulla scacchiera) e poi modificarlo per raggiungere una soluzione (o una soluzione migliore, se esiste). Questi metodi fanno parte dei cosiddetti algoritmi di ottimizzazione il cui scopo è massimizzare (minimizzare) il valore di una funzione di valutazione che misura la bontà (costo) di una soluzione.

La ricerca locale, quindi, è basata sull'esplorazione iterativa di “soluzioni vicine”, che possono migliorare la soluzione corrente mediante modifiche locali (non è detto che sia ottima globalmente).

4.1.1.1 Struttura dei “vicini”

Una struttura dei “vicini” è una funzione F che assegna a ogni soluzione s dell'insieme di soluzioni S un insieme di soluzioni $N(s)$ sottoinsieme di S .

La scelta della funzione F è fondamentale per l'efficienza del sistema e definisce l'insieme delle soluzioni che possono essere raggiunte da s in un singolo passo di ricerca dell'algoritmo.

Questi algoritmi partono con una soluzione di tentativo generata in modo casuale e cerca di migliorare la soluzione corrente muovendosi verso i vicini.

Se fra i vicini c'è una soluzione migliore, essa va a sostituire quella corrente, altrimenti termina in un massimo (minimo) locale di ricerca.

Questo metodo può essere rappresentato come il movimento su una superficie che rappresenta lo spazio degli stati alla ricerca della vetta più elevata o più bassa.

- Un algoritmo di ricerca locale completo trova sempre un minimo/massimo
- Un algoritmo di ricerca locale ottimo trova sempre un minimo/massimo globale

4.1.2 Ricerca in Salita (Hill Climbing)

L'algoritmo di ricerca in salita è un algoritmo ciclico che, ad ogni passo, raggiunge una posizione migliore all'interno di un'area di ricerca, se esiste, seguendo la direzione di massima pendenza.

Hill climbing viene talvolta chiamata ricerca locale golosa perché cattura uno stato buon vicino senza pensare in anticipo a dove andare dopo.

Questo algoritmo deve affrontare tre tipi di situazioni che lo possono “bloccare”.

Nella rappresentazione dello spazio di ricerca come una superficie, corrispondono al raggiungimento di:

- Massimi locali
- Pianori
- Crinali



Figure 21: Un "paesaggio" dello spazio degli stati unidimensionale in cui l'elevazione corrisponde alla funzione obiettivo. L'obiettivo è trovare il massimo globale. La ricerca hill-climbing modifica lo stato attuale per cercare di migliorarlo, come mostrato dalla freccia. Le varie caratteristiche topografiche sono definite nel testo.

Il metodo per superare questi problemi è fare ripartire la ricerca da un altro punto scelto a caso.

Questa variante ad ogni iterazione mantiene in memoria il risultato migliore e, periodicamente o al verificarsi di condizioni di "blocco", riparte con una nuova iterazione da uno stato scelto in modo random.

Dopo un certo numero fisso di iterazioni o dopo che per un certo numero di iterazioni questo risultato non viene migliorato, l'algoritmo termina ritornando il risultato migliore.

Se il problema è NP-completo (generalmente un problema intrattabile), la soluzione non può che essere fornita in un tempo esponenziale poiché, per tale classe di problemi, lo spazio degli stati ha un numero esponenziale di massimi locali.

4.1.3 Simulated Annealing

L'algoritmo di simulated annealing ha un comportamento molto simile all'algoritmo di ricerca in salita.

La differenza consiste nell'eseguire, ad ogni passo, una mossa casuale.

Se la mossa migliora la situazione, allora viene sempre eseguita, altrimenti viene eseguita con probabilità $e^{\Delta E/T}$ dove:

- ΔE è il peggioramento (< 0) causato dall'esecuzione della mossa
- T è un valore che tende a zero con l'aumentare dei cicli e quindi rende sempre meno probabile l'esecuzione di mosse negative.

4.1.3.1 Esempio

Se lasciamo rotolare la palla, si fermerà al minimo locale.

Se scuotiamo la superficie, possiamo far rimbalzare la palla fuori dal minimo locale.

Il trucco è scuotere abbastanza forte da far rimbalzare la palla fuori dai minimi locali ma non abbastanza da rimuoverla dal minimo globale.

L'algoritmo di simulated-annealing consiste nell'iniziare agitando forte (cioè ad alta temperatura) e quindi ridurre gradualmente l'intensità dell'agitazione (cioè abbassare la temperatura).

In pratica cerca di evitare il problema dei massimi locali, seguendo la strategia in salita, ma ogni tanto fa un passo che non porta a un incremento in salita.

4.1.4 Meta-euristiche

Si definiscono meta-euristiche l'insieme di algoritmi, tecniche e studi relativi all'applicazione di criteri euristici per risolvere problemi di ottimizzazione (quindi migliorando la ricerca locale con criteri abbastanza generali).

4.1.4.1 Meta-conoscenza

Meta-conoscenza = Conoscenza sulla conoscenza.

Può risolvere i problemi precedentemente citati (vasta applicazione non solo per il controllo).

VANTAGGI:

- Il sistema è maggiormente flessibile e modificabile; un cambiamento nella strategia di controllo implica semplicemente il cambiamento di alcune meta-regole
- La strategia di controllo è semplice da capire e descrivere
- Potenti meccanismi di spiegazione del proprio comportamento

Altri esempi di Meta-Euristiche:

- **ANT colony optimization:**
 - Ispirata al comportamento di colonie di insetti. Tali insetti mostrano capacità globali nel trovare, ad esempio, il cammino migliore per arrivare al cibo dal formicaio (algoritmi di ricerca cooperativi)
- **Tabu search:**
 - La sua caratteristica principale è l'utilizzo di una memoria per guidare il processo di ricerca in modo da evitare la ripetizione di stati già esplorati
- **Algoritmi genetici:**
 - Algoritmi evolutivi ispirati ai modelli dell'evoluzione delle specie in natura. Utilizzano il principio della selezione naturale che favorisce gli individui di una popolazione che sono più adatti ad uno specifico ambiente per sopravvivere e riprodursi
 - Ogni individuo rappresenta una soluzione con il corrispondente valore della funzione di valutazione (fitness). I tre principali operatori sono: selezione, mutazione e ricombinazione.

4.1.4.2 Meta-euristiche e scelta degli operatori

Le meta-euristiche sono strategie generali usate per guidare la ricerca di soluzioni ottimali o quasi ottimali nei problemi complessi.

Il testo nell'immagine parla di come le regole e i task possano essere inseriti in un'agenda e selezionati in base a determinati criteri.

Due modi di influenza nella scelta delle regole:

1. Influenza implicita

- Insita nel controllo del sistema
- L'utente non interviene direttamente ma il sistema prende decisioni basate su criteri predefiniti

2. Influenza esplicita

- L'utente assegna priorità numeriche alle regole
- Il sistema sceglie sempre la regola con la priorità più alta

Fattori che influenzano la selezione delle regole:

- Lo stato della memoria di lavoro (il contesto attuale del problema)
- La presenza di particolari regole applicabili (dipende dalla situazione corrente)
- L'esecuzione precedente di regole correlate (l'algoritmo può apprendere o adattarsi)
- L'andamento dell'esecuzione in base a scelte precedenti (strategie basate sull'esperienza)

Questa logica è tipica dei sistemi basati sulla conoscenza e dell'intelligenza artificiale simbolica, dove si usano regole per prendere decisioni.

È anche una tecnica usata negli algoritmi di pianificazione e nei motori inferenziali.

4.2 Problemi con vincoli

Problemi con vincoli (CSP-Constraint Satisfaction Problems) cercano di trovare soluzioni efficienti andando a incrementare la definizione di un singolo stato.

Usiamo una rappresentazione fattorizzata per ogni stato: un insieme di variabili, ognuna delle quali ha un valore.

Un problema è risolto quando ogni variabile ha un valore che soddisfa tutti i vincoli sulla variabile.

Qui l'idea è quella di eliminare in una volta grandi porzioni dello spazio di ricerca identificando le combinazioni di variabile/valore che violano i vincoli.

4.2.1 Formulazione dei problemi CSP

Problema: descritto da tre componenti:

1. $X = X_1, X_2, \dots, X_n$ insieme di variabili
2. $D = D_1, D_2, \dots, D_n$ insieme di domini dove: $D_i = v_1, \dots, v_k$ è l'insieme dei valori possibili per X_i
3. $C = C_1, C_2, \dots, C_m$ insieme di vincoli (relazioni tra le variabili)

Stato: un assegnamento [parziale/completo] di valori a variabili $\{X_i = v_i, X_j = v_j \dots\}$

Stato iniziale = $\{\}$

Azioni: assegnamento di un valore a una variabile.

Soluzione (goal test): un assegnamento completo (le variabili hanno tutte un valore) e consistente (i vincoli sono tutti soddisfatti).

Come esprimere i vincoli

- Forma generale: $\langle \text{ambito}, \text{relazione} \rangle$
 - Ambito: tupla di variabili che partecipano nel vincolo
 - Relazione: un insieme di tuple di valori

Se si dispone già di un algoritmo efficiente di risoluzione per un generico problema CSP, è più facile risolvere un problema utilizzando questo che progettare una soluzione personalizzata utilizzando un'altra tecnica di ricerca. Inoltre, i risolutori CSP possono essere più veloci di una ricerca nello spazio degli stati perché il risolutore CSP può eliminare rapidamente grandi campioni dello spazio di ricerca.

4.2.2 Tipi di vincoli

I vincoli possono essere:

- Unari (es. “ x pari”);
- Binari (es. “ $x > y$ ”);
- Di grado superiore (es. $x + y = z$) comunque riconducibili a vincoli binari, introducendo più variabili.

Vincoli globali:

- Es. Tutti Valori Diversi, come nelle righe e colonne del Sudoku;
- Es. Ogni lettera una cifra distinta nella cripto-aritmetica.

Finora potevamo solo ricercare la soluzione nel grafo degli stati.

Adesso possiamo fare anche un minimo di ragionamento che ci porta a restringere i domini e quindi al limitare la ricerca: **Propagazione di vincoli**.

4.2.3 Ricerca in problemi CSP

Ad ogni passo si assegna una variabile, la massima profondità della ricerca è fissata dal numero di variabili n . L'ampiezza dello spazio di ricerca è $|D_1| \times |D_2| \times \dots \times |D_n|$ è la cardinalità del dominio di X_i .

Il fattore di diramazione:

- Teoricamente pari a nd al primo passo; $(n-1)d$ al secondo ... le foglie sarebbero $n! \times d^n$;
- Riduzione drastica dello spazio di ricerca dovuta al fatto che il goal-test è commutativo (l'ordine non è importante).

4.2.4 Strategie di ricerca

- **Generate and Test:** Si genera in profondità una soluzione e si controlla: poco efficiente
- **Ricerca con backtracking (BT):** Ad ogni passo si assegna una variabile e si controllano i vincoli, si torna indietro in caso di fallimento

Nel backtracking si ha il controllo anticipato della violazione dei vincoli, in quanto è inutile andare avanti fino alla fine e poi controllare.

4.2.4.1 Problemi nell'uso del backtracking

- **Trashing:** Continua a ripetere le stesse assegnazioni di variabili fallite
- **Inefficienza:** Può esplorare aree dello spazio di ricerca che probabilmente non avranno successo

4.2.5 Scelta delle variabili

- **MRV (Minimum Remaining Values):** Scegliere la variabile che ha meno valori legali [residui], la variabile più vincolata. Si scoprono prima i fallimenti (fail first);
- **Euristica del grado:** Scegliere la variabile coinvolta in più vincoli con le altre variabili (la variabile più vincolante o di grado maggiore). Da usare a parità di MRV.

4.2.6 Scelta dei valori

Una volta scelta la variabile come scegliere il valore da assegnare?

1. **Valore meno vincolante:** quello che esclude meno valori per le altre variabili direttamente collegate con la variabile scelta
 - Meglio valutare prima un assegnamento che ha più probabilità di successo
 - Se volessimo tutte le soluzioni l'ordine non sarebbe importante

4.2.7 Propagazione di vincoli

- **Verifica in avanti (Forward Checking o FC):**
 - Assegnato un valore ad una variabile si possono eliminare i valori incompatibili per le altre variabili direttamente collegate da vincoli (non si itera)
- **Consistenza di nodo e d'arco:**
 - Si restringono i valori dei domini delle variabili tenendo conto dei vincoli unari e binari su tutto il grafo (si itera finché tutti i nodi ed archi sono consistenti).

4.2.7.1 Forward checking

- Tieni traccia dei valori legali rimanenti per le variabili non assegnate
- Termina la ricerca quando una variabile non ha valori legali



Figure 22: Forward checking

Forward checking propaga le informazioni da variabili assegnate a variabili non assegnate, ma non fornisce il rilevamento anticipato per tutti gli errori.
NT e SA non possono essere blu!

4.2.7.2 Consistenza di nodo

- Un nodo è consistente se tutti i valori nel suo dominio soddisfano i vincoli unari
- Una rete di vincoli è nodo-consistente se tutti i suoi nodi sono consistenti
- I vincoli unari quindi possono essere risolti restringendo opportunamente i domini delle variabili

4.2.7.3 Consistenza d'arco

Nel grafo di vincoli, un arco tra X e Y , (X, Y) , è consistente rispetto a X , se per ogni valore x di X c'è almeno un valore y di Y consistente con x .

Si dice anche: X è arco-consistente rispetto a Y .

Se un arco (X, Y) non è consistente rispetto a X (o Y) si cerca di renderlo tale, rimuovendo valori dal dominio di X (o di Y).

Se il dominio di una variabile viene modificato vanno ricontrollati tutti gli archi con i vicini.

Si itera fino a che tutte le variabili sono arco-consistenti.



Figure 23: Consistenza d'arco

La forma più semplice di propagazione che rende ogni arco coerente.

$X \rightarrow Y$ è consistente se e solo se per ogni valore di x di X .

C'è un valore consentito y in Y .

Se X perde un valore, i vicini di X devono essere ricontrollati

La coerenza dell'arco rileva gli errori prima del semplice forward checking.

WA=red and Q=green viene subito riconosciuto come un vicolo cieco, cioè un'istanza parziale impossibile.

L'algoritmo di consistenza d'arco può essere eseguito sia prima che dopo ogni assegnazione.

4.2.8 Complessità di AC-3

Devono essere controllati tutti gli archi (sia c il # archi).

Se durante il controllo di un arco (X, Y) il dominio di X si restringe vanno ricontrollati tutti gli archi adiacenti (Z, X) con $Z \neq Y$.

Il controllo di consistenza di un arco ha complessità d^2 , se d è la dimensione massima dei domini.

Un arco deve essere controllato al massimo d volte.

Complessità: $O(c d^3)$... polinomiale.

4.2.9 La risoluzione di un CSP richiede ancora la ricerca

- Ricerca:
 - Può trovare buone soluzioni, ma deve esaminare le non-soluzioni lungo il cammino
- Constraint Propagation:
 - Può escludere non-soluzioni, ma questo non è lo stesso che trovare soluzioni
- Si possono usare entrambi – constraint propagation & search:
 - Eseguendo la propagazione del vincolo in ogni fase della ricerca

4.2.10 Backtracking “cronologico”

Supponiamo di avere Q=red, NSW=green, V=blue, T=red, si cerca di assegnare SA.

Il fallimento genera un backtracking “cronologico” e provando con tutti i valori alternativi si continua a fallire.



Figure 24: Backtracking cronologico

4.2.11 Backtracking “intelligente”

Si considerano alternative solo per le variabili che hanno causato il fallimento (Q, NSW, V), l'insieme dei conflitti.

Backtracking guidato dalle dipendenze.

5 Agente Logico

5.1 Introduzione al ragionamento automatico logico con la logica proposizionale

Si progettano agenti che possono costruire rappresentazioni del mondo, applicare processi di inferenza per derivare nuove rappresentazioni e usarle per dedurre cosa fare, nel paper “Programs with Common Sense” (1968), John McCarthy esprime l'idea di agenti che usano il ragionamento logico per mediare tra percetti e azioni e l'indipendenza fra il ragionamento logico e l'algoritmo che lo implementa.

Sembra che gli uomini conoscano le cose a priori e ciò che sanno li aiuta ad agire.

Infatti, l'intelligenza degli esseri umani non passa attraverso meccanismi puramente reattivi, ma tramite processi di ragionamento, che operano su rappresentazioni interne della conoscenza.

Nell'IA, questo approccio all'intelligenza è incarnato in agenti basati sulla conoscenza, ed è alla base dei nostri sistemi intelligenti.

Gli agenti “problem-solving” sanno le cose, ma solo in un senso molto limitato e non flessibile, ad esempio, nel modello di transizione per l'8-puzzle, la conoscenza di ciò che fa una azione è nascosta all'interno del codice specifico del dominio della funzione RISULTATO, che può quindi essere utilizzato per prevedere l'esito delle azioni ma non per dedurre che due tessere non possono occupare lo stesso spazio.

Anche le rappresentazioni usate dagli agenti di risoluzione dei problemi sono molto limitanti, infatti, in un ambiente parzialmente osservabile, l'unica scelta di un agente per rappresentare ciò che sa sullo stato attuale è elencare tutti i possibili stati concreti: una prospettiva senza speranza in ambienti vasti.

CSP ha introdotto l'idea di rappresentare gli stati come assegnazioni di valori a variabili; consentendo, quindi, ad alcune parti dell'agente di lavorare in modo indipendente dal dominio ed algoritmi più efficienti.

La logica, invece, è una classe generale di rappresentazioni per supportare agenti basati sulla conoscenza, quindi, gli agenti logici, hanno una rappresentazione esplicita della conoscenza con cui è possibile ragionare (ovvero una rappresentazione interna strutturata del modello del dominio) e sono in grado di manipolare questa conoscenza per dedurre cose nuove a “livello di conoscenza”.

Gli agenti basati su conoscenza possono:

- Accettare nuovi compiti sotto forma di obiettivi descritti in modo esplicito
- Acquisire rapidamente le competenze ricevendo informazioni o imparando nuove conoscenze sull'ambiente
- Adattarsi ai cambiamenti dell'ambiente aggiornando le relative conoscenze

La conoscenza e il ragionamento hanno un ruolo cruciale nella gestione degli ambienti parzialmente osservabili; attraverso il ragionamento sulla conoscenza generale e lo stato corrente si possono dedurre aspetti nascosti dello stato del mondo, diventare più flessibili ed essere inseriti in un mondo che evolve.

5.1.1 Studio degli stati nascosti

Ad esempio, prima di agire con una cura, il medico deve diagnosticare la malattia, ovvero dedurre dai sintomi, qualcosa che è nascosto e non direttamente osservabile; in questo caso una parte della conoscenza del medico

ha la forma di regole apprese dai libri, ma, un'altra parte è composta da schemi di associazione che lui stesso potrebbe trovare difficile descrivere in modo esplicito.

Anche l'utilizzo del linguaggio naturale richiede la deduzione di stati nascosti, ad esempio, con la frase "John vide il diamante attraverso il vetro e lo desiderò" noi ragioniamo, forse inconsciamente, in base alla nostra conoscenza del valore relativo dei materiali e deduciamo che "lo" si riferisce al diamante e non al vetro.

Inoltre il linguaggio naturale richiede anche la capacità di astrazione della descrizione simbolica, infatti, molti fenomeni linguistici sono sistemici (ovvero pertinenti solo ad uno specifico sistema) e la loro complessità trascende la possibilità di un semplice pattern-matching.

5.1.2 Sistematica

La Sistematica fa riferimento ad un'estensione concettuale, metodologica e culturale, della Teoria Generale dei Sistemi, in ambito scientifico è un settore di studi, a cavallo tra matematica e scienze naturali, che si occupa dell'analisi delle proprietà e della costituzione di un sistema in quanto tale; in poche parole, si riferisce ai principi e alle applicazioni dei metodi basati sul concetto di sistema, sull'interazione delle varie parti e sull'auto-organizzazione delle stesse.

La teoria si compone essenzialmente della teoria dei sistemi dinamici (semplici e complessi) e della teoria del controllo ed è alla base di diverse discipline come l'automatica, la robotica eccetera eccetera...

Il ragionamento ci permette di trattare un numero virtualmente infinito di frasi, utilizzando un deposito finito di conoscenze comuni, gli agenti risolutori di problemi, però, trovano difficile gestire questo tipo di ambiguità, perché la loro rappresentazione delle contingenze è intrinsecamente esponenziale, quindi è necessario studiare agenti basati su una conoscenza più simile a quella umana.

Possiamo individuare tre modi di ragionare tipicamente umani:

- Il pensiero logico: deduttivo (dall'idea generale ai casi particolari) o induttivo (dai casi particolari all'idea generale), è molto utile per sistemare, organizzare e definire
- Il pensiero analogico: mette in relazione una cosa con l'altra, cercando analogie o diversità, è utile per aprire nuove vie con l'uso di metafore, visualizzazioni e similitudini
- Il pensiero sistemico: realizza visioni sintetiche di situazioni molto complesse, ricche di dati ed interdipendenti l'una dall'altra (Presuppone che si costruisca all'interno della mente uno scenario globale, che rappresenti il sistema di riferimento su cui poi si va a lavorare localmente)

5.1.3 Basi di Conoscenza (knowledge Bases)

Le basi di conoscenza sono un insieme di dichiarazioni (sentences) scritte in un linguaggio formale, e permettono di usare un approccio dichiarativo per la costruzione di un agente, ovvero (contrariamente all'approccio procedurale in cui la conoscenza del progettista viene codificata direttamente nel programma sotto forma, appunto, di codice) si può dire all'agente ciò che deve sapere (conoscenza iniziale) per poi interrogarlo per sapere cosa fare, con le risposte, dell'agente, che discendono dalla base di conoscenza che noi gli abbiamo fornito.

La base di conoscenza deve prevedere meccanismi per aggiungere nuove formule e per le dichiarazioni, e sia la fase in cui si forniscono all'agente le conoscenze (fase TELL) che quella di interrogazione (fase ASK) possono comportare un processo di inferenza, ovvero la derivazione di nuove formule a partire da quelle conosciute.

Gli agenti possono essere visualizzati:

- A livello della loro conoscenza (Knowledge level), ovvero ciò che sanno, indipendentemente da come lo si è implementato
- A livello di codifica di procedure (implementation level), cioè, strutture dei dati in KB (Knowledge Bases) e algoritmi che le manipolano

5.1.4 Agente logico basato su una KB

Algoritmo specializzato = ragionamento di uso generale + conoscenza specializzata

In questo modo è possibile:



Figure 25: Basi di conoscenza

- Date alcune condizioni dedurre il goal
- Dato il goal dedurre le possibili cause

In ogni caso è però impossibile riconoscere il falso.

Un agente di questo tipo deve essere in grado di:

- Rappresentare stati, azioni, etc. . .
- Incorporare nuove percezioni
- Aggiornare la rappresentazione interna del mondo
- Dedurre proprietà nascoste del mondo
- Dedurre azioni appropriate

L'agente è indipendente dal livello dell'implementazione e ci interessa sapere cosa fa a livello di conoscenza non i dettagli di come la ottiene, infine, un agente di successo, deve combinare sia elementi **dichiarativi** che **procedurali** e non può basarsi su solo uno dei 2 approcci.

Per risolvere qualsiasi tipo di problema di IA ci serve definire 2 cose:

- la conoscenza riguardante il problema
- dei metodi per manipolarla

Per la definizione della conoscenza necessitiamo di definire 2 livelli:

- **livello della conoscenza:** definizione dei fatti che vogliamo rappresentare
- **livello simbolico:** rappresentazione di questi fatti attraverso i cosiddetti metodi per la rappresentazione della conoscenza

Uno di questi linguaggi per la rappresentazione della conoscenza è la **logica**, che, non è utilizzata solo in intelligenza artificiale ma anche:

- Per la progettazione di reti logiche
- Nei DB relazionali, come linguaggio per l'interrogazione intelligente
- Come linguaggio di specifica di programmi per eseguire prove formali di correttezza
- Come un vero e proprio linguaggio di programmazione (programmazione logica e PROLOG)

5.1.5 Logica moderna o logica matematica

La logica così come la intendiamo è una disciplina relativamente recente, le cui origini si possono far risalire ai lavori di George Boole (1854, logica di Boole) e di Gottlob Frege (1879), nella storia del pensiero occidentale, tuttavia, la tradizione della logica si può far risalire alla Grecia classica, dove, l'obiettivo di questa disciplina, per i filosofi greci, era stabilire sotto quali condizioni si possa dire che un'argomentazione è valida.

Ciò che caratterizza la logica moderna è l'utilizzo di **metodi formali** (proprio per questo è anche detta **logica formale** o **logica simbolica**), inoltre è anche detta **logica matematica** per due motivi:

- Utilizza un approccio di tipo matematico
- Uno dei suoi obiettivi è chiarire alcuni problemi concernenti i fondamenti della matematica

La logica e i metodi formali sono necessari sia per realizzare sistemi informatici ‘intelligenti’ (che hanno capacità di ragionamento), sia per analizzare e progettare sistemi che siano dimostrabilmente corretti e sicuri, quindi, la logica, è la scienza che fornisce all’uomo gli strumenti indispensabili per verificare con sicurezza la correttezza dei ragionamenti.

La logica fornisce strumenti formali per:

- **Esprimere** le deduzioni in termini di operazioni su espressioni simboliche
- **Dedurre** conseguenze da certe premesse
- **Studiare** la verità o falsità di certe proposizioni data la verità o falsità di altre proposizioni
- **Stabilire** la consistenza e la validità di una data teoria

Il modo per inferire nuova conoscenza in logica è basato sul **ragionamento deduttivo**, ovvero, un ragionamento che, condotto correttamente, mantiene la **veridicità delle premesse**, e, se queste conoscenze da cui partiamo (le premesse appunto), allora anche le nuove conoscenze che otteniamo in base a queste (le **conclusioni**) sono vere.

Un esempio tipico di ragionamento deduttivo è il Sillogismo (o ragionamento concatenato): Gli uomini sono mortali, Socrate è un uomo, allora Socrate è mortale

Il sillogismo fu introdotto da Aristotele che osservò che la veridicità delle premesse determina quella della conclusione.

Ogni sillogismo si compone di tre parti:

1. **Premessa maggiore**: contiene il termine ”maggiori” (predicato della conclusione).
 - Es.: Ogni animale è mortale.
2. **Premessa minore**: contiene il termine ”minore” (soggetto della conclusione).
 - Es.: Ogni uomo è animale.
3. **Conclusione**: collega i termini maggiore e minore.
 - Es.: Ogni uomo è mortale.

Aristotele classificò i termini in base al rapporto tra contenente e contenuto, dimostrando che una corretta struttura garantisce la validità della conclusione.

5.2 Logica

Sillogismo: Tipico esempio di ragionamento deduttivo.

Logica matematica: Un ragionamento è corretto/valido se e solo se non esiste il caso dove le proposizioni sono vere e la conclusione falsa.

La conclusione è la conseguenza logica delle premesse.

Logica moderna: Pone il compito di rappresentare in modo formale la corrispondenza tra espressioni linguistiche e significato inteso.

Nel processo di traduzione (formalizzazione) dal linguaggio naturale alla rappresentazione proporzionale, le relazioni studiate riguardano la struttura dei ragionamenti e non il ”senso” comune delle formule nell’ambito discorsivo di riferimento.

In logica si studia la relazione tra le formule di un sistema logico-simbolico in cui:

- Il linguaggio delle formule è definito con precisione
- Il significato delle formule è stabilito in modo non ambiguo

Assiomi: Conoscenza iniziale nella logica.

Teoremi: Nuove conoscenze dedotte dagli assiomi.

Per dedurre nuovi teoremi si usano le regole di inferenza. Ogni regola di inferenza è composta da due parti:

- Un insieme di premesse separate da virgole
- Le conclusioni

5.2.1 Logica: Linguaggio

La logica è un linguaggio formale per rappresentare delle informazioni e per trarre delle conclusioni.

Sintassi: Definisce le proposizioni ammissibili del linguaggio.

Semantica: Definisce il “significato” delle proposizioni cioè la verità di una proposizione in un mondo.

5.2.2 Ragionamento logico: Implicazione

Implicazione: Da una cosa ne segue logicamente un'altra.

$$KB \models \alpha$$

La base di conoscenza KB implica α se e solo se α è vero in tutti i mondi dove KB è vero.

Se KB è vera nel mondo reale, allora anche α derivata da KB tramite un processo di inferenza corretto è vera nel mondo reale.

Le formule sono configurazioni fisiche dell'agente, il ragionamento è il processo di costruzioni di nuove configurazioni a partire dalle vecchie.

Il ragionamento logico dovrebbe assicurare che le nuove configurazioni rappresentino aspetti del mondo reale che sono effettive conseguenze degli aspetti del mondo rappresentati dalle vecchie configurazioni.

5.2.3 Models

- Si dice che m è un modello di una proposizione α se α è vera in m
- $M(\alpha)$ è l'insieme di tutti i modelli di α
- Quindi $KB \models \alpha$ iff $M(KB) \subseteq M(\alpha)$



Figure 26: Basi di conoscenza

Approfondimento su slide "Intro_IA-21-AgenteLogico-1" Whumpus Possible Models

5.2.4 Inferenza nel modello Wumpus

L'esempio mostra come la definizione di implicazione può essere applicata per trarre conclusioni, cioè per eseguire l'inferenza logica.

Questo tipo di algoritmo di inferenza è detto **model checking**, perché enumera tutti i modelli possibili per verificare che α sia vero in tutti i modelli in cui KB è vero, cioè che $M(KB) \subseteq M(\alpha)$

5.2.4.1 Model Checking

Per capire i suoi limiti si può pensare all'insieme di tutte le conseguenze di KB come un pagliaio e α come un ago.

L'implicazione è come l'ago nel pagliaio e l'inferenza è come un metodo per trovarlo.

Formalmente uso un algoritmo di inferenza i per poter derivare α da KB .

5.2.5 Inferenza

$KB \vdash \alpha$ = la frase α può essere derivata da KB mediante procedura di inferenza i .

Validità (soundness, truth-preserving): i è valida sound se ogni volta $KB \vdash \alpha$, è anche vero che $KB \models \alpha$, la validità è una proprietà altamente desiderabile.

Completezza: i è completo se ogni volta $KB \models \alpha$, è anche vero che $KB \vdash \alpha$.

Un algoritmo di inferenza che deriva solo frasi logicamente implicate è chiamato truth-preserving algorithm.

Una procedura di inferenza scorretta essenzialmente inventa le cose mentre procede.

È facile vedere che il model checking è una procedura valida perché enumera tutte le possibilità, ma è inefficiente.

5.2.6 Problema del “grounding”

Grounding: La connessione tra i processi di ragionamento logico e l’ambiente reale in cui esiste l’agente.

Le regole generali sono prodotte da un processo di costruzione della frase chiamato apprendimento (learning):

- Dall’agente
- Dall’esperienza umana

L’apprendimento può fallire.

Potrebbe essere il caso che i wumpus causino odori tranne il 29 febbraio negli anni bisestili, che è quando fanno il bagno.

Pertanto, KB potrebbe non essere vero nel mondo reale, ma con buone procedure di apprendimento, c’è motivo di crederlo.

5.2.7 Logica classica

Si suddivide in due classi principali:

- Logica proposizionale
- Logica dei predicati (o del primo ordine)

Permettono di esprimere proposizioni (cioè frasi) e relazioni tra proposizioni.

La principale differenza tra le due classi è in termini di espressività, nella logica dei predicati è possibile esprimere variabili e quantificazioni, mentre questo non è possibile nella logica proposizionale.

5.2.7.1 Limiti

In questa logica una proposizione può essere vera o falsa.

Il vantaggio è che le cose si semplificano enormemente, ma la semplificazione è ottenuta appiattendo l’analisi logica dell’affermazione su due soli connettivi: la negazione e la congiunzione.

Il che non significa che non ci siano altri tipi di inferenza con cui l’uomo produce nuova conoscenza, solo che la logica classica non è in grado di descriverli.

5.3 Logica proposizionale

E’ la logica più semplice ma non molto espressiva, in quanto non possiamo esprimere delle variabili ma solo enumerare tutti gli elementi.

Diede importanza a quei connettivi che rendono enunciati semplici complessi, cioè quelli che possono essere sia veri che falsi.

Regole di inferenza:

- Modus ponens
- Modus tollens
- Sillogismo ipotetico

- Sillogismo disgiuntivo

Basate sull'uso di diversi connettivi logici quali "e", "poiché", "se...allora", "oppure", attribuendovi i corrispondenti valori di verità attraverso apposite tavole.

Le argomentazioni possibili sono ridotte in quattro schemi di validità, detti anapodittivi (indimostrati/indimostrabili).

La logica proposizionale è limitata nel suo carattere espressivo.

Problema di base: non si possono collegare "parti" delle varie proposizioni, tuttavia è molto utilizzata ancora oggi, per esempio nella realizzazione di circuiti elettronici.

5.3.1 Sintassi

- I simboli P_1 e P_2 e le costanti *true* e *false* sono proposizioni.
- Se S è una proposizione anche $\neg S$ è una proposizione (not).
- Se S_1 e S_2 sono proposizioni, allora anche $S_1 \wedge S_2$ è una proposizione (and).
- Se S_1 e S_2 sono proposizioni, allora anche $S_1 \vee S_2$ è una proposizione (or).
- Se S_1 e S_2 sono proposizioni, allora anche $S_1 \Rightarrow S_2$ è una proposizione.
- Se S_1 e S_2 sono proposizioni, allora anche $S_1 \Leftrightarrow S_2$ è una proposizione.
- $S_1 \Rightarrow S_2$ Implicazione o condizionale
- $S_1 \Leftrightarrow S_2$ Equivalenza o bicondizionale

5.3.2 Semantica

Il concetto semantico fondamentale per il linguaggio della logica proposizionale è quello di valutazione.

Tavole di verità:

P	0	$\neg P$	$P \wedge 0$	$P \vee 0$	$P \Rightarrow 0$	$P \Leftrightarrow 0$
False	False	True	False	False	True	True
False	True	True	False	True	True	False
True	False	False	False	True	False	False
True	True	False	True	True	True	True

La semantica del calcolo proposizionale deve specificare come si fa, dato un modello, a calcolare il valore di verità di qualsiasi formula.

Questo viene fatto ricorsivamente fino ad arrivare alle formule atomiche dei cinque connettivi.

5.4 Inference by Theorem Proving

L'implicazione può essere ottenuta dimostrando (provando) teoremi, applicando regole di inferenza direttamente alle frasi presenti nella nostra base di conoscenza per costruire una prova della validità della frase desiderata senza consultare tutti modelli.

Se il numero di modelli è grande ma la lunghezza della dimostrazione è breve, la dimostrazione di teoremi può essere più efficiente del model checking.

Equivalenza logica (primo concetto): Due proposizioni sono logicamente equivalenti iff (se e solo se) sono vere negli stessi: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

Validità and soddisfacibilità (secondo concetto):

- Una proposizione è valida se è vera in tutti modelli
- La validità è collegata all'inferenza tramite il Teorema di deduzione:
 - $KB \models \alpha$ if and only if $(KB \alpha)$ è valido
- Una proposizione è soddisfacibile se è vera in qualche modello
- Una proposizione è non-soddisfacibile se è vera in nessun modello;
- La soddisfacibilità è collegata all'inferenza tramite quanto segue:
 - $KB \models \alpha$ if and only if $(KB \alpha)$ è non-soddisfacibile

5.4.1 Inferenza e prova di teoremi

- I metodi di prova di teoremi si possono dividere in:
 - Applicazione della regola di inferenza:
 - Si producono nuove sentenze valide a partire dalle prime
 - Prova = una sequenza di applicazione della regola di inferenza
 - Può utilizzare le regole di inferenza come operatori in un algoritmo di ricerca standard
 - In genere richiedono la trasformazione delle frasi in una forma normale

Model checking:

- Enumerazione delle tabelle di verità (sempre esponenziale in n)
- Tramite backtracking o tecniche euristiche (valida ma non completa)

5.4.2 Proof methods

Regola del Modus Ponens (un modo per affermare): Ogni volta che sono date le formule $\alpha \Rightarrow \beta$, α allora si può inferire β .

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

Regola del And-Elimination (eliminazione degli and):

$$\frac{\alpha \wedge \beta}{\alpha}$$

Tramite le tabelle di verità si verifica che Modus Ponens e And-Elimination sono validi sempre. Quindi danno inferenze corrette indipendentemente dal contesto senza la necessità di enumerare i modelli.

Tutte le equivalenze logiche possono essere utilizzate come regole di inferenza.

La ricerca di dimostrazioni è un'alternativa all'enumerazione dei modelli. In molti casi pratici trovare una dimostrazione può essere più efficiente perché la dimostrazione può ignorare proposizioni irrilevanti, non importa quante ce ne siano.

5.4.3 Monoliticità della logica

L'insieme di frasi implicite può solo aumentare man mano che le informazioni vengono aggiunte alla base di conoscenza.

Monotonicità significa che le regole di inferenza possono essere applicate ogni volta che si trovano premesse adeguate nella base di conoscenza.

Logiche non monotone: Violano questa norma e catturano una proprietà tipica degli uomini: cambiare idea.

5.4.4 Proof by resolution

Gli algoritmi di ricerca come la ricerca di approfondimento iterativo sono completi nel senso che troveranno qualsiasi obiettivo raggiungibile, ma se le regole di inferenza disponibili sono inadeguate, l'obiettivo non è raggiungibile.

La regola di risoluzione è una singola regola di inferenza che produce un algoritmo di inferenza completo quando accoppiato con qualsiasi algoritmo di ricerca completo.

5.4.5 Logica proposizionale - Regole di inferenza

La regola di risoluzione unitaria prende una clausola, cioè una disgiunzione dei letterali, e un letterale e produce una nuova clausola.

- Rivoluzione unitaria: $\frac{\alpha \vee \beta, \neg \beta}{\alpha}$
- Risoluzione: $\frac{\alpha \vee \beta, \neg \beta \vee \gamma}{\alpha \vee \gamma}$

- oppure: $\frac{\neg\alpha \Rightarrow \beta, \beta \Rightarrow \gamma}{\neg\alpha \Rightarrow \gamma}$

Un singolo letterale può essere visto come una disgiunzione di un letterale, noto anche come clausola unitaria.

5.4.6 Conjunctive normal form (CNF): forma normale della KB

La regola di risoluzione si applica solo alle clausole (cioè disgiunzioni di letterali), quindi sembrerebbe pertinente solo alle basi di conoscenza e alle query costituite da clausole.

Ogni frase della logica proposizionale è logicamente equivalente a una congiunzione di clausole.

Una frase **coniuntiva** espressa come congiunzione di clausole si dice in forma congiuntiva normale o forma normale o CNF.

Conversion to CNF:

1. Eliminare $\Leftrightarrow$, sostituire $\alpha \Leftrightarrow \beta$ con $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
2. Eliminare $\Rightarrow$, sostituire $\alpha \Rightarrow \beta$ con $\neg\alpha \vee \beta$
3. Muovere "¬ inwards" usando le regole di de Morgan e la doppia negazione
4. Applicare la legge distributiva ($\wedge$ over $\vee$):

$$(a) (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

5.4.7 Horn clauses and definite clauses

Alcune basi di conoscenza del mondo reale soddisfano determinate restrizioni sulla forma delle frasi che contengono, il che consente loro di utilizzare un algoritmo di inferenza più ristretto ed efficiente.

Proposizione definita: Una disgiunzione di letterali di cui esattamente uno è positivo.

Clausola di Horn: Una disgiunzione di letterali di cui al massimo uno è positivo.

Le basi di conoscenza contenenti solo clausole definite sono interessanti per tre ragioni:

1. Ogni proposizione definita può essere scritta come un'implicazione la cui premessa è una congiunzione di letterali positivi e la cui conclusione è un singolo letterale positivo
2. Inferenza con le clausole di Horn può essere fatta attraverso algoritmi di forward-chaining and backward-chaining che sono facilmente comprensibili dall'uomo
3. La decisione sulla implicazione con le clausole di Horn può essere fatta in un tempo che è lineare nella dimensione della base di conoscenza

5.4.8 Forward and backward chaining

L'algoritmo di forward-chaining determina se un singolo simbolo di proposizione q , la query, è implicato da una base di conoscenza di clausole definite.

Inizia da fatti noti (letterali positivi) nella base di conoscenza.

Se tutte le premesse di un'implicazione sono note, la sua conclusione viene aggiunta all'insieme dei fatti noti.

Il forward-chaining (concatenamento in avanti) è un esempio del concetto generale di ragionamento basato sui dati (data driven), ovvero ragionamento in cui il focus dell'attenzione inizia con i dati noti.

Può essere utilizzato all'interno di un agente per trarre conclusioni dalle percezioni in entrata, spesso senza una specifica domanda in mente.

L'algoritmo di backward-chaining funziona a ritroso dalla query.

Se la query q è nota per essere vera, non è necessario alcun lavoro. In caso contrario, l'algoritmo trova quelle implicazioni nella base di conoscenza la cui conclusione è q .

Se tutte le premesse di una di queste implicazioni possono essere dimostrate vere (concatenando all'indietro), allora q è vero.

5.5 La logica dei predicati

5.5.1 Limiti della logica proposizionale

La logica proposizionale ha molte interessanti proprietà:

- È completa, tutte le conseguenze logiche sono derivabili per via sintattica e viceversa
- È decidibile in modo automatico, al massimo verifica di tutte le combinazioni

Il difetto principale è la semplicità del linguaggio:

- Non è possibile rappresentare la struttura interna delle affermazioni (solo V o F) e quindi mettere in evidenza legami logici più sottili

La conseguente semplicità delle strutture semantiche:

- Solo un insieme $\{0, 1\}$
- Nessuna possibilità di caratterizzare strutture più complesse

Dal punto di vista della rappresentazione il mondo che osserviamo è fatto di oggetti e relazioni tra oggetti.

5.5.2 Logica del primo ordine (1)

Mentre la logica proposizionale presuppone che il mondo contenga fatti, la logica del primo ordine (come il linguaggio naturale) presuppone che il mondo contenga oggetti e relazioni.

La logica dei predicati permette di rappresentare:

- Oggetti: persone, cose, numeri etc.
- Relazioni: fratello, maggiore, parte di etc.
- Proprietà: rosso, primo, grande etc.
- Funzioni: successore, somma, padre di etc.

Il linguaggio della logica del primo ordine è costruito attorno a oggetti e relazioni e può esprimere fatti su alcuni o tutti gli oggetti universali.

5.5.2.1 Logica del primo ordine vs logica proposizionale vs altre logiche

Impegno ontologico (ontological commitment) assunto da ogni linguaggio: questo è ciò che assume sulla natura della realtà.

Impegno epistemologico (epistemological commitment) assunto da ogni linguaggio: stati di conoscenza che il linguaggio consente rispetto a ciascun fatto.



Figure 27: Formal languages and their ontological and epistemological commitments.

5.5.2.2 Logica del primo ordine (2)

Come ogni logica, la logica predicativa del primo ordine o elementare (FOL) è costituita da tre componenti:

- Un linguaggio formale, che modella un frammento del linguaggio naturale
- Una semantica del linguaggio formale, che definisce le condizioni sotto cui un'espressione del linguaggio è vera oppure è falsa

- Un calcolo, ovvero un algoritmo per stabilire la validità di frasi e argomentazioni in modo puramente meccanico (regola di inferenza)

La logica del primo ordine assume che tutte le rappresentazioni riguardino un insieme non vuoto (e per il resto arbitrario) di individui, detto dominio.

Fatto: il sussistere di una proprietà di un determinato individuo (“Barbara è bionda”) oppure una relazione tra individui (“Alberto è più alto di Barbara”).

I modelli di un linguaggio logico sono le strutture formali che costituiscono i mondi possibili in esame. Ogni modello collega il vocabolario delle frasi logiche agli elementi del mondo possibile, in modo che la verità di ogni frase possa essere determinata.

5.5.2.3 Esempio

La figura mostra un modello con cinque oggetti:

- Riccardo Cuor di Leone, re d’Inghilterra dal 1189 al 1199
- Suo fratello minore, il malvagio re Giovanni, che regnò dal 1199 al 1215
- Le gambe sinistre di Richard e John
- Una corona



Figure 28: A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

5.5.2.4 Logica del primo ordine: Sintassi

Espressione o formula: Sequenza di simboli appartenenti all’alfabeto.

Formule ben formate: Frasi sintatticamente corrette del linguaggio.

Si ottengono attraverso combinazione di formule atomiche, utilizzando i connettivi e i quantificatori

Simboli

Insieme di:

- Simboli di costante C: A,B,C,Giovanni
- Simboli di predicato (o relazione) F: Tondo, Fratello
- Simboli di funzione F: Padre_di, Quadrato
- Simboli di variabile V X, Y (entità non note del dominio)

Termini

I simboli di costante sono termini,

es. Giovanni.

Applicando una funzione n-adica a una n-pla di termini si ottiene un termine,

es. Padre_di(Giovanni).

Formule atomiche

Formata da un simbolo di predicato seguito da una lista di termini.

Es: Fratello(Riccardo,Giovanni) Sposati(Madre_di(Riccardo),Padre_di(Riccardo)).

Le formule atomiche possono essere combinate per formare delle altre formule complesse:



Figure 29: Riassunto sintassi

- Congiunzione: $A \& B$
- Disgiunzione: $A \vee B$
- Implicazione: $A \rightarrow B$
- Negazione: $\neg A$

Le formule con variabili possono essere interpretate tramite i quantificatori:

- Universali: $\forall X p(X)$
- Esistenziali: $\exists X p(X)$.

5.5.2.5 Logica del primo ordine: Semantica

Per dare un significato a una formula bisogna interpretarla come un'affermazione sul dominio del discorso.

- Occorre associare un significato ai simboli.
- Ogni sistema formale è la modellizzazione di una certa realtà (ad esempio la realtà matematica).
- Un'interpretazione è la costruzione di un rapporto fra i simboli del sistema formale e tale realtà (chiamata anche dominio del discorso).
- Ogni formula atomica o composta della logica dei predicati del primo ordine può assumere il valore vero o falso in base alla frase che rappresenta nel dominio del discorso.

Un dominio D è un insieme non vuoto (anche infinito).

Una interpretazione si ottiene associando:

- Ad ogni simbolo costante un elemento di D ;
- Ad ogni simbolo di funzione una funzione su D ;
- Ad ogni predicato n -ario una relazione n -aria su D .

Ad ogni formula atomica si assegna un valore vero o falso.

Ad ogni formula complessa si assegna un valore vero o falso utilizzando le tavole di verità.

5.5.3 I quantificatori

Una volta che abbiamo una logica che consente di rappresentare gli oggetti, è naturale voler esprimere le proprietà di intere raccolte di oggetti, invece di enumerare gli oggetti per nome.

I quantificatori ci permettono di farlo. La logica del primo ordine contiene due quantificatori standard, chiamati quantificatore universale ed esistenziale

5.5.3.1 Qualificatore universale $\forall$

Risolve il problema di come esprimere regole generali.

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

- La frase dice: "Per ogni x , se x è un re, allora x è una persona"
- Il simbolo x è chiamato variabile

- Una variabile è un termine a sé stante e, come tale, può anche fungere da argomento di una funzione, ad esempio $LeftLeg(x)$
- Un termine senza variabili è chiamato termine di base (**ground term**)

Intuitivamente, la frase $\forall x P$, dove P è una qualsiasi espressione logica, dice che P è vero per ogni oggetto x . Più precisamente, $\forall x P$ è vero in un dato modello se P è vero in tutte le possibili **interpretazioni estese** costruite dall'interpretazione data nel modello, dove ogni interpretazione estesa specifica un elemento di dominio a cui x si riferisce.

Sbagliato è l'utilizzo della congiunzione invece dell'implicazione.

5.5.3.2 Qualificatore esistenziale $\exists$

Si riesce a dare una proposizione su qualche oggetto nell'universo senza nominarlo, ma usando un quantificatore esistenziale.

$$\exists x Crown(x) \wedge OnHead(x, John)$$

La frase $\exists x P$ dice che P è vera per almeno un oggetto x .

Più precisamente, $\exists x$ per cui P è vero in un dato modello se P è vero in almeno un'interpretazione estesa che assegna x a un elemento del dominio.

Proprio come $\Rightarrow$ sembra essere il connettivo naturale da usare con $\forall$, $\wedge$ è il connettivo naturale da usare con $\exists$.

5.5.4 Quantificatori annidati

I quantificatori consecutivi dello stesso tipo possono essere scritti come un unico quantificatore con più variabili:

$$\forall x, y Siblings(x, y) \Rightarrow Siblings(y, x)$$

In altri casi l'ordine è importante.

$$\forall x (Crown(x) \vee (\exists x Brother(Richard, x)))$$

Qui la x in Fratello (Riccardo, x) è quantificata dal quantificatore esistenziale.

La regola è che la variabile appartiene al quantificatore più interno che la menziona; quindi non sarà soggetta a nessun'altra quantificazione.

5.5.5 Quantificatori annidati: connessioni tra $\forall$ e $\exists$

Affermare che a tutti non piacciono i broccoli equivale ad affermare che non esiste qualcuno a cui piacciono, e viceversa.

"Il gelato piace a tutti" significa che non c'è nessuno a cui non piaccia il gelato.

5.5.6 Logica del primo ordine: uguaglianza

Il simbolo di uguaglianza indica che due termini si riferiscono allo stesso oggetto.

$$Father(John) = Henry$$

Per dire che Riccardo ha almeno due fratelli, scriveremmo:

$$\exists x, y Fratello(x, Riccardo) \wedge Fratello(y, Riccardo) \wedge \neg(x = y)$$

La frase $\exists x, y Fratello(x, Riccardo) \wedge Fratello(y, Riccardo)$ non ha il significato voluto.

Esempi in slide "Intro_IA-22-AgenteLogico-2" pagine 48-49

5.5.7 Logica del primo ordine: Il mondo dei wumpus

Si può osservare che gli assiomi del primo ordine sono molto più concisi, in quanto catturano in modo naturale esattamente ciò che si deve dire.

Le azioni nel mondo del wumpus possono essere rappresentate da termini logici: Turn(Right), Turn(Left), Forward, Shoot, Grab.

Per determinare quale sia il migliore, il programma agente esegue la query: ASKVARs($\exists a$ BestAction($a, 5$)).

Esempi in slide "Intro_IA-22-AgenteLogico-2" pagine 54-58

5.5.8 Logica del primo ordine e regole di inferenza e forma a clausole

Una clausola è una formula priva di quantificatori ed è formata da una disgiunzione di termini.

Regola di risoluzione: Regola di inferenza, da $A \vee B$, $B \vee C$ deduci $A \vee C$.

Il metodo di risoluzione permette di dimostrare un teorema mediante il metodo della **refutazione**, cioè si dimostra che la negazione del teorema è falsa.

5.6 Modus Ponens Generalizzato

Per trasformare la base di conoscenza in una formula clausole devo utilizzare il modus ponens generalizzato in grado di eliminare i quantificatori universali.

Modus Ponens: Dall'implicazione e dalla premessa dell'implicazione si può inferire la conclusione ($a \rightarrow B$, $A \Rightarrow B$).

Esempio in slide "Intro_IA-22-AgenteLogico-2" pagina 62

5.6.1 Forma a Clausole e Modus Ponens Generalizzato

Con il Modus Ponens generalizzato posso eliminare i quantificatori in un qualche ordine.

Per le formule a clausole (adatte per ragionare con un'unica regola di risoluzione), vanno eliminati i quantificatori e gli AND, per cui: $\exists x \text{ Possiede}(\text{Jenny}, x) \wedge \text{Barbie}(x)$ diventa due clausole:

- $\text{Possiede}(\text{Jenny}, x)$
- $\text{Barbie}(x)$

5.6.2 Modus Ponens Generalizzato e inferenza

Con il Modus Ponens generalizzato possiamo:

- Partire dalle formule nella KB e generare nuove conclusioni che portino a nuove inferenze (**concatenazione in avanti**)
- Partire da qualcosa da dimostrare, trovare implicazioni che ci permettono di concludere e stabilire le premesse necessarie (**concatenazione all'indietro**)

5.6.3 Inferenza nella logica del primo ordine

Concatenazione in avanti

Quando un nuovo fatto p è aggiunto alla KB

- Si fanno scattare tutte le regole le cui premesse sono soddisfatte da p o sono già note
- Quindi si aggiunge la conclusione alla KB
- Si continua nella catena finché si trova una risposta (supponendo che ne basti una) o non è più possibile aggiungere nuovi fatti. La concatenazione in avanti è guidata dai dati

Concatenazione all'indietro

Quando si formula una domanda

- Se si conosce un fatto p che soddisfa la query ritorna il puntatore al fatto
- Per ogni regola le cui conseguenze soddisfano la domanda si cerca di provare se ogni premessa della regola soddisfa la query in un meccanismo di concatenazione all'indietro

La concatenazione all'indietro è la base per la programmazione logica.

Esempi in slide "Intro_IA-22-AgenteLogico-2" pagine 69-70

5.6.4 Vantaggi della logica

I vantaggi principali della logica sono:

- **Precisione**, la logica ha una semantica chiara e definita per la quale esistono metodi standardizzati per determinare il significato di una espressione;
- **Flessibilità**, la logica rappresenta la conoscenza in modo dichiarativo, quindi in modo indipendente dal suo uso;
- **Modularità**, le asserzioni della logica possono entrare in una base di conoscenza in modo indipendente le une dalle altre.

5.6.5 Limiti della logica

I limiti principali della logica sono:

- **Inadeguatezza espressiva**, non può rappresentare mondi dinamici, non può rappresentare conoscenze probabilistiche;
- **Monotonicità**, l'aggiunta di un teorema non provoca la cancellazione di nessun teorema precedente;
- **Inefficienza**, la dimostrazione automatica basata esclusivamente sulla logica si fonda su processi di ricerca, la ricerca è inerentemente esponenziale ed eventuali euristiche non ne cambiano la natura combinatoria.

5.6.6 Ingegneria della conoscenza nella logica del primo ordine

Sviluppare una base di conoscenza in logica del primo ordine richiede:

- Un attento processo di analisi del dominio;
- Scelta di un vocabolario;
- Codifica degli assiomi necessari a supportare le inferenze richieste.

Knowledge engineer è una persona che studia un particolare dominio, impara quali concetti sono importanti in quel dominio e crea una rappresentazione formale degli oggetti e delle relazioni nel dominio.

5.6.7 The knowledge-engineering process

1. Identificare il compito della base di conoscenza (task):

- (a) Si deve delineare la gamma di domande che la base di conoscenza dovrà supportare e i tipi di fatti che saranno disponibili per ogni specifica istanza di problema
- (b) Il compito determinerà quale conoscenza deve essere rappresentata per collegare le istanze del problema alle risposte
- (c) Questa fase è analoga al processo PEAS per la progettazione di agenti

2. Raccogliere la conoscenza rilevante (knowledge acquisition):

- (a) L'ingegnere della conoscenza potrebbe essere già un esperto del dominio, o potrebbe aver bisogno di lavorare con esperti reali per estrarre ciò che sanno
- (b) In questa fase, la conoscenza non è rappresentata formalmente
- (c) L'idea è quella di comprendere l'ambito della base di conoscenza, come determinato dal compito, e di capire come funziona effettivamente il dominio

3. Definire un vocabolario di predicati, funzioni e costanti (ontologia di dominio):

- (a) Ovvero, tradurre i concetti importanti a livello di dominio in nomi a livello logico
- (b) E' un processo creativo che dipende dallo stile dell'ingegnere della conoscenza
- (c) L'ontologia determina quali tipi di cose esistono, ma non determina le loro proprietà specifiche e le loro interrelazioni

4. Codificare la conoscenza generale riguardante il dominio:

- (a) Scrivere gli assiomi per tutti gli elementi

- (b) In questo modo si fissa (per quanto possibile) il significato dei termini, consentendo all'esperto di verificare il contenuto
 - (c) Spesso questa fase rivela concetti errati o lacune nel vocabolario che devono essere risolte tornando alla fase 3 e iterando il processo
5. Codificare una descrizione della specifica istanza del problema:
- (a) Se l'ontologia è ben pensata, questo passo sarà facile
 - (b) Si tratta di scrivere semplici frasi atomiche su istanze di concetti che fanno già parte dell'ontologia
6. Interrogare la procedura di inferenza e ottenere da essa risposte:
- (a) È qui che si trova la ricompensa
 - (b) Possiamo lasciare che la procedura di inferenza operi sugli assiomi e sui fatti specifici del problema per ricavare i fatti che ci interessa conoscere.
In questo modo, evitiamo di dover scrivere un algoritmo di soluzione specifico per l'applicazione.
7. Fare il debugging della base di conoscenza:
- (a) Le risposte del passo precedente saranno corrette per la base di conoscenza così come è stata scritta, assumendo che la procedura di inferenza sia corretta, ma potrebbero non essere quelle che l'utente si aspetta
 - (b) Ad esempio, se manca un assioma, alcune interrogazioni non potranno essere soddisfatte dalla base di conoscenza

5.6.8 Prolog

Un programma PROLOG ha le seguenti caratteristiche:

- Un programma è una sequenza di formule
- Sono ammesse solo clausole di Horn
- I termini possono essere simboli di costanti, variabili, termini funzionali
- Le interrogazioni includono congiunzioni, disgiunzioni, variabili e termini funzionali
- Non ci sono antecedenti negati, ma un goal $\text{not } P$ si considera dimostrato se il sistema fallisce nel dimostrare P .

Per provare: <http://www.swi-prolog.org/>

5.7 Cenni di Pianificazione

(Argomento facoltativo)

Slide "Intro_IA-23-AgenteLogico-Pianificazione"

6 Modelli Predittivi

6.1 Machine Learning and Data

Il machine learning è un tipo di intelligenza artificiale che permette al sistema del computer di migliorare automaticamente le performance su una task, imparando da dati ed esempi, senza essere programmato esplicitamente per ogni azione.

Involve lo sviluppo di algoritmi e modelli statistici che permettono al computer di imparare modelli e fare predizioni o decisioni basate sui dati in input.

Concetto	Definizione	Caratteristiche
Intelligenza Artificiale	Un ampio campo di studio che si concentra sulla creazione di macchine intelligenti in grado di eseguire compiti che normalmente richiedono l'intelligenza umana, come il riconoscimento vocale, la presa di decisioni e la risoluzione di problemi.	• Risolve problemi complessi attraverso ragionamento e decisioni simili a quelli umani. • Include una vasta gamma di sotto-discipline, tra cui il machine learning e il deep learning. • Può essere basata su regole o guidata dai dati. • Può essere applicata in numerosi settori e ambiti.
Machine Learning	Un sottoinsieme dell'intelligenza artificiale che prevede lo sviluppo di algoritmi e modelli statistici che permettono ai computer di apprendere dai dati e fare previsioni o prendere decisioni senza essere esplicitamente programmati per ogni situazione.	• Si concentra sulla progettazione di algoritmi che migliorano le proprie prestazioni nel tempo apprendendo dai dati. • Può essere supervisionato, non supervisionato, semi-supervisionato o basato sul reinforcement learning. • Può essere utilizzato per problemi di regressione e classificazione. • È una componente chiave del deep learning.
Deep Learning	Un sottoinsieme del machine learning che prevede l'addestramento di reti neurali artificiali su grandi quantità di dati per eseguire compiti complessi come il riconoscimento di immagini e del parlato.	• Utilizza reti neurali artificiali con più strati per apprendere e prendere decisioni. • Si concentra sullo sviluppo di reti neurali profonde con molti strati per modellare relazioni complesse nei dati. • Può essere supervisionato, non supervisionato o semi-supervisionato. • Utilizza grandi quantità di dati per addestrare le reti.

6.1.1 Possibili applicazioni del ML

- Riconoscimento di immagini e discorsi
- Predizione del traffico
- Raccomandazione di prodotti
- Veicoli automatici
- Filtro spam
- Riconoscimento frodi
- Lingua tradotta automaticamente

6.1.2 ML come un processo di analisi dati

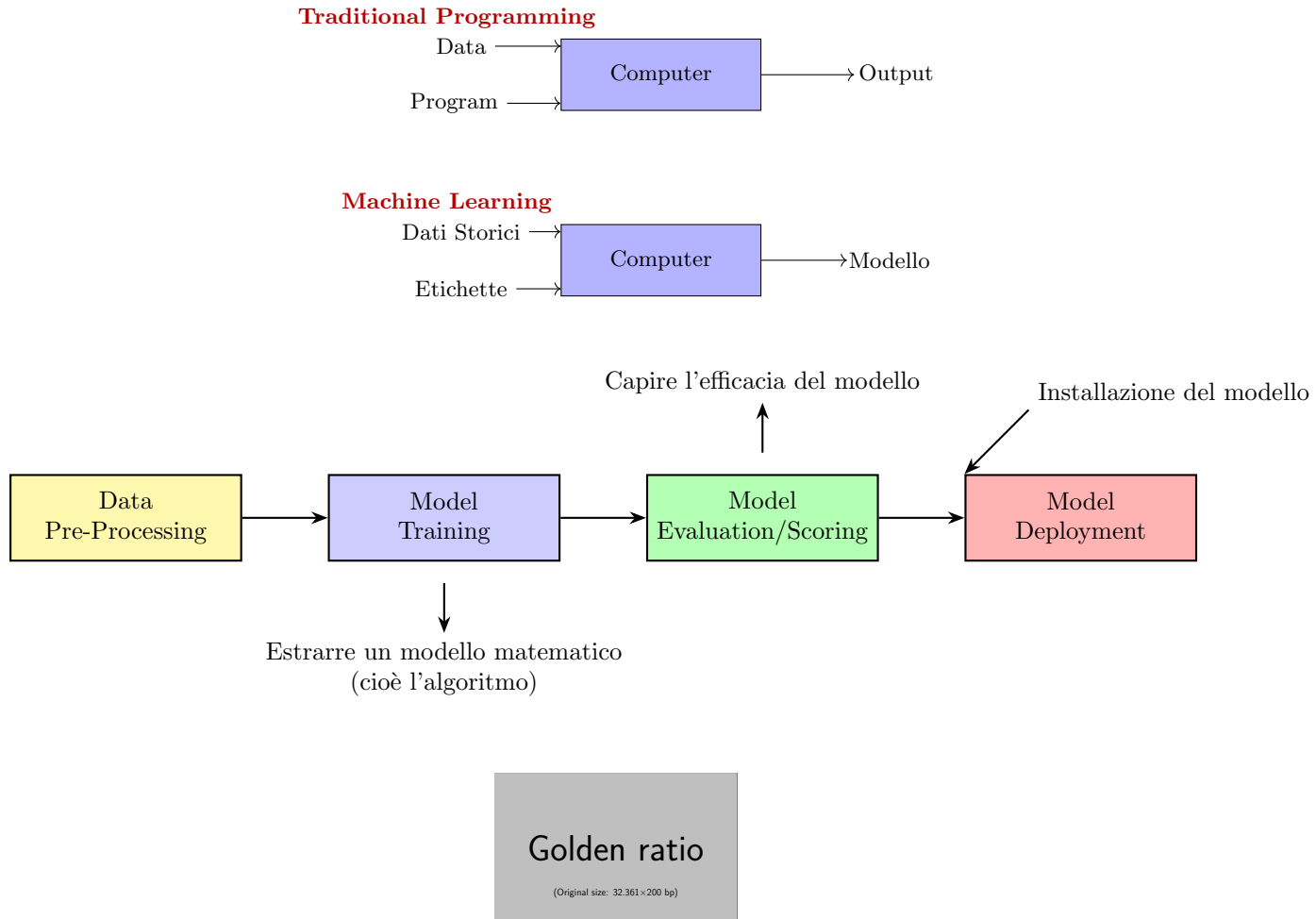


Figure 30: Analisi dei dati tramite ML

6.1.3 Dove troviamo questa metodologia: Data science

E' la disciplina che studia il processo di analisi dei dati. Necessitiamo di questo approccio perché abbiamo a disposizione il mondo dei big data.



Figure 31: Grafico Data Explosion



Figure 32: Big Data and DL

6.2 L'approccio data-driven, l'analisi dei dati e dei modelli predittivi

Modellare il mondo tramite un processo di analisi dei dati:

- Dai dati che vengono dal mondo reale;
- Alla definizione di un sistema guidato dai dati per il supporto alla decisione.

6.2.1 Perché abbiamo bisogno di una metodologia scientifica per l'analisi dei dati

Nell'approccio subsimbolico i dati sono essenziali.

La conoscenza viene appresa tramite essi

- Si trovano pattern ricorrenti e correlazioni nei dati e questo serve all'agente per classificare la situazione nuova che sta vedendo

Il paradigma prevalente dell'analisi dei dati parte dal presupposto che:

- I dati sono il punto di partenza di ogni analisi
- E' possibile ottenere l'oggettività nei nostri modelli sul mondo collezionando dati in modo imparziale
- I dati parlano, cioè guidano la nostra mente nella costruzione dei modelli

I dati non sono imparziali

- Si collezionano i dati rilevanti per il problema e questo può essere collegato al tipo di soluzione/narrazione che vogliamo dare al problema

I dati non sono necessariamente sufficienti

- Non solo siamo guidati silenziosamente dalle nostre congetture nella selezione dei dati, ma se proviamo a sopprimere nozioni o schemi esplicativi, non abbiamo modo di sapere se i dati che abbiamo raccolto sono quelli più rilevanti per comprendere o risolvere i problemi in questione
- Non abbiamo alcun meccanismo per motivare la ricerca di ulteriori dati che possano fornire conferme, confutazioni o ulteriori approfondimenti

I dati non parlano, tanto meno spiegano

- Non si è a conoscenza di un processo oggettivo e deduttivo che da una mole di dati generi una teoria. Quindi i dati non parlano
- Non si può nemmeno interpretare i dati senza ricorrere a un contesto teorico

Il problema:

Le informazioni che cercheremo e troveremo ci basteranno, non vedendo cosa possa essere pericoloso.

Quindi i dati motivano le congetture, il tentativo ci permette di distinguere le ipotesi e guidarci per la selezione dei dati, motivando la ricerca e la scoperta di essi.

6.3 Data Science e una metodologia per l'analisi dei dati

6.3.1 Data Science

E' l'insieme di principi metodologici (basati sul metodo scientifico) e tecniche multidisciplinari volto a interpretare ed estrarre conoscenza dai dati attraverso la relativa fase di analisi da parte di un esperto.

6.3.2 I cinque passi della Data Science

I cinque passi fondamentali per la scienza dei dati sono i seguenti:

1. Porre una domanda interessante
2. Ottenere i dati
3. Esplorare i dati
4. Creare un modello per i dati
5. Comunicare e presentare i risultati

I punti 2, 3, 4 e 5 sono i passi più tecnici di un processo di scienza dei dati.

6.4 Data Science - Modelli predittivi

6.4.1 Trovarle correlazioni fra i dati

I coefficienti di correlazione sono una misura quantitativa che descrive l'intensità dell'associazione/relazione fra due variabili (compresa tra -1 e 1).

La correlazione fra due insiemi di dati ci dice quanto si “muovono insieme”.

Gli algoritmi di Machine Learning cercano e sfruttano queste relazioni tra i dati per offrire previsioni accurate.

In generale, una correlazione tenterà di misurare una relazione lineare fra le variabili.

- Una correlazione positiva significa che quando una variabile aumenta, anche l'altra tende ad aumentare
- Una correlazione negativa significa che quando una variabile aumenta, l'altra tende a diminuire
- Minima correlazione è a 0

Se non viene individuata una correlazione in questo modo, ciò non significa che non esiste alcuna relazione fra le variabili, ma solo che non esiste una linea “best fit” per le linee.

Quando i grafici e le statistiche mentono, in realtà mentono le persone.

Uno dei modi più facili per ingannare consiste nel **confondere la correlazione e la causalità**.

- La correlazione è una metrica quantitativa compresa fra -1 e 1 che misura come si spostano due variabili l'una rispetto all'altra e stabilisce il grado con il quale le variabili cambiano insieme;
- La causalità è l'idea che una variabile influenzi un'altra e determini effettivamente il valore di un'altra.

Molto spesso capita che due variabili siano, sì, correlate, ma senza alcuna relazione di causalità fra di esse. Potrebbe entrare in gioco un fattore di confusione.

Fattore di confusione: Esiste “in agguato” una terza variabile non individuata e che funge da collegamento fra le due variabili.

Correlazione o causalità? In alcuni casi le variabili potrebbero non avere proprio nulla a che fare l'una con l'altra! Il tutto potrebbe essere semplicemente frutto di coincidenza.

Vi sono molte variabili che sono correlate ma senza alcuna relazione di causalità.

Attenzione al paradosso di Simpson

Il paradosso stabilisce che una correlazione fra due variabili può essere completamente invertita considerando fattori differenti.

Questo significa che anche se un grafico potrebbe mostrare una correlazione positiva, queste variabili possono diventare anti-correlate prendendo in considerazione un altro fattore.

6.4.1.1 Esempio

Il consumo di mozzarella determina il numero di lauree in ingegneria civile a livello mondiale. I due andamenti sono molto simili, eppure non esiste un legame causale tra essi.



Figure 33: Mozzarella consumption graph

Esempio nel pacco di slide "Intro_IA-32-Modelli Predittivi" a pagine 37, 38, 39

6.5 Machine learning

6.5.1 Che cosa si intende con machine learning?

Dare ai computer la capacità delle macchine di apprendere dai dati, senza ricevere regole esplicite da un programmatore.

Il machine learning si occupa della capacità di trarre dai dati determinati pattern (segni), anche se i dati contengono errori (rumore).

In machine learning, al modello non viene detto qual è la soluzione migliore; piuttosto riceve vari esempi del problema e gli viene chiesto di decidere qual è la soluzione migliore.

6.5.2 ML (algoritmi predittivi) vs algoritmi classici

Riconoscimento di un volto:

- Classico
 - Nell'algoritmo viene inserito un codice che definisce un volto come una forma tondeggiante, con due occhi, capelli, naso e così via
 - L'algoritmo ricercherà nella fotografia queste caratteristiche “cablate” e dirà se è stato in grado o meno di trovarle
- Predittivo
 - Non viene mai detto che cos'è un volto: vengono solo forniti degli esempi (training set), alcuni con volti, altri senza
 - Il compito del modello di machine learning è trovare la differenza.
Una volta individuata la differenza, usa queste informazioni per accettare una nuova immagine e predire se contiene o meno un volto.

6.5.3 Il machine learning non è perfetto

Quasi nessun modello di machine learning tollera l'impiego di dati “sporchi”, con valori mancanti o valori categorici.

- I dati usati sono già stati pre-elaborati e ripuliti.

Ogni riga di un dataset ripulito rappresenta una singola osservazione dell'ambiente che stiamo tentando di modellare.

- Se l'obiettivo è quello di trovare le relazioni esistenti fra le variabili, allora si deve partire dal presupposto che fra queste variabili esista in effetti una relazione.

Gli algoritmi non sono in grado di comunicare che tale relazione, in realtà, non esiste.

La macchina è molto abile, ma fatica a collocare le cose nel loro contesto:

- L'output è una serie di numeri e metriche che tentano di quantificare l'efficacia del modello
- Compito dell'essere umano valutare queste metriche e comunicare i risultati

La maggior parte dei modelli di machine learning è sensibile alla rumorosità dei dati.

- Questo significa che i modelli si confondono quando si includono dati insensati.

6.6 Algoritmi predittivi

6.6.1 Tipi di ML (algoritmi predittivi)

La scelta dell'algoritmo:

- Ha impatto sulle modalità di preparazione dei dati
- Comporta una susseguente attività di sistemazione dati

Si possono classificare per:

- Scopo
- Caratteristiche delle modalità di apprendimento e tecniche utilizzate:
 - Tipi di dati/strutture organiche utilizzati (albero/grafo/rete neurale)
 - Campo della matematica dal quale attinge maggiormente (statistica/calcolo delle probabilità)
 - Livello di calcolo richiesto per l'addestramento (apprendimento profondo - deep learning)

6.6.2 Classificazione degli algoritmi per scopo

Permette di identificare quali algoritmi siano adatti ad un particolare problema predittivo

- Classificazione
- Regressione
- Clustering
- Association rules
- Serie temporali

6.6.2.1 Classificazione

Individuazione dell'appartenenza di un elemento ad una classe.

L'output della classificazione è categorico e quindi può assumere un numero finito di possibili valori.

Agli algoritmi di classificazione appartengono i classificatori lineari, gli alberi decisionali e le reti neurali.



Figure 34: Classificazione

6.6.2.2 Regressione

L'output del modello è un valore numerico, che si vuole approssimare tramite una funzione dei dati di input.

La variabile di output è continua e può assumere un numero infinito di valori.

Algoritmi che appartengono a questa categoria sono: la regressione lineare.



Figure 35: Regressione

6.6.2.3 Clustering

Il raggruppamento degli elementi di un dataset in gruppi (i cluster) utilizzando soltanto le informazioni contenute nei dati di input (apprendimento non supervisionato).

L'output del clustering (ovvero la categorizzazione) non è noto a priori. I raggruppamenti sono realizzati in base alla similarità dei punti.

Es. clustering: k-means.

6.6.3 Classificazione per modalità di apprendimento

Il procedimento con cui l'algoritmo impara dai dati di input è chiamato training.

- Supervisionato
- Non supervisionato
- Semi-supervisionato

6.6.3.1 Supervisionato

Modelli di analisi predittiva $\Rightarrow$ capacità di prevedere il futuro sulla base del passato.

Il machine learning con supervisione richiede l'impiego di un determinato tipo di dati: i dati etichettati.

L'apprendimento con supervisione funziona usando delle parti dei dati per prevedere un'altra parte.

Si devono separare i dati in due parti:

1. I **predittori**, che sono le colonne che verranno usate per effettuare la previsione.
 - (a) Sono chiamate anche caratteristiche, input, variabili o variabili indipendenti.
2. La **risposta**, che è la colonna che vogliamo prevedere.
 - (a) Questo è chiamato anche risultato, etichetta, target e variabile dipendente.

L'apprendimento con supervisione tenta di trovare una relazione fra i predittori e la risposta per effettuare una previsione.

Questa relazione rappresenta una regola generale di classificazione detta modello.

Una volta costruito il modello, la macchina lo utilizza per classificare le nuove istanze.

6.6.3.2 Non supervisionato

Non tentano di trovare una relazione fra i predittori e una specifica risposta e pertanto non vengono usati per effettuare previsioni di alcun tipo.

Al contrario, vengono utilizzati per trovare nei dati forme di organizzazione e di rappresentazione precedentemente sconosciute.

Possono ridurre le dimensioni dei dati condensando insieme più variabili.

Trovare dei gruppi di osservazioni che si comportano allo stesso modo e raggrupparli (clustering).

Vantaggi: Non richiede dati etichettati.

Difetti: Si perde ogni potere predittivo ed è difficile capire se si comporta correttamente

6.6.3.3 Semi-supervisionato

Lavorano su un insieme di dati che solo in parte possiede già una classificazione.

Solitamente, la parte di dati già classificata è una piccola percentuale del dataset, ma è sufficiente a rendere l'algoritmo più preciso.

Utile quando vi è grande disponibilità di dati non classificati ed il costo per classificarli manualmente è molto alto.

6.7 Costruire un modello di ML

6.7.1 Programmazione classica e ML

Nella programmazione classica dato un problema e un certo insieme di dati di input, il programmatore realizza un algoritmo che produce un certo risultato (un insieme di dati di output).

Se non cambia il problema, l'algoritmo è lo stesso per qualunque configurazione dei dati.

L'output è il risultato dell'applicazione dell'algoritmo ai dati.

Nel Machine Learning dato un problema e un certo insieme di dati di input, viene fornita un'esperienza (sotto forma di soluzioni desiderate o reazioni dell'ambiente alla macchina).

Sulla base di questi fattori, il metodo di ML costruisce un modello di algoritmo che si adatta ai dati.

L'output è l'algoritmo, cioè il risultato dell'apprendimento della macchina.

6.7.2 Gli algoritmi di ML

Modello di ML vs Algoritmo

- Un algoritmo è una sequenza di istruzioni logicamente organizzate e definite che descrive un processo o una procedura computazionale
- Un modello è un software che viene inserito nell'algoritmo e che ci serve per trovare la soluzione al nostro problema
 - Poiché spesso non conosciamo la vera soluzione, queste vengono chiamate predizioni.

Un **modello addestrato** di machine learning verrà poi implementato all'interno di un algoritmo che se eseguito dalla macchina restituisce sulla base degli input l'output più corretto secondo il modello che ha al suo interno. Il modello deve essere addestrato in una fase precedente.

Un modello addestrato di ML è un file che è il risultato di un programma in grado modificare alcuni parametri del modello sulla base di dati di training.

Costruire un modello nell'apprendimento automatico significa creare una rappresentazione matematica generalizzando e imparando dai dati di addestramento.

Un algoritmo di machine learning è un metodo matematico per individuare schemi ricorrenti in un set di dati.

Gli algoritmi di machine learning sono spesso ricavati da statistiche, calcolo e algebra lineare.

Il **processo di esecuzione** di un algoritmo di machine learning su un set di dati (chiamati dati di addestramento) e l'ottimizzazione dell'algoritmo per la ricerca di determinati schemi o risultati viene detto addestramento del modello.

I **parametri del modello** sono variabili interne del modello di ML che vengono appresi e stimati durante l'addestramento dal set di dati.

La **funzione risultante** (memorizzata in un file/software) con regole e strutture di dati viene chiamata **modello addestrato** di machine learning.

6.7.2.1 Implementazione del modello di machine learning

L'implementazione è il processo con cui un modello di machine learning viene reso disponibile per l'utilizzo in un ambiente di destinazione.

Il linguaggio più utilizzato è Python e la libreria più famosa Scikit-learn.

6.8 Problematiche comuni agli algoritmi di ML

Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 75-84

6.8.1 Hyperparameter tuning

6.8.2 Grid Search (o parameter sweep)

6.8.3 Random Search

6.8.4 Overfitting

Accade quando un modello è troppo complesso ed eccessivamente adattato ai dati di training. La crescita della complessità del modello diminuisce l'errore predittivo sui dati del training set.

Un modello sovradimensionato è un modello matematico che contiene più parametri di quelli che possono essere giustificati dai dati.

All'aumentare della complessità del modello diminuisce anche l'errore sul test set, ma solo fino ad un certo punto:

- L'errore infatti ritorna a crescere superata una certa soglia di complessità
- La crescita di questo errore segnala che si sta entrando nel territorio dell'overfitting

3 termini importanti:

- **Bias:** Rappresenta quanto, in media, le previsioni di un modello sono lontane dalla realtà
- **Varianza:** Indica di quanto le stime variano attorno alla media
- **Underfitting:** In questo caso il modello è troppo semplice per poter avere in media una buona performance predittiva

6.8.5 Utilizzo di un approccio addestramento/test

6.8.5.1 Capacità di generalizzazione

6.9 Basic algorithms in machine learning

6.9.1 Algoritmi di classificazione

La classificazione individua l'appartenenza ad una classe.

Cerca di predire (restituire) l'uscita più probabile dando in ingresso un numero limitato di input (predittori, caratteristiche /features) l'uscita (l'etichetta).

6.9.2 Classificazione

A partire da un fenomeno l'esperto di ML ne esegue l'analisi e individua le sue principali caratteristiche o features.

Queste features sono organizzate in osservazioni o cases e corrispondenti ai valori target o labels.

A questo punto la macchina si "addestra" (fase di training) a riconoscere quali valori delle features producono certi target (si dice che riconosce dei pattern nei dati) e costruisce un modello che si adatta ai target richiesti. Terminata la fase di addestramento, il modello è pronto per avere in input nuove osservazioni e prevedere quale potrebbe essere il valore target di uscita, corredato di una valutazione della bontà della previsione.

6.9.3 Classificatore Naive (ingenuo) Bayes

Il Naive Bayes è un algoritmo classificatore probabilistico basato sul teorema di Bayes.

Calcola la probabilità di ogni etichetta per un determinato oggetto osservando le sue caratteristiche.

Poi, sceglie l'etichetta con la probabilità maggiore.

Per usare il classificatore bayesiano occorre conoscere o stimare le probabilità a priori e condizionali del problema.

Cosa fa il teorema di Bayes:

- Si supponga di voler classificare della frutta (si faccia finta che siano solo due per semplificare la spiegazione) che viene mostrata a un osservatore
- Per gli esseri umani, ma allo stesso modo deve essere fatto per le macchine, determinare la tipologia di frutta che si sta osservando, avviene esaminando determinate caratteristiche (feature) estratte dall'osservazione della frutta, attraverso opportune tecniche
- Parte dal presupposto dell'indipendenza fra le variabili e usa le probabilità per classificare gli oggetti in base alle loro caratteristiche
- La teoria bayesiana di decisione svolge un ruolo importante solo quando risultano conosciute alcune informazioni a priori sugli oggetti.

Il Naive Bayes parte dal presupposto dell'indipendenza fra le variabili e usa le probabilità per classificare gli oggetti in base alle loro caratteristiche.

Approfondimento su slide "Intro-IA-32-Modelli Predittivi" pagine 91-110

6.9.3.1 Esempio

Un frutto può essere classificato nella classe "mele" se è di colore rosso, ha un diametro di circa 10 cm e ha una forma rotonda.

Sono tre caratteristiche differenti.

Un algoritmo di Naive Bayes classifier considera ognuna di queste caratteristiche come un contributo indipendente alla probabilità complessiva che il frutto sia una mela.

Non considera le possibili correlazioni tra le caratteristiche.

6.9.4 La probabilità condizionata e il teorema di Bayes

Dati due eventi A e B , si definisce probabilità condizionata $P(A|B)$ la probabilità del verificarsi dell'evento A , condizionata al fatto che sia verificato l'evento B : $P(A|B) = P(A \cap B)/P(B)$.

Il Teorema di Bayes si ricava mettendo in rapporto $P(A|B) = P(A \cap B)/P(B)$ e $P(B|A) = P(A \cap B)/P(A)$ ottenendo $P(A|B) = P(B|A) \cdot P(A)/P(B)$.

Questo ci permette di esprimere la probabilità a posteriori $P(A|B)$ che una determinata ipotesi A sia verificata in presenza del verificarsi di una evidenza B .

Approfondimento su slide "Intro.IA-32-Modelli Predittivi" pagine 91-110

6.9.5 Classificatore di bayes vantaggi e svantaggi

Vantaggi:

- Utilizzo semplice e facilmente intuibile il funzionamento
- EFFICIENTE in termini computazionali
- Non ha bisogno di grandi quantità di dati
- Ancora oggi risolve egregiamente alcuni problemi di classificazione con discreta efficienza.

Svantaggi:

- Richiede conoscenza di tutti i dati del problema
- L'algoritmo fornisce un'approssimazione "ingenua" (naive) del problema perché non considera la correlazione tra le caratteristiche dell'istanza.

Approfondimento su slide "Intro.IA-32-Modelli Predittivi" pagine 91-110

6.9.6 Alberi decisionali (Decision Trees)

Sono uno dei metodi di calcolo principalmente utilizzati nel data mining, in particolare per determinare a quale categoria appartiene un elemento, in base al valore dei suoi attributi noti.

Un albero decisionale è una struttura a grafo che include un nodo radice, da cui partono dei rami che arrivano a dei nodi figli.

I nodi terminali sono detti "nodi foglia".

1. Ogni nodo interno denota un test su un attributo
2. Ogni ramo indica il risultato di un test
3. Ogni nodo foglia contiene un'etichetta di classe
4. Il nodo più in alto nell'albero è il nodo radice



Figure 36: Esempio albero decisionale

Ogni nodo interno rappresenta un test su un attributo: Weather, Time, Hungry.

Ogni nodo foglia rappresenta una classe: Walk, Bus.

I vantaggi di avere un albero decisionale sono i seguenti:

- Non richiede alcuna conoscenza di dominio
- È facile da capire

- Le fasi di apprendimento e classificazione di un albero decisionale sono semplici e veloci

L'algoritmo si fonda sul concetto di **entropia della teoria dell'informazione**.

Ora, per ciascun attributo del nostro dataset, dobbiamo trovare quella ripartizione, basata sui valori dell'attributo, per la quale il guadagno è massimo.

La ripartizione che massimizza il guadagno corrisponde al primo livello dell'albero, sotto l'insieme totale.

A questo punto si ripete il processo per ciascuno dei sottoinsiemi che, al loro interno, presentano elementi appartenenti a classi diverse.

Il processo si ferma quando i sottoinsiemi contengono elementi solo di una classe, oppure quando il continuare con la suddivisione non porta miglioramenti dell'accuratezza.

Gli algoritmi degli alberi decisionali includono **tecniche di pruning** che hanno lo scopo di limitare la crescita dell'albero.

Il pruning evita che l'albero decisionale si adatti troppo ai dati di training, causando l'overfitting del modello.

Vantaggi:

- La produzione di regole come parte dell'output dell'algoritmo, consentendo una facile interpretazione dei risultati
- Possono gestire feature con relazioni lineari e non lineari con l'output

Svantaggi:

- Sono instabili, il che significa che un piccolo cambiamento nei dati può portare a un grande cambiamento nella struttura dell'albero decisionale ottimale
- Sono spesso relativamente imprecisi. Molti altri predittori funzionano meglio con dati simili

La regressione predice un valore numerico specifico.

Determinano il valore di una variabile continua in base alle feature di input.

[Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 111-124](#)

6.9.7 Support Vector Machines

[Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 125-131](#)

6.9.8 Fuzzy rules based systems

[Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 131-136](#)

6.9.9 Algoritmi di regressione

6.9.9.1 Regressione lineare

- Assume che la relazione tra la variabile target e le variabili di input sia lineare
- La relazione comprende anche una variabile di errore, ovvero una variabile casuale non rilevata, che aggiunge rumore alla relazione lineare

Si assume che l'errore abbia una distribuzione normale con media 0 e varianza costante e quindi non cambi al variare dei valori delle feature di input. Sia:



Figure 37: Regressione lineare

- y il vettore che rappresenta la variabile target;
- X la matrice delle feature;

- β (Beta) il vettore dei parametri da stimare;
- ε (Epsilon) variabile casuale dell'errore.

$$y = X \cdot \beta + \varepsilon$$

6.9.9.2 Regressione logistica

Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 137-141

6.9.10 Algoritmi semi-supervisionati

Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 142-146

6.9.11 Algoritmi di clustering

Tipico problema non supervisionato

E' un insieme di metodi per raggruppare oggetti in classi omogenee.

Un cluster è un insieme di oggetti che presentano tra loro delle similarità, ma che presentano dissimilarità con oggetti in altri cluster.

L'input di un algoritmo di clustering è costituito da un campione di elementi, l'output è dato da un certo numero di cluster in cui gli elementi del campione sono suddivisi in base a una misura di similarità.

La cluster analysis è ampiamente utilizzata:

- Ricerche di mercato
- Riconoscimento di pattern
- Raggruppamento di clienti in base ai comportamenti d'acquisto
- Posizionamento dei prodotti
- Analisi dei social network, per il riconoscimento di community di utenti
- Identificazione degli outliers

Outliers

La loro identificazione può essere interessante per due scopi:

- L'eliminazione di questi valori anomali, che potrebbero essere causati da errori
- L'isolamento di questi casi che magari rivestono una certa importanza per il business

Algoritmo k-means:

1. Definiamo il numero k di cluster desiderati
2. Partizioniamo l'insieme in K cluster, assegnando a ciascuno di essi degli elementi scelti a caso
3. Calcoliamo i centroidi di ciascun cluster k con la formula in alto
4. Calcoliamo la distanza degli elementi del cluster dal centroide, ottenendo un errore quadratico, con la formula in basso
5. A questo punto si riassegnano gli elementi del campione in base al più vicino centroide
6. Si ripetono i passaggi 2, 3, 4 e 5 finché il valore minimo dell'errore totale non è raggiunto, oppure finché i membri dei cluster non si stabilizzano, oppure finché non si raggiunge un numero massimo di iterazioni, predefinito

L'algoritmo k-means è sensibile all'allocazione iniziale dei valori e, in base ad essa, potrebbe convergere a un minimo locale della funzione d'errore e non al minimo assoluto.

Sempre a causa dell'allocazione iniziale casuale, i risultati del k-means potrebbero essere diversi ad ogni esecuzione sugli stessi dati. Inoltre è molto sensibile al rumore nei dati e alla presenza di outliers.

6.9.12 Model ensembles

Approfondimento su slide "Intro_IA-32-Modelli Predittivi" pagine 163-174

6.9.13 Reti Neurali

Approfondimento su slide "Intro.IA-32-Modelli Predittivi" pagine 175-201

6.10 Il test e la valutazione dei modelli predittivi

Le modalità di valutazione sono diverse a seconda del tipo di algoritmo:

- I modelli di classificazione
- I modelli di regressione
- I modelli di clustering (utilizzate anche nei task di regressione).

6.10.1 Hold-out e cross validation

Si prende un numero finito k di parti (fold) di uguali dimensioni.

Per ogni subset si considera $k-1$ delle parti come set di addestramento e la parte rimanente come set di collaudo.

Quindi si calcoliamo una determinata metrica per ogni parte.

Alla fine si calcola la media dei risultati.



Figure 38: Holdout Strategy



Figure 39: K-fold Strategy

6.10.2 Cross validation e matrice di confusione

Dalla cross validation scaturiscono metriche di valutazione che consentono di effettuare la scelta dell'algoritmo o della parametrizzazione migliore.

La più utilizzata è la matrice di confusione che altro non è che una cross tabulazione delle classi reali e delle classi predette.



Figure 40: Matrice di confusione

6.10.2.1 Matrice di confusione

- Il quadrante (1,1), che indica quanti elementi della classe positiva sono stati correttamente individuati dall'algoritmo (veri positivi o True Positive o TP).
- Il quadrante (0,0), che indica quanti elementi della classe negativa sono stati correttamente individuati dall'algoritmo (veri negativi o True Negative o TN).

- Il quadrante (0,1), che indica quanti elementi della classe negativa reale sono stati posti dall'algoritmo nella classe positiva (falsi positivi o False Positive o FP).
- Il quadrante (1,0), che indica quanti elementi della classe positiva reale sono stati posti dall'algoritmo nella classe negativa (falsi negativi o False Negative o FN).

6.10.2.2 Matrice di confusione e metriche di valutazione del modello

- Accuracy: $\frac{(TP+TN)}{(P+N)}$
- Precision (o Positive Predictive Value): $\frac{TP}{(TP+FP)}$
- Sensitivity (o Recall o True Positive Rate): $\frac{TP}{(TP+FN)} = \frac{TP}{P}$
- Specificity (o True Negative Rate): $\frac{TN}{(TN+FP)} = \frac{TN}{N}$

Attenzione: L'accuracy è una misura che potrebbe essere fuorviante.

Per ottimizzare l'algoritmo è bene impiegare la metrica più adeguata al problema e cioè:

- L'**accuracy** quando desideriamo che la maggior parte degli elementi siano correttamente classificati, indipendentemente dalla produzione di falsi positivi o falsi negativi.
- La **sensitivity** quando vogliamo massimizzare i True Positive senza però far crescere troppo i False Positive.
- La **precision** quando vogliamo massimizzare i veri positivi e minimizzare i falsi negativi.
- La **specificity** quando occorre massimizzare il numero di veri negativi.

6.10.2.3 Matrice di confusione e F-measure

F-measure è una misura dell'accuratezza di un test e si deriva dalla matrice di confusione.

La misura tiene in considerazione precisione e recupero del test, dove la precisione è il numero di veri positivi diviso il numero di tutti i risultati positivi, mentre il recupero è il numero di veri positivi diviso il numero di tutti i test che sarebbero dovuti risultare positivi (ovvero veri positivi più falsi negativi).

$$F\ measure = \frac{2 \cdot sensitivity \cdot precision}{sensitivity + precision} = \frac{2|TP|}{2|TP| + |FP| + |FN|}$$

6.10.3 Valutazione dei modelli di clustering

La misura più comunemente utilizzata è lo scarto quadratico medio (**SSE - Sum of Squared Error**). Per ogni punto l'errore è la distanza dal centroide del cluster a cui esso è assegnato

$$SSE = \sum_{l=1}^K \sum_{x \in C_i} dist^2(m_l, x)$$

x è un punto appartenente al cluster C_i e m_i è il rappresentante del cluster C_i

$$m_i = \sum_{x \in C_i} x$$

Ovviamente il valore di SSE si riduce incrementando il numero dei cluster K .

7 Esami Passati

Gli esami sono generalmente composti da domande a risposta multipla, che però chiedono di motivare la risposta, di seguito saranno elencate le domande, con le relative "crocette", la risposta corretta sarà quella in grassetto, verrà inoltre fornita un eventuale motivazione della risposta.

7.1 Domanda 1

Una possibile definizione Intelligenza Artificiale generale (Artificial General Intelligence) si colloca in:

1. Sistemi orientati agli obiettivi, progettati per eseguire compiti singoli con abilità che talvolta superano quelle umane.
2. **Sistemi orientati allo sviluppo un modello di intelligenza artificiale, ispirata al funzionamento del cervello umano e pertanto in grado di trovare soluzioni in qualsiasi tipo di situazione, a prescindere dalla natura specifica del problema.**
3. Sistemi basati su modelli logici/matematici in grado di eseguire compiti il più velocemente possibile.

Motivare adeguatamente la risposta.

Risposta

L'Intelligenza Artificiale Generale (AGI) è un tipo di intelligenza artificiale che ha come obiettivo quello di sviluppare un sistema in grado di risolvere una vasta gamma di compiti simili a quelli che un essere umano può affrontare, senza limitarsi a compiti specifici.

In altre parole, l'AGI dovrebbe essere generalista, capace di adattarsi a qualsiasi tipo di problema, come farebbe una persona, applicando un ragionamento flessibile e una comprensione contestuale che non dipendano dalla specificità di un compito.

7.2 Domanda 2

Un agente razionale:

1. **Prende le decisioni sulla base delle conoscenze e dei dati che ha a disposizione al fine di ottenere il miglior risultato atteso tra quelli possibili.**
2. Esegue sempre la prima azione possibile perché l'importante è essere veloci nell'agire
3. Esegue l'azione più giusta secondo i principi della sua coscienza e sulla base dei dati a disposizione.

Motivare adeguatamente la risposta.

Risposta

Un agente razionale è definito in intelligenza artificiale come un'entità che agisce in modo da massimizzare le sue aspettative di successo, cioè le probabilità di ottenere il miglior risultato possibile, dato lo stato attuale delle sue conoscenze e delle risorse disponibili.

L'idea è che l'agente prende decisioni in modo razionale, in funzione delle informazioni che ha, delle sue capacità e dell'ambiente in cui opera.

L'agente non deve necessariamente agire immediatamente, ma deve pianificare la sua azione in modo da ottimizzare il risultato atteso.

7.3 Domanda 3

Il test di Turing è usato sin dagli inizi dell'Intelligenza artificiale per:

1. **Valutare sistemi che operano come gli esseri umani**
2. Valutare sistemi che pensano come gli esseri umani
3. Valutare sistemi che agiscono sempre in modo razionale

Motivare adeguatamente la risposta.

Risposta

Il test di Turing è stato pensato per valutare se una macchina è in grado di imitare il comportamento umano in modo tale che un osservatore umano non sia in grado di distinguere se sta interagendo con una macchina o con un altro essere umano.

Il test non si concentra su come la macchina "pensa" o sulla sua coscienza; piuttosto, si concentra sul comportamento osservabile, cioè se la macchina opera in modo simile a un essere umano

7.4 Domanda 4

Che cosa studia l'explainable AI (XAI)?

Risposta

Studia il motivo per cui è stata presa una decisione dall'AI in modo che i modelli di AI possano essere più interpretabili per gli utenti umani e consentire loro di capire perché il sistema è arrivato a una decisione specifica.

7.5 Domanda 5

Risoluzione di un problema tramite ricerca:

1. Spiegare come opera un algoritmo di ricerca di una soluzione in uno spazio degli stati, quale è il suo output
2. Descrivere quindi i principali algoritmi che adottano strategie non informate.

Queste strategie riescono a trovare soluzioni ottime e complete?

Risposta 1

Lo spazio degli stati è l'insieme di tutti gli stati raggiungibili dallo stato iniziale con una qualunque sequenza di operatori.

E' caratterizzato da:

- Uno stato iniziale in cui l'agente sa di trovarsi (non noto a priori);
- Un insieme di azioni possibili che sono disponibili da parte dell'agente (Operatori che trasformano uno stato in un altro);
- Un cammino è una sequenza di azioni che conduce da uno stato a un altro.

Il processo di ricerca segue questi passi

1. **Definizione dello spazio degli stati:** Lo spazio degli stati è una rappresentazione formale di tutte le possibili configurazioni di un problema, a partire da uno stato iniziale e arrivando a uno stato obiettivo.
2. **Espansione dei nodi:** L'algoritmo inizia dal nodo iniziale (stato iniziale) e esplora i nodi vicini (stati successivi) mediante azioni che portano a nuovi stati.
3. **Strategia di esplorazione:** A seconda dell'algoritmo, la ricerca esplora i nodi in ordine specifico.
4. **Condizione di terminazione:** L'algoritmo termina quando raggiunge uno stato che soddisfa la condizione di obiettivo (soluzione) o quando esaurisce lo spazio di ricerca.
5. **Output dell'algoritmo:** L'output di un algoritmo di ricerca è solitamente un cammino che collega lo stato iniziale allo stato obiettivo. Se il problema è di tipo di ottimizzazione, l'output potrebbe anche essere la soluzione ottimale.

Risposta 2

Non richiedono nessuna conoscenza specifica relativa al problema. Non hanno informazioni aggiuntive oltre a quelle già fornite dalla definizione del problema.

Possono solo generare successori e distinguere uno stato obiettivo da uno stato non obiettivo.

Tra i principali algoritmi non informati ci sono:

1. **Ricerca in ampiezza (Breadth-First Search, BFS):**
 - **Descrizione:** Esplora il grafo degli stati livello per livello, iniziando dal nodo iniziale e visitando tutti i nodi a una distanza di un passo prima di passare ai nodi a distanza maggiore.
 - **Proprietà:** È completo (trova una soluzione se esiste), ma non è sempre ottimale (se i costi per passaggio sono diversi, può non trovare la soluzione migliore).
2. **Ricerca in profondità (Depth-First Search, DFS):**
 - **Descrizione:** Esplora profondamente uno dei rami del grafo degli stati, prima di tornare indietro e esplorare gli altri rami.
 - **Proprietà:** È completo solo se lo spazio degli stati è finito. Non è garantito che trovi la soluzione ottimale, specialmente in spazi infiniti, e può entrare in loop infiniti se non è implementata correttamente.
3. **Ricerca in profondità limitata (Depth-Limited Search, DLS):**
 - **Descrizione:** È una variante della DFS con un limite di profondità per evitare la ricerca infinita.
 - **Proprietà:** È completo solo se esiste una soluzione entro il limite di profondità, ma può non essere ottimale.
4. **Ricerca con ritorno su stati precedenti (Iterative Deepening Search, IDS):**

- **Descrizione:** Combina la profondità limitata e la ricerca in ampiezza, eseguendo iterazioni della DFS con limiti di profondità crescenti.
- **Proprietà:** È completo e garantisce di trovare la soluzione ottimale (se tutte le azioni hanno lo stesso costo). Tuttavia, ha una complessità computazionale superiore rispetto alla ricerca in ampiezza, poiché riesegue le stesse operazioni più volte.

5. Ricerca Best-First (Best-First Search):

- **Descrizione:** È un algoritmo di ricerca che sceglie quale nodo esplorare in base a un criterio di "priorità", senza sapere se il nodo porterà effettivamente alla soluzione ottimale.
- **Proprietà:** Non è garantito che sia ottimale e potrebbe non essere completo, a meno che non si usino strategie specifiche di gestione dei nodi

Le strategie non informate di ricerca sono complete in molti casi (come la BFS e la IDS), ma non sono necessariamente ottimali.

Alcuni di questi algoritmi potrebbero non garantire di trovare la soluzione migliore se non sono applicati in condizioni specifiche (ad esempio, con costi uniformi tra le azioni)

7.6 Domanda 6

Nella progettazione di un agente logico, con "grounding di un problema" indichiamo:

1. **La connessione tra i processi di ragionamento logico e l'ambiente reale in cui esiste l'agente**
2. Il processo di inferenza corretto
3. La traduzione della conoscenza iniziale di un problema in un insieme di asserzioni e regole
4. La procedura con cui una proposizione A può essere derivata dalla KB base di conoscenza

Motivare adeguatamente la risposta.

Risposta

Nel contesto della progettazione di un agente logico, il termine "grounding" si riferisce al processo di collegamento delle rappresentazioni simboliche (come proposizioni o concetti) di un agente logico con oggetti concreti o stati del mondo reale. In altre parole, il grounding riguarda la connessione tra il linguaggio formale (come la logica) usato dall'agente e la realtà fisica o l'ambiente in cui l'agente agisce.

7.7 Domanda 7

Per la costruzione di un agente basato sulla conoscenza si utilizza:

1. Approccio procedurale
2. **Approccio dichiarativo**

Motivare adeguatamente la risposta

Risposta

Un agente basato sulla conoscenza sfrutta l'approccio dichiarativo perché il suo scopo è rappresentare e manipolare la conoscenza in modo esplicito, piuttosto che fornire una sequenza di passi operativi specifici come nell'approccio procedurale.

L'approccio dichiarativo permette all'agente di usare la sua base di conoscenza per ragionare e prendere decisioni in modo più flessibile e adattabile.

7.8 Domanda 8

Tradurre le seguenti frasi inglesi in logica proposizionale: Cercare di conservare il più possibile la struttura della frase:

1. Se e solo se Paolo gioca, Luca va a correre
2. La gara sarà finita quando sia Luca che Paolo avranno terminato 10Km di bici e 5 di corsa
3. Solo se Luca mette gli occhiali riuscirà a leggere quel cartello
4. Usciremo solo se non sarà buio

Risposta 1

$$P \leftrightarrow L$$

Risposta 2

$$(L_b \wedge L_c) \wedge (P_b \wedge P_c) \rightarrow G$$

Risposta 3

$$L_o \rightarrow L_c$$

Risposta 4

$$U \rightarrow \neg B$$

7.9 Domanda 9

Una possibile definizione di sistemi di Intelligenza Artificiale debole si colloca in:

1. **Sistemi orientati agli obiettivi, progettati per eseguire compiti singoli con abilità che talvolta superano quelle umane.**
2. Sistemi orientati allo sviluppo un modello di intelligenza artificiale, ispirata al funzionamento del cervello umano e pertanto in grado di trovare soluzioni in qualsiasi tipo di situazione, a prescindere dalla natura specifica del problema.
3. Sistemi basati su modelli logici/matematici in grado di eseguire compiti il più velocemente possibile.

Motivare adeguatamente la risposta.

Risposta

L'Intelligenza Artificiale debole (o IA debole) si riferisce a sistemi di intelligenza artificiale progettati per svolgere compiti specifici e ben definiti, come riconoscere oggetti, tradurre lingue o giocare a giochi.

Questi sistemi non sono dotati di consapevolezza o comprensione, ma sono altamente specializzati nel risolvere compiti particolari, spesso con prestazioni che possono superare quelle umane in situazioni limitate e ben definite.

7.10 Domanda 10

Un agente risolutore di problemi ricerca la soluzione di un problema:

1. Mentre attua una mossa possibile per le sue capacità dallo stato corrente
2. Sulla base del solo stato obiettivo indipendentemente dalla conoscenza delle sue possibilità
3. **Selezionando quella sequenza di azioni possibili per l'agente stesso che porta all'obiettivo dal suo stato corrente**

Motivare adeguatamente la risposta

Risposta

Quando un agente affronta un problema, deve identificare una sequenza di azioni che lo porti dallo stato iniziale allo stato obiettivo.

Questo richiede che l'agente esplori lo spazio degli stati possibili, selezionando azioni appropriate ad ogni passo per progredire verso l'obiettivo. Il processo di ricerca di una soluzione implica la valutazione delle possibilità di azione in ogni stato e la determinazione di un percorso che colleghi lo stato iniziale a quello finale.

7.11 Domanda 11

Un chatbot di AI supera il test di Turing se

1. Risponde a domande su diversi argomenti anche difficili
2. **Imitare le risposte umane in modo ingannevolmente e realistico**
3. Durante la valutazione adotta un comportamento freddo e calcolatore

Motivare adeguatamente la risposta.

Risposta

La capacità di imitare le risposte umane in modo ingannevole e realistico è fondamentale per superare il test. Secondo Turing, un chatbot che imita in modo convincente il comportamento umano, a tal punto che il giudice non riesce a distinguere tra una macchina e un umano, può essere considerato in grado di "pensare" o di "simulare" l'intelligenza umana.

7.12 Domanda 12

Che cosa si intende per “AI black boxes”?

Risposta

I sistemi Machine Learning spesso funzionano come una scatola nera, che fa previsioni e decisioni come fanno gli umani, ma senza essere in grado di comunicare le ragioni per farlo.

7.13 Domanda 13

Risoluzione di un problema tramite ricerca:

1. Illustrare cosa rappresenta uno spazio degli stati e che cosa la ricerca restituisce come soluzione.
2. Spiegare come operano gli algoritmi con strategie informate descrivendo in modo particolare le caratteristiche di una funzione di valutazione

Risposta 1

Lo spazio degli stati è l'insieme di tutti gli stati raggiungibili dallo stato iniziale con una qualunque sequenza di operatori.

E' caratterizzato da:

- Uno stato iniziale in cui l'agente sa di trovarsi (non noto a priori);
- Un insieme di azioni possibili che sono disponibili da parte dell'agente (Operatori che trasformano uno stato in un altro);
- Un cammino è una sequenza di azioni che conduce da uno stato a un altro.

Il processo di ricerca segue questi passi:

1. **Definizione dello spazio degli stati:** Lo spazio degli stati è una rappresentazione formale di tutte le possibili configurazioni di un problema, a partire da uno stato iniziale e arrivando a uno stato obiettivo.
2. **Espansione dei nodi:** L'algoritmo inizia dal nodo iniziale (stato iniziale) e esplora i nodi vicini (stati successivi) mediante azioni che portano a nuovi stati.
3. **Strategia di esplorazione:** A seconda dell'algoritmo, la ricerca esplora i nodi in ordine specifico.
4. **Condizione di terminazione:** L'algoritmo termina quando raggiunge uno stato che soddisfa la condizione di obiettivo (soluzione) o quando esaurisce lo spazio di ricerca.
5. **Output dell'algoritmo:** L'output di un algoritmo di ricerca è solitamente un cammino che collega lo stato iniziale allo stato obiettivo. Se il problema è di tipo di ottimizzazione, l'output potrebbe anche essere la soluzione ottimale.

Risposta 2

Gli algoritmi di ricerca con strategie informate (o euristiche) sono utilizzati per risolvere problemi in modo più efficiente rispetto agli algoritmi di ricerca senza informazione (come la ricerca esaustiva, ad esempio, la ricerca a forza bruta o la ricerca in ampiezza).

Questi algoritmi utilizzano informazioni aggiuntive, generalmente sotto forma di una funzione di valutazione, per guidare la ricerca verso soluzioni promettenti.

- **Euristica:** Fornisce una stima del costo o della distanza rimanente.
- **Direzionalità:** Aiuta a indirizzare la ricerca verso l'obiettivo.
- **Ammissibilità:** Se usata in un algoritmo come A*, deve essere ammissibile per garantire l'ottimalità.
- **Efficienza:** Una buona funzione di valutazione deve essere veloce da calcolare e non comportare un onere computazionale significativo.

7.14 Domanda 14

Nella progettazione di un agente logico, con “formalizzazione di un problema” indichiamo:

1. La connessione tra i processi di ragionamento logico e l’ambiente reale in cui esiste l’agente
2. Il processo di inferenza corretto
3. **La traduzione della conoscenza iniziale di un problema in un insieme di asserzioni e regole**
4. La procedura con cui una proposizione A può essere derivata dalla KB base di conoscenza

Motivare adeguatamente la risposta.

Risposta

La formalizzazione di un problema implica la traduzione della conoscenza (spesso descritta in linguaggio naturale) in una rappresentazione logica formale, che può includere:

- **Asserzioni** (fatti o dichiarazioni che descrivono lo stato del mondo o le relazioni tra gli oggetti).
- **Regole** (prescrizioni logiche su come la conoscenza può essere manipolata o come nuovi fatti possono essere derivati).

Questa formalizzazione è fondamentale affinché l’agente possa eseguire il ragionamento logico in modo sistematico e corretto, utilizzando la sua base di conoscenza per risolvere il problema.

7.15 Domanda 15

Per la costruzione di un agente logico si utilizza:

1. Approccio procedurale
2. **Approccio dichiarativo**

Motivare adeguatamente la risposta.

Risposta

Questo approccio consente di rappresentare la conoscenza in modo formale, in termini di logica e regole, e di dedurre nuove informazioni tramite inferenza, senza la necessità di specificare dettagliatamente il controllo e le procedure.

7.16 Domanda 16

Tradurre le seguenti frasi inglesi in logica proposizionale: Cercare di conservare il più possibile la struttura della frase

1. Chiara riesce a lavorare se Marta studia,
2. Qualora Chiara si sia in vacanza al mare e Paola sia in vacanza in montagna, la casa in città rimarrà vuota
3. Chiara è amica di Marta e viceversa
4. Sole se non pioverà, arriveremo in tempo.

Risposta 1

$S \rightarrow L$ (Se Marta studia, allora Chiara riesce a lavorare)

Risposta 2

$(M \wedge P) \rightarrow V$ (Se Chiara è in vacanza al mare e Paola è in vacanza in montagna, allora la casa in città rimarrà vuota)

Risposta 3

$A \wedge B$ (Chiara è amica di Marta e Marta è amica di Chiara)

Risposta 4

$\neg R \rightarrow T$ (Se non piove, allora arriveremo in tempo)

7.17 Domanda 17

Tradurre in logica del primo ordine le seguenti affermazioni, commentandone i passi:

1. Le rane sono tutte verdi
2. Alcune rane non sono verdi
3. Nessuno lettore legge tutti i libri
4. In una biblioteca, esiste almeno un libro di storia

Risposta 1

$\forall x (Rana(x) \rightarrow Verde(x))$

Risposta 2

$\exists x (Rana(x) \wedge \neg Verde(x))$

Risposta 3

$\neg \exists x (Lettore(x) \wedge \forall y (Libro(y) \rightarrow Legge(x, y)))$

Risposta 4

$\exists x (Libro(x) \wedge Storia(x))$

7.18 Domanda 18

In un processo di Data Science si deve estrarre conoscenza dai dati utili per un certo obiettivo. Dato l'obiettivo che si vuole ottenere quali sono i passi che si devono fare per estrarre la conoscenza?

Risposta

I cinque passi per estrarre conoscenza dai dati in un processo di Data Science sono:

1. **Porre una domanda interessante:** Identificare il problema o l'obiettivo da raggiungere.
2. **Ottenere i dati:** Raccogliere o acquisire dati rilevanti per il problema in esame.
3. **Esplorare i dati:** Analizzare i dati per capire meglio la loro struttura, rilevanza e qualità.
4. **Creare un modello per i dati:** Applicare metodi di modellazione per rappresentare le relazioni nei dati.
5. **Comunicare e presentare i risultati:** Condividere i risultati e le conclusioni ottenute in modo chiaro e comprensibile.

7.19 Domanda 19

Dopo aver illustrato cosa sono gli ingressi e le uscite di un modello di ML, spiegare la differenza un modello di clustering ed uno di classificazione. Si saprebbe descrivere un algoritmo di classificazione?

Risposta

Ingressi e uscite di un modello di ML:

- Gli ingressi (input) sono i dati grezzi o le caratteristiche rilevanti che vengono forniti al modello per apprendere o fare previsioni. Ad esempio, nel caso della previsione del prezzo di una casa, gli ingressi possono includere la metratura, il numero di stanze, la posizione geografica, ecc.
- Le uscite (output) rappresentano i risultati generati dal modello dopo aver elaborato gli ingressi. Possono essere valori continui (per modelli di regressione) o categorie (per modelli di classificazione). Ad esempio, nel caso precedente, l'uscita sarebbe il prezzo previsto della casa.

Differenza tra modelli di clustering e classificazione:

- Un modello di clustering suddivide i dati in gruppi basati su somiglianze senza conoscere le categorie a priori.
È un approccio non supervisionato.
- Un modello di classificazione, invece, assegna etichette predefinite ai dati in base a un processo supervisionato, dove il modello apprende dalle coppie di ingressi e uscite fornite durante l'addestramento.

Esempio di algoritmo di classificazione:

Un esempio è l'albero di decisione, che utilizza una struttura gerarchica per classificare i dati attraverso una serie di regole logiche; in cui ogni nodo rappresenta una variabile e ogni ramo un possibile valore della variabile, conducendo a una decisione finale (le foglie).

7.20 Domanda 20

Quando si dice che un modello di ML è andato in overfitting? Ce ne possiamo accorgere durante l'addestramento? E quando un modello di ML è in underfitting?

Risposta

Un modello è in overfitting quando si adatta troppo ai dati di addestramento, perdendo capacità di generalizzazione ed è rilevabile confrontando prestazioni su addestramento e test.

È in underfitting quando non apprende sufficientemente dai dati, mostrando basse prestazioni su entrambi i set.

7.21 Domanda 21

Tradurre in logica del primo ordine le seguenti affermazioni, commentandone i passi:

1. Le farfalle sono colorate
2. Alcune farfalle non sono rosse.
3. Nessuno poliziotto arresta tutti i ladri
4. In una pattuglia, esiste almeno un poliziotto graduato.

Risposta 1

$\forall x(Farfalla(x) \rightarrow Colorata(x))$

Risposta 2

$\exists x(Farfalla(x) \wedge \neg Rossa(x))$

Risposta 3

$\neg \exists x(Poliziotto(x) \wedge \forall y(Ladro(y) \rightarrow Arresta(x, y)))$

Risposta 4

$\exists x(Poliziotto(x) \wedge Graduato(x))$

7.22 Domanda 22

In un processo di Data Science, la fase della preparazione dei dati richiede in genere molta attenzione e tempo. Si descriva che cosa si intende per normalizzazione dei dati, selezione delle features e gestione dei dati mancanti.

Risposta

- Normalizzazione dei dati: Porta i dati su una scala comune, migliorando la precisione del modello.
- Selezione delle features: Determina le variabili più rilevanti per il modello, riducendo la complessità.
- Gestione dei dati mancanti: Possono essere trattati imputando valori mancanti (mettere dei valori plausibili al loro posto) o eliminando i dati incompleti.

7.23 Domanda 23

Dopo aver illustrato cosa sono gli ingressi e le uscite di un modello di ML, spiegare la differenza un modello di regressione ed uno di classificazione.

Si saprebbe descrivere un algoritmo di classificazione?

Risposta

Ingressi e uscite di un modello di ML:

- Gli ingressi (input) sono i dati grezzi o le caratteristiche rilevanti che vengono forniti al modello per apprendere o fare previsioni. Ad esempio, nel caso della previsione del prezzo di una casa, gli ingressi possono includere la metratura, il numero di stanze, la posizione geografica, ecc.
- Le uscite (output) rappresentano i risultati generati dal modello dopo aver elaborato gli ingressi. Possono essere valori continui (per modelli di regressione) o categorie (per modelli di classificazione). Ad esempio, nel caso precedente, l'uscita sarebbe il prezzo previsto della casa.

Differenza tra modelli di regressione e classificazione:

- Un modello di regressione ha come output un valore numerico, che si vuole approssimare tramite una funzione dei dati di input, la variabile di output è continua e può assumere un numero infinito di valori.
- Un modello di classificazione, invece, assegna etichette predefinite ai dati in base a un processo supervisionato, dove il modello apprende dalle coppie di ingressi e uscite fornite durante l'addestramento.

Esempio di algoritmo di classificazione:

Un esempio è l'albero di decisione, che utilizza una struttura gerarchica per classificare i dati attraverso una serie di regole logiche; in cui ogni nodo rappresenta una variabile e ogni ramo un possibile valore della variabile, conducendo a una decisione finale (le foglie).

7.24 Domanda 24

Nella validazione di un task di classificazione si fa riferimento spesso al parametro accuratezza. Cosa rappresenta e quando può essere una misura ingannevole.

Risposta

L'accuratezza è una metrica che misura la percentuale di previsioni corrette effettuate da un modello di classificazione rispetto al totale delle previsioni e si calcola dividendo il numero di previsioni corrette per il numero totale di previsioni effettuate.

Ad esempio, se un modello classifica correttamente 90 elementi su 100, l'accuratezza sarà del 90%.

L'accuratezza può risultare ingannevole nei dataset squilibrati, dove una classe è rappresentata in modo molto maggiore rispetto alle altre; ad esempio, se il 90% dei dati appartiene a una sola classe e il modello predice sempre quella classe, l'accuratezza sarà del 90%, ma il modello non avrà imparato a distinguere correttamente le classi, in questi casi, metriche come la precisione, il recall e il punteggio F1 sono più utili per valutare le reali prestazioni del modello.

7.25 Domanda 25

Una possibile definizione Intelligenza Artificiale debole si colloca in:

1. **Sistemi che operano in alcuni campi come se fossero intelligenti**
2. Sistemi che pensano come gli esseri umani
3. Sistemi che agiscono razionalmente

Motivare brevemente la risposta.

Risposta

L'intelligenza artificiale debole si riferisce a sistemi che simulano l'intelligenza umana in compiti specifici, senza possedere una vera consapevolezza o coscienza.

Questi sistemi agiscono in modo intelligente solo in determinati contesti, ma non possiedono una comprensione generale del mondo, come potrebbe fare un'intelligenza artificiale forte

7.26 Domanda 26

Un agente razionale:

1. **Esegue sempre l'azione corretta in funzione del contesto e delle sue possibilità**
2. Esegue la prima azione possibile
3. Esegue l'azione più giusta secondo i suoi principi di etica

Motivare brevemente la risposta.

Risposta

Un agente razionale è un'entità che compie azioni che massimizzano il raggiungimento dei suoi obiettivi, tenendo conto dell'ambiente e delle sue capacità.

Non è solo un agente che agisce senza riflettere, ma uno che seleziona la miglior azione in base al contesto.

7.27 Domanda 27

Problem solving: illustrare cosa fa l'algoritmo di ricerca di una soluzione spiegando che cosa è una strategia di ricerca e i nodi frontiera e come sono relazionati fra di loro

Risposta

Un algoritmo di ricerca di una soluzione esplora lo spazio delle possibili soluzioni partendo dallo stato iniziale fino a raggiungere lo stato finale.

La strategia di ricerca è la tecnica utilizzata per selezionare quale stato esplorare successivamente.

La strategia determina l'ordine in cui i nodi vengono esplorati e può essere basata su costi, profondità, o altri criteri.

I nodi frontiera sono gli stati che sono pronti per essere esplorati (non ancora visitati).

Ogni nodo nella frontiera rappresenta un'opportunità per espandere la ricerca. Man mano che i nodi vengono esplorati e si trovano soluzioni, i nodi vengono rimossi dalla frontiera e nuovi nodi vengono aggiunti.

7.28 Domanda 28

Che cosa rappresenta la funzione di valutazione nelle strategie informate?

Fornire un esempio di algoritmo con una strategia informata.

Risposta

La funzione di valutazione in una strategia informata (come la ricerca A^*) è una funzione che stima quanto un dato stato sia vicino alla soluzione finale, combinando il costo per arrivare a quello stato e una stima del costo rimanente per raggiungere la soluzione.

Esempio:

L'algoritmo di ricerca A^* utilizza una funzione di valutazione $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo per arrivare al nodo n , e $h(n)$ è una stima del costo per arrivare alla soluzione a partire da n .

7.29 Domanda 29

Spiegare che significa, nella progettazione di un agente logico, la seguente affermazione:

Specialized algorithm = specialized knowledge + general-purpose reasoning

Risposta

L'affermazione implica che un algoritmo specializzato combina due componenti:

- Conoscenza specializzata, che è il sapere concreto e mirato riguardante un dominio specifico (ad esempio, conoscenze su un particolare campo o problema);
- Ragionamento di uso generale, che è un approccio logico per trattare e manipolare tale conoscenza, applicabile in vari contesti

7.30 Domanda 30

La logica proposizionale è una logica

1. Classica
2. Probabilistica
3. Temporale

Motivare brevemente la risposta

Risposta

La logica proposizionale è una logica classica in cui le proposizioni sono trattate come entità che possono essere vere o false, senza considerare probabilità, tempo o altre variabili.

7.31 Domanda 31

Tradurre in logica del primo ordine le seguenti affermazioni, commentandone i passi:

1. Per ogni fetta di torta, c'è qualcuno che la mangia
2. Esiste almeno una torta che non è di colore bianco

Risposta 1

- **Traduzione:** $\forall x(FettaDiTorta(x) \rightarrow \exists y(Persona(y) \wedge Mangia(y, x)))$
- **Commento:** Per ogni fetta di torta x , esiste qualcuno y (una persona) che mangia quella fetta.

Risposta 2

- **Traduzione:** $\exists x(Torta(x) \wedge \neg ColoreBianco(x))$
- **Commento:** Esiste almeno un oggetto x che è una torta e non ha colore bianco.

7.32 Domanda 32

Che ruolo ha la conoscenza in un processo di analisi di Business Intelligence

Risposta

La conoscenza è fondamentale in un processo di Business Intelligence (BI), poiché consente di trasformare i dati grezzi in informazioni utili per prendere decisioni strategiche.

La BI si basa su dati storici, analisi predittive e reportistica per generare insight che supportano l'efficacia decisionale e la gestione del business.

7.33 Domadna 33

Spiegare la differenza fra un modello di ML per il clustering e uno per la classificazione

Risposta

- Clustering: È un tipo di apprendimento non supervisionato dove l'obiettivo è raggruppare dati simili in cluster. Non sono disponibili etichette per il training, quindi l'algoritmo cerca di identificare delle strutture nei dati (ad esempio, K-means).
- Classificazione: È un tipo di apprendimento supervisionato in cui l'algoritmo apprende da un set di dati etichettati per poi assegnare etichette a nuovi dati (ad esempio, alberi decisionali, SVM)

7.34 Domanda 34

Nella matrice di confusione relativo ad un caso di classificazione binaria cosa rappresentano gli elementi definiti come falsi positivi e falsi negativi

Risposta

- Falsi positivi (FP): Sono i casi in cui il modello ha erroneamente classificato una classe negativa come positiva.
- Falsi negativi (FN): Sono i casi in cui il modello ha erroneamente classificato una classe positiva come negativa.

Questi due tipi di errori sono importanti per calcolare metriche come la precisione, il richiamo e l'accuratezza.

7.35 Domanda 35

Una possibile definizione Intelligenza Artificiale forte si colloca in:

1. Sistemi che imitano gli esseri umani
2. **Sistemi che pensano e sono in grado di prendere decisioni in modo autonomo**
3. Sistemi che agiscono razionalmente

Motivare brevemente la risposta

Risposta

L'Intelligenza Artificiale forte implica che il sistema non solo simuli l'intelligenza umana, ma possieda anche una vera capacità di pensiero e consapevolezza.

Questi sistemi sono in grado di prendere decisioni autonome, come gli esseri umani, comprendendo e ragionando sul mondo che li circonda

7.36 Domanda 36

Un agente razionale:

1. **Esegue sempre l'azione corretta in funzione del contesto e delle sue possibilità**
2. Esegue la prima azione possibile
3. Esegue l'azione più giusta secondo i suoi principi di etica

Motivare brevemente la risposta

Risposta

Un agente razionale prende decisioni che massimizzano la probabilità di raggiungere il suo obiettivo, considerando il contesto e le risorse disponibili.

Non si limita a scegliere l'azione prima disponibile, né a seguire principi etici, ma seleziona sempre l'azione che porta al miglior risultato possibile in base alle sue conoscenze e capacità.

7.37 Domanda 37

Problem solving: illustrare cosa rappresenta uno spazio degli stati e il ruolo degli operatori nella ricerca e costruzione di una soluzione.

Risposta

Lo spazio degli stati è l'insieme di tutte le possibili configurazioni che un sistema può assumere durante il processo di risoluzione di un problema.

Ogni stato rappresenta una possibile situazione intermedia, mentre la soluzione è raggiunta quando si arriva allo stato finale desiderato.

Gli operatori sono le azioni che possono essere applicate per passare da uno stato a un altro.

Ogni operazione modifica lo stato attuale in un nuovo stato, e il loro insieme consente di esplorare lo spazio degli stati alla ricerca della soluzione.

Gli operatori sono quindi fondamentali per costruire la sequenza di azioni necessarie per risolvere un problema.

7.38 Domanda 38

Parlare brevemente del ruolo dell'euristica nelle strategie informate.

Fornire un esempio di algoritmo con una strategia informata.

Risposta

L'euristica è una funzione che guida la ricerca in modo da esplorare più rapidamente le soluzioni promettenti e ridurre il numero di stati da esaminare.

Nelle strategie informate, l'euristica fornisce una stima di quanto un dato stato sia vicino alla soluzione finale, ottimizzando il processo di ricerca.

Esempio di algoritmo:

L'algoritmo di ricerca A^* utilizza una funzione euristica per stimare il costo rimanente di un percorso, combinandola con il costo già sostenuto, per determinare la sequenza di azioni migliore da intraprendere per arrivare alla soluzione.

7.39 Domanda 39

Dovendo progettare un agente logico spiegare brevemente cosa significa tenere separato il motore inferenziale dalla base di conoscenza specifica e che vantaggi si ottengono.

Risposta

Separare il motore inferenziale dalla base di conoscenza permette di creare un agente più flessibile e riutilizzabile. Il motore inferenziale è responsabile della deduzione delle conclusioni a partire dalle informazioni, mentre la base di conoscenza contiene i fatti e le regole su cui si basa il ragionamento.

Questo approccio consente di aggiornare o sostituire la base di conoscenza senza dover modificare il motore inferenziale, migliorando così la manutenibilità e l'adattabilità dell'agente.

7.40 Domanda 40

La logica del primo ordine è una logica:

1. Probabilistica
2. Classica
3. Temporale

Motivare brevemente la risposta

Risposta

La logica del primo ordine è una logica classica perché si basa su principi deterministici e dichiara che ogni affermazione è vera o falsa.

In questa logica, si usano predicati, funzioni e quantificatori per esprimere proposizioni più complesse rispetto alla logica proposizionale

7.41 Domanda 41

Tradurre in logica del primo ordine le seguenti affermazioni, commentandone i passi:

1. C'è una torta che è mangiata da tutti
2. Per ogni fetta di torta c'è un piattino e una forchetta

Risposta 1

- Traduzione: $\exists x(\text{Torta}(x) \wedge \forall y(\text{Persona}(y) \rightarrow \text{Mangia}(y, x)))$
- Commento: Esiste un oggetto x che è una torta e per ogni y che è una persona, y mangia x .

Risposta 2

- Traduzione: $\forall x(\text{FettaDiTorta}(x) \rightarrow \exists y \exists z(\text{Piattino}(y) \wedge \text{Forchetta}(z) \wedge \text{Usa}(y, x) \wedge \text{Usa}(z, x)))$
- Commento: Per ogni fetta di torta x , esistono un piattino y e una forchetta z tali che il piattino y e la forchetta z sono usati per quella fetta.

7.42 Domanda 42

Su che dati e a che cosa vuole ottenere un processo di Business Intelligence in una azienda

Risposta

Un processo di Business Intelligence raccoglie e analizza dati provenienti da varie fonti aziendali, come transazioni di vendita, dati finanziari, performance dei dipendenti e feedback dei clienti.

L'obiettivo principale è ottenere informazioni utili che supportano la presa di decisioni strategiche, migliorando l'efficienza operativa, individuando tendenze di mercato, e ottimizzando le risorse aziendali.

7.43 Domanda 43

Spiegare la differenza fra l'uscita di un algoritmo di regressione ed uno di classificazione.

Risposta

- Regressione: L'uscita di un algoritmo di regressione è un valore continuo.
Questo tipo di algoritmo viene utilizzato per prevedere variabili quantitative, come ad esempio il prezzo di una casa o la temperatura di una città.
- Classificazione: L'uscita di un algoritmo di classificazione è un valore discreto o un'etichetta che appartiene a una classe definita.
Questo tipo di algoritmo è utilizzato per assegnare un'osservazione a una delle categorie predefinite, come "spam" o "non spam" in un filtro di posta elettronica

7.44 Domanda 44

Nella validazione di un task di classificazione si fa riferimento spesso al parametro accuratezza.

Cosa rappresenta e quando può essere una misura ingannevole

Risposta

L'accuratezza rappresenta la percentuale di predizioni corrette rispetto al totale delle predizioni effettuate.

È una misura utile quando le classi sono bilanciate.

Tuttavia, può essere ingannevole quando c'è uno squilibrio nelle classi (ad esempio, in un dataset con molte più osservazioni di una classe rispetto all'altra), poiché un modello potrebbe semplicemente prevedere sempre la classe maggioritaria ottenendo comunque un'accuratezza elevata, senza fare previsioni corrette per la classe minoritaria.

In questi casi, altre metriche come precisione, richiamo o F1-score sono più indicative delle performance del modello.